Bayesian latent class models for prevalence estimation and diagnostic test evaluation

Simon Dufour and Juan Carlos Arango Sabogal

2026-03-26

Contents

1 About this document **7**

1.1 Software and libraries used 7

1.2 Some notation 10

2 Distributions **13**

2.1 Prior distributions 13

2.2 Distributions for likelihood functions 19

3 Running Bayesian models **23**

3.1 The data 23

3.2 The model 24

3.3 Generating initial values 28

3.4 Running a model in JAGS 29

3.5 Model diagnostic 30

3.6 Getting our estimates 35

4 Exercise 1 - Proportions **39**

4.1 Data 39

4.2 Questions 40

4.3 Answers 42

5 Two imperfect tests **47**

5.1 Assuming conditional independence 47

5.2 Allowing for conditional dependence 53

6 Exercise 2 - Two tests **59**

6.1 Questions 59

6.2 Answers 60

7 Multiple populations **65**

7.1 Likelihood function with 2 populations 66

7.2 Organizing the data with 2 populations 67

7.3 The JAGS model with 2 populations 67

7.4 Conditional dependence 71

7.5 Hierarchical latent class models 75

8 Exercise 3 - Multiple populations **91**

8.1 Questions 91

8.2 Answers 92

9 Multiple diagnostic tests **95**

9.1 Likelihood function with 3 tests 96

9.2 Organizing the data with 3 tests 97

9.3 The JAGS model with 3 tests 97

9.4 Conditional dependence 100

10 Exercise 4 - Multiple tests **105**

10.1 Questions 105

10.2 Answers 106

11 Accuracy varying with covariate **109**

11.1 Likelihood function when accuracy varies as function of covariate 110

11.2 Organizing the data when accuracy varies as function of covariate 110

11.3 The JAGS model when accuracy varies as function of covariate . 111

12 Exercise 5 - Accuracy varying with covariate **115**

12.1 Questions 115

12.2 Answers 116

13 Extensions **119**

14 Sensitivity analyses **121**

14.1 Model's assumptions 121

14.2 Choice of priors 122

15 Sample size **127**

15.1 Method when comparing to a gold standard 127

15.2 Method for the Hui-Walter BLCM 130

16 Exercise 6 - Sample size estimation and study design **137**

16.1 Questions 137

16.2 Answers 138



Chapter 1

About this document

In this document you will find indications on how to use various R libraries and functions for estimating true disease prevalence and accuracy of diagnostic tests in a Bayesian framework, along with various exercises. We suppose that you do have some basic R skills. If you have not worked with R before or feel a bit rusty, here are some resources to help you to prepare:

- Chapters 1, 2, and 4 of the CodeCademy "Learn R" course will provide a good overview of the basic concepts required for this workshop.
- If you are familiar with R and want to do some further reading, Hadley Wickham's "R for Data Science" is a great resource.

Remember, there are often many different ways to conduct a given piece of work in R. Throughout this document, we tried to stick with the simpler approaches (e.g., the shortest code, the minimal number of R libraries).

1.1 Software and libraries used

To conduct the exercises from this book, we will need to install three software (R, RStudio, and JAGS) and a few R packages. The installation steps are described below. These instructions are reproduced in parts from R-project.org, Rstudio.com, and various R packages' manuals.

1.1.1 R download

R is a language and environment for statistical computing and graphics. R provides a wide variety of statistical (linear and nonlinear modelling, classical

statistical tests, time-series analysis, classification, clustering, …) and graphical techniques, and is highly extensible.

R can be downloaded here. You can choose in the list a CRAN mirror close to your institution; for instance, in Canada we use the mirror from Simon Fraser University, Burnaby.

We used the 4.4.1 version of R to develop this book.

1.1.2 RStudio download

RStudio is an integrated development environment (IDE) for R. It includes a console, syntax-highlighting editor that supports direct code execution, as well as tools for plotting, history, debugging and workspace management.

RStudio is available in open source and commercial editions and runs on the desktop (Windows, Mac, and Linux) or in a browser connected to RStudio Server or RStudio Workbench.

A free RStudio Desktop Open Source License can be downloaded here.

We used the 2024.04.2+764 version of RStudio to develop this book.

1.1.3 R libraries

To install a given R library, open RStudio, click Packages, choose load package, choose the package you need (e.g., epiR).

For the workshop, the following R packages will be used:

1.1.3.1 R2jags

Version 0.8-5 R2jags manual The R2JAGS package provides convenient functions to call rjags (see next) and JAGS (a software specialized in Bayesian analyses; see below) from R. It automatically writes the data and scripts in a format readable by JAGS. After the JAGS process has finished, it is possible to read the resulting data into R using the facilities of the coda and mcmcplots packages for further analyses of the output.

1.1.3.2 rjags

Version 4-15 rjags manual The rjags package provides an interface from R to the JAGS library for Bayesian data analysis. JAGS uses Markov Chain Monte Carlo (**MCMC**) to generate a sequence of dependent samples from the posterior distribution of the parameters.

1.1.3.3 coda

Version 0.19-4.1 coda manual
The `coda` package provides functions for summarizing and plotting the output from MCMC simulations, as well as diagnostic tests of convergence to the equilibrium distribution of the Markov chain.

1.1.3.4 mcmcplots

Version 0.4.3 mcmcplots manual
The `mcmcplots` package provides a function (`mcmcplot`) that produces common MCMC diagnostic plots in an html file that can be viewed from a web browser. When viewed in a web browser, hundreds of MCMC plots can be viewed efficiently by scrolling through the output as if it were any typical web page.

1.1.3.5 epiR

Version 2.0.75 epiR manual
The `epiR` package contains tools for the analysis of epidemiological and surveillance data. For the workshop we will mainly use `epiR` functions that can return shape parameters for different distributions, based on expert elicitation, to produce and visualize prior distributions.

1.1.4 JAGS

`JAGS` is a free software package for performing Bayesian inference Using Gibbs Sampling. `JAGS` stands for "Just Another Gibbs Sampler". It is a program for analysis of Bayesian models using MCMC simulation.

You will have to install `JAGS` on your computer for the `R2JAGS` library to work. `JAGS` can be downloaded here.

- Click on the "Download latest version JAGS-4.3.1.html" green button;
- Wait a few seconds;
- Open up the JAGS-4.3.1.html file that was downloaded;
- Pick the appropriate installation depending on the version of `R` that is installed on your computer;
  - If you have installed `R` version 4.2.0 or later, then JAGS-4.3.1 can be installed.

1.2 Some notation

Throughout the document, you will find examples of R code along with comments. **The R code used always appear in grey boxes** (see the following example). This is the code that you will be able to use for your own analyses. **Lines that are preceded by a # sign are comments**, they are skipped when R reads them. Following each grey box with R code, another **grey box with results from the analysis is presented**.

For instance, this is a R code where I am simply asking to show main descriptive statistics for the *speed* variable of the *cars* dataset (note that this dataset is already part of R).

```
#This is a comment. R will ignore this line

#The summary() function can be use to ask for a summary of various R objects. For a va
summary(cars$speed)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     4.0    12.0    15.0    15.4    19.0    25.0
```

Throughout this document we will use:
- Italics for datasets or variables. For instance, the *speed* variable of the dataset *cars*.
- Shaded boxes for R libraries (e.g., `episensr`) and functions (e.g., `summary()`).

In R we can first call a given library and then use the functions related to this library or we can type the name of the library followed by `::` and then the function. For instance the two following pieces of code are equivalent:

```
library(ggplot2)
ggplot(data=cars, mapping=(aes(x=speed)))+
  geom_histogram()

##OR##

ggplot2::ggplot(data=cars, mapping=(aes(x=speed)))+
  geom_histogram()
```

The latter may improve reproducibility, but at the expense of longer codes. Throughout the document, we will always first call the library and then run the functions to keep codes short.

One last thing, when using a given function, it is not mandatory to name all the arguments, as long as they are presented in the sequence expected by this function. For instance, the `ggplot()` function that we used in the previous

chunk of code is expecting to see first a dataset (`data=`) and then a mapping attribute (`mapping=`) and, within that mapping attribute a x variable (`x=`). We could shorten the code by omitting all of these. The two following pieces of code are, therefore, equivalent:

```
library(ggplot2)
ggplot(data=cars, mapping=(aes(x=speed)))+
  geom_histogram()

##OR##

library(ggplot2)
ggplot(cars, (aes(speed)))+
  geom_histogram()
```

Throughout the document, however, we will use the longer code with all the arguments being named. Since you are learning these new functions, it would be quite a challenge to use the shorter code right from the start. But you could certainly adopt the shorter codes later on.

LET'S GET STARTED!

Chapter 2

Distributions

When using latent class models (**LCM**) to estimate disease prevalence or diagnostic test accuracy, we will need to use various distributions. There are two situations where we will use distributions:

1) as prior distributions to describe our a priori knowledge on a given unknown parameter.

2) as a part of the likelihood function that will be used to link observed data with unknown parameters.

Some distributions will more often be used for the first objective, others will mainly be used for the latter objective. In the following sections, we will cover the most useful distributions.

2.1 Prior distributions

One of the first things that we will need for any Bayesian analysis is a way to generate and visualize a prior distribution corresponding to some scientific knowledge that we have about an unknown parameter. Generally, we will use information such as the mean and variance or the mode and the 2.5th (or 97.5th) percentile to find a corresponding distribution. Here we present three of the most important distributions. We will give more details on each of them in the following sections:

- Normal distribution: defined by a mean (*mu*) and its standard deviation (*SD*) or variance (*tau*). In some `R` packages and software, `rjags` and JAGS for instance, the inverse of the variance (*1/tau*) is used to specify a given normal

distribution.
Notation: dNorm(*mu*, *1/tau*)

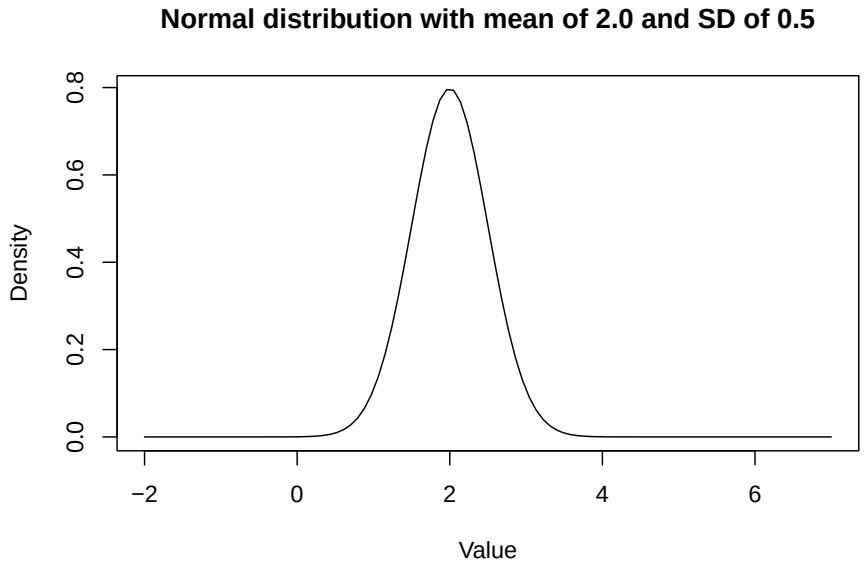
- Uniform distribution: defined by a minimum (*min*) and maximum (*max*).
Notation: dUnif(*min*, *max*)

- Beta distribution: bounded between 0 and 1, beta distributions are defined by two shape parameters (*a*) and (*b*).
Notation: dBeta(*a*, *b*)

2.1.1 Normal distribution

The `dnorm()` function can be used to generate a given Normal distribution and the `curve()` function can be used to visualize the generated distribution. These functions are already part of basic `R` functions, there is no need to upload a `R` package.

```
curve(dnorm(x,
            mean=2.0,
            sd=0.5),                                          # We indicate mea
      from=-2, to=7,                                          # We indicate lim
      main="Normal distribution with mean of 2.0 and SD of 0.5",     # Adding a title
      xlab = "Value",
      ylab = "Density")                                  # Adding titles for axes
```



Note that a Normal distribution with mean of zero and a very large *SD* provides

very little information. Such distribution would be referred to as **uniform or flat distribution** (A.K.A.; a vague distribution).

```
curve(dnorm(x,
            mean=0.0,
            sd=10000000000),
      from=-100, to=100,
      main="A flat Normal distribution",
      xlab = "Value",
      ylab = "Density")
```

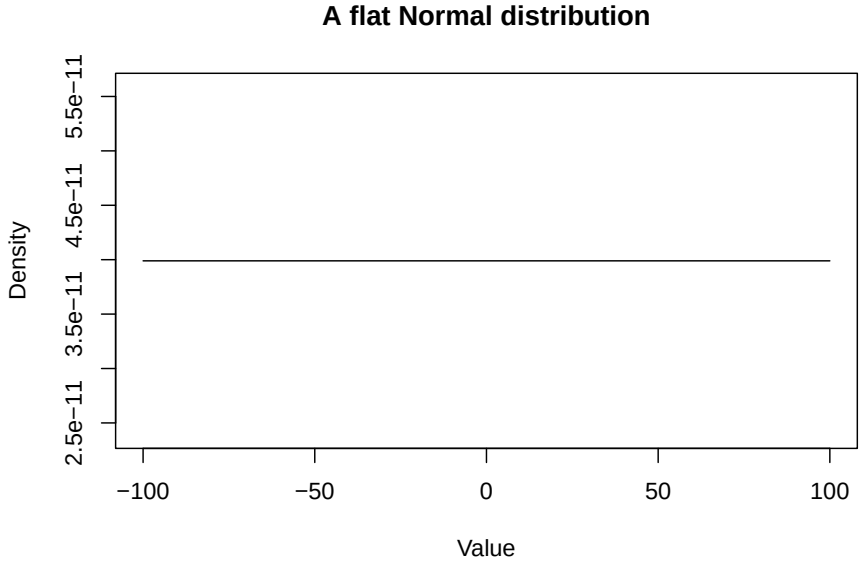


Figure 2.1: Density curve of a flat Normal distribution.

2.1.2 Uniform distribution

In the same manner, we could visualize a uniform distribution using the `dunif()` function. In the following example, we assumed that any values between -5.0 and 5.0 are equally probable.

```
curve(dunif(x,
            min=-5.0,
            max=5.0),
```

```
      from=-10, to=10,
      main="Uniform distribution with -5.0 and 5.0 limits",
      xlab = "Value",
      ylab = "Density")
```

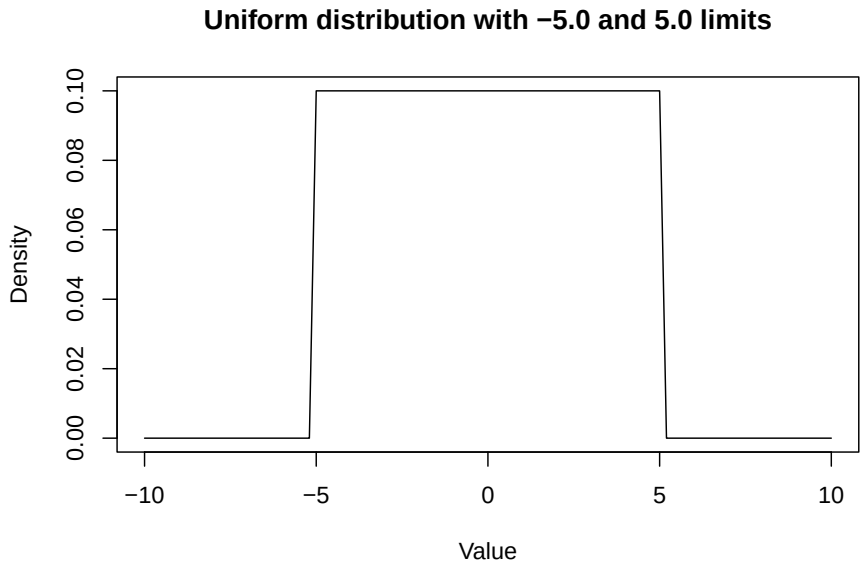


Figure 2.2: Density curve of a Uniform distribution.

2.1.3 Beta distribution

Beta distributions are another type of distributions that will specifically be used for parameters that are proportions (i.e., bounded between 0.0 and 1.0). For this specific workshop, they will be very handy, since sensitivity, specificity, and prevalence are all proportions. Beta distributions are defined by two shape parameters (a) and (b), but these parameters have no direct interpretation, as compared for instance, to the normal distribution where the first parameter is the mean and the second parameter is a measure of the variance (e.g., the standard deviation, the variance, or the inverse of the variance, depending on the software used). Nonetheless, the a and b parameters of a beta distribution are also related to the mean and the spread of this distribution, but as follows:

$a = mu * psi$
$b = psi * (1 - mu)$

Where *mu* is the mean of the distribution and *psi* is a factor that controls the spread of the distribution. Moreover, *psi* is also the sum of the a and b parameters of the beta distribution:

$psi = a + b$

From the above equations, we could also deduct that the mean of a beta distribution, *mu*, can be computed as:

$mu = a/(a + b)$

Given that there is not direct interpretation of the a and b parameters of a beta distribution, it is easier to rely on a software to find the a and b parameters that will correspond to a desired distribution with a certain mode (or mean) and variance. The `epi.betabuster()` function from the `epiR` library can be used to define a given prior distribution based on previous knowledge. When we use the `epi.betabuster()` function, it creates a new `R` object containing different elements. Among these, one element will be named *shape1* and another *shape2*. These correspond to the a and b shape parameters of the corresponding beta distribution.

For instance, we may know that the most likely value for the sensitivity of a given test is 0.85 and that we are 97.5% certain that it is greater than 0.75. With these values, we will be able to find the a and b shape parameters of the corresponding beta distribution.

Within the `epi.betabuster()` function, the `imsure=` argument will be important to indicate whether the next argument `x=0.75` describe the:
- 2.5th percentile (`imsure="greater than"`) or the
- 97.5th percentile (`imsure="less than"`).

Typically, when the mode of a distribution is <50% we suggest to use the 97.5th percentile to describe the spread of the distribution. When the mode is >50% we suggest to use the 2.5th percentile.

```
library(epiR)

# Sensitivity of a test as Mode=0.85, and we are 97.5% sure >0.75
rval <- epi.betabuster(mode=0.85,                    # We create a new object named rval
                       conf=0.975,
                       imsure="greater than",
                       x=0.75)

rval$shape1                 #View the a shape parameter in rval
```

```
## [1] 62.661
```

```
rval$shape2                 #View the b shape parameter in rval
```

```
## [1] 11.88135
```

```
#plot the prior distribution
curve(dbeta(x,
            shape1=rval$shape1,
            shape2=rval$shape2),
      from=0, to=1,
      main="Prior for test's sensitivity",
      xlab = "Proportion",
      ylab = "Density")
```

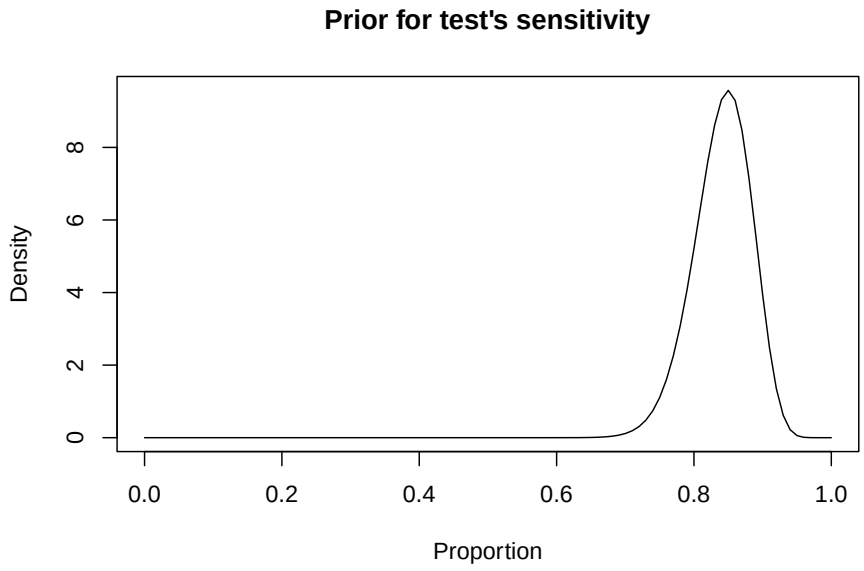


Figure 2.3: Density curve of a Beta distribution for a test sensitivity.

Note that a dBeta(1.0, 1.0) is a uniform beta distribution.

```
#plot the prior distribution
curve(dbeta(x,
            shape1=1.0,
            shape2=1.0),
      from=0, to=1,
      main="A Beta(1.0, 1.0) or flat distribution",
```

```
      xlab = "Proportion",
      ylab = "Density")
```

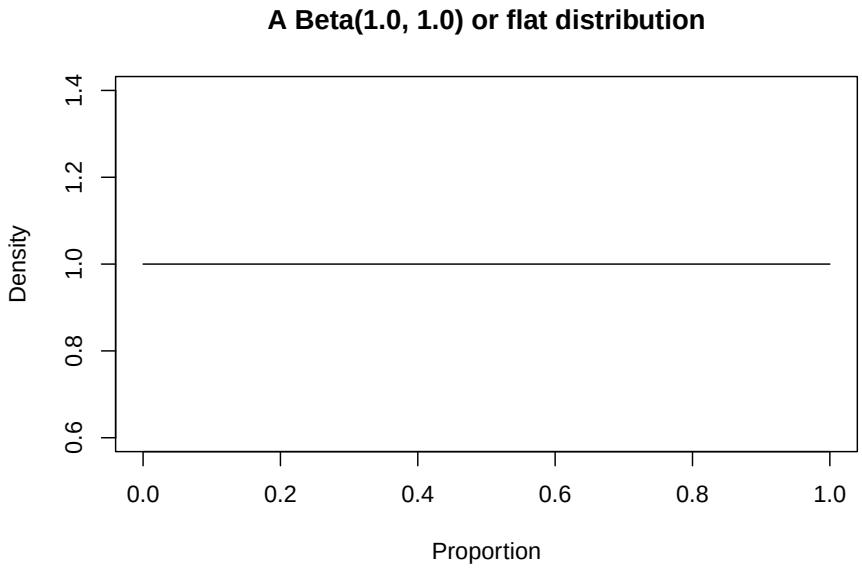


Figure 2.4: Density curve of a Beta(1.0, 1.0) distribution.

2.2 Distributions for likelihood functions

In many situations, a distribution will be used as a part of the likelihood function to link observed data to unknown parameters. The ones we will most frequently use are:

- Binomial distribution: For variables that can take the value 0 or 1. Binomial distributions are defined by the probability (P) that the variable takes the value 1 and a number of "trials" (n).
Notation: dBin(P, n)

- Multinomial distribution: For qualitative variables that can take more than 2 values. We can use the multinomial distribution to represent multiple probabilities, all bounded between 0 and 1, and that together will add up to 1. Multinomial distributions are defined by k probabilities ($P1$, $P2$, ..., Pk) and the number of observations (n).
Notation: dmulti($P[1:k]$, n)

Table 2.1: Cross-classified results from two diagnostic tests

	Test A+	Test A-
Test B+	n1	n3
Test B-	n2	n4

2.2.1 Binomial distribution

If we have a variable that can take only two values, healthy or diseased (0 or 1), we can estimate an unknown parameter such as the proportion (P) of diseased individuals (i.e., the disease prevalence), based on the observed data (a number of positive individuals (T) AND a total number of individuals (n)). For instance, if we had 30 diseased ($T = 30$) out of 100 tested individuals ($n = 100$) we could estimate the unknown parameter P using this very simple likelihood function:

$T \sim dbin(P, n)$

2.2.2 Multinomial distribution

A last type of distribution that we will use in our LCM is the multinomial distribution. When an outcome is categorical with >2 categories, we could use a multinomial distribution to describe the probability that an individual has the value "A", or "B", or "C", etc. In our context, the main application of this distribution will be for describing the combined results of two (or more than two) diagnostic tests. For instance, if we cross-tabulate the results of Test A and Test B, we have four potential outcomes:

-Test A+ and Test B+ (lets call *n1* the number of individuals with that combination of tests results and *P1* the probability of falling in that cell)
-Test A+ and Test B- (*n2* and *P2*)
-Test A- and Test B+ (*n3* and *P3*)
-Test A- and Test B- (*n4* and *P4*)

We can illustrate this as follow:

Thus we could describe the probabilities (*P1* to *P4*) of an individual falling into one of these cells of the 2x2 table as a multinomial distribution. In this specific case, we would say that the combined results of the 2 tests (*n1* to *n4*) and the total number of individual tested (n), which are the observed data, are linked to the 4 probabilities (*P1* to *P4*; the unknown parameters) as follows:

$n[1:4] \sim dmulti(P[1:4], n)$

Which means: the value of *n1* (or *n2*, *n3*, or *n4*) is determined by the probability of falling into the "Test A+ and Test B+" cell, which is *P1* (or *P2*, *P3*, or *P4* for the other cells), and by the total number of individuals tested (n). Nothing too

surprising here... If I have a probability of 0.30 to fall in a given cell, and I have tested 100 individuals, I should find 30 individuals in that cell. Wow! Still, the multinomial distribution is nice because it will ensure that all our probabilities (*P1* to *P4*) will sum up to exactly 1.0.

Chapter 3

Running Bayesian models

Different software and `R` packages can be used to run Bayesian models. Whether it is a LCM or a more conventional regression model is not very important at this stage. In this document we will use a software called `JAGS` and the `R2jags` package. **So, if not already done, you will have to download JAGS here and install it on your computer.**

Basically, to run any Bayesian model, we will always need the same three things:

- Data
- A model containing:
  - A likelihood function linking observed data to unknown parameters
  - A prior distribution for each unknown parameter
- An initial value for each unknown parameter to start the Markov chains
  - These values can also be generated randomly by the software, but specifying them will improve reproducibility

3.1 The data

This is the easy part. If we were to run, for instance, a "conventional" regression analysis, we would have to import in `R` a complete dataset (e.g., a CSV file) with a row for each observation and a column for each variable. However, when using LCM to estimate disease prevalence or diagnostic accuracy, the dataset is simply the number of individuals in each category. For instance, to estimate a simple prevalence ($Prev$), we could have 30 individuals that tested positive (variable T) out of a total of 100 individuals (variable n). In such case, the dataset could be created as follow:

```
#Creating a dataset
datalist <- list(T=30, n=100)
```

If we want to check that it works, we could ask to see this new R object.

```
datalist
```

```
## $T
## [1] 30
##
## $n
## [1] 100
```

3.2 The model

3.2.1 The likelihood function

Let's start with a simple example, estimating a single proportion. For estimating a proportion we could describe the likelihood function linking unknown parameters to observed data as follows:

$T \sim dbin(Prev, n)$

In other words: the disease prevalence ($Prev$), an unknown parameter, can be linked to the observed number of positive individuals (T) and the total number of tested individual (n) through the binomial function.

In this case, the values of T and n are the observed data and $Prev$ is the only unknown parameter.

3.2.2 Priors for unkown parameters

We will have to specify a prior distribution for each unknown parameter (using the $\sim$ sign).

In the preceding example, we only have one unkown parameter, $Prev$, which is a proportion. Theoretically such parameter could take values from 0 to 1, so a beta distribution would be one way to describe its prior distribution. We could use the `epi.betabuster()` function from the `epiR` library to define a given beta distribution based on previous knowledge, as described in the previous sections. Moreover, if we want to use a flat prior, we could simply choose the value 1.0 for the a and b shape parameters.

In the following code, I am creating R objects for a and b for $Prev$ beta prior distributions.

```
Prev.shapea <- 1          #a shape parameter for Prev
Prev.shapeb <- 1          #b shape parameter for Prev
```

With informative priors, for instance, if we know from the literature that the prevalence is expected to be 0.25 with 95CI of (0.20, 0.30), we could instead use the following code and assign these values to *Prev.shapea* and *Prev.shapeb*:

```
library(epiR)

# Prevalence as Mode=0.25, and we are 97.5% sure <0.30
rval <- epi.betabuster(mode=0.25,
                       conf=0.975,
                       imsure="less than",
                       x=0.30)

rval$shape1                #View the a shape parameter in rval
```

```
## [1] 81.872
```

```
rval$shape2                #View the b shape parameter in rval
```

```
## [1] 243.616
```

```
#The two lines below would be used to assign the generated values for later usage. Below we decid
Prev.shapea <- rval$shape1
Prev.shapeb <- rval$shape2
```

3.2.3 Creating the JAGS model

Our Bayesian model has to be presented to JAGS as a text file (i.e., a .txt document). To create this text file, we will use the `paste0()` function to write down the model and then the `write.table()` function to save it as a .txt file.

An interesting feature of the `paste0()` function is the possibility to include `R` objects in the text. For instance, we may want to have a generic model that will not be modified, but within which we would like to change the values used to describe the priors. We could include these "external" values within our `paste0()` function. Thus, we just have to change the value of the `R` object that our model is using, rather than rewriting a new model. In the previous section, we created two `R` objects called *Prev.shapea* and *Prev.shapeb* (which were the a and b shape parameters for the prior distribution of our proportion *Prev*). To describe our prior distribution, we can ignore this previous work that we have done and directly create a short text as follows:

```
text1 <- paste0("Prev ~ dbeta(1, 1)")       #Creating a short text named "text1"
text1                                       #Asking to see the text1 object
```

```
## [1] "Prev ~ dbeta(1, 1)"
```

However, if the same model is to be applied later with a different prior distribution, it may be more convenient to leave the a and b shape parameters as R objects. Thus, if we change the prior distribution and run the script again, we would run the entire analyses with the newly created distribution. For instance, the code below would use the a and b shape parameters that we created above.

```
text2 <- paste0("Prev ~ dbeta(",Prev.shapea,", ",Prev.shapeb,")")          #Creating
text2                                                                      #Asking t
```

```
## [1] "Prev ~ dbeta(81.872, 243.616)"
```

When using `paste0()`, any text appearing outside of sets of quotation marks (e.g., *Prev.shapea* and *Prev.shapeb* in the preceding script) will be considered as R objects that need to be included in the text. Thus, later, we could just change the values for *Prev.shapea* and *Prev.shapeb* and our model will be automatically modified.

Some important notes about the text file containing the model before we start with R2jags:

- JAGS is expecting to see the word "model" followed by curly brackets { }. The curly brackets will contain the likelihood function **AND** the priors.
- Similarly to R, any line starting with a # will be ignored (these are comments).
- Similarly to R, <- means equal.
- The ~ symbol means "follows a distribution".
- Similarly to R, JAGS is case sensitive (i.e., P does not equal p).
- Loops can be used with keyword "for" followed by curly brackets.
- Array can be indicated using squared brackets [ ].

In the example below, we create an R object called *model_single_prop* using the `paste0()` function and then save it as a text file using the `write.table()` function. We will check this text file below and then explain the code for the model.

```
# Specify the model
model_single_prop <- paste0("model{

#=== LIKELIHOOD ===#

  T ~ dbin(Prev, n)


#=== PRIOR  ===#

  Prev ~ dbeta(",Prev.shapea,", ",Prev.shapeb,")    ## Prior for Prev

}")

#write as a text (.txt) file
write.table(model_single_prop,
            file="model_single_prop.txt",
            quote=FALSE,
            sep="",
            row.names=FALSE,
            col.names=FALSE)
```

Here is a snapshot of the final result (i.e., the text file):

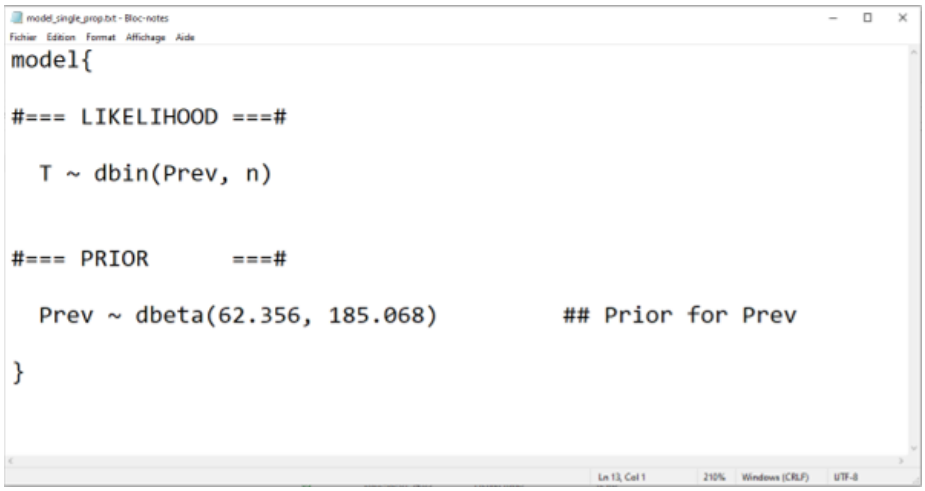


Figure 3.1: **Text file with the model for estimating a single proportion.**

In the preceding script, the `paste0()` function was used to create the text, as an `R` object, that will be needed for the .txt file. This text is mainly "static",

except for the a and b parameters of the beta distribution used to describe our prior knowledge on disease prevalence. We can see that, if we were to modify and then run the preceding script where we assigned values to these parameters, and then run again the `paste0()` function, our text file would be updated with new values for our prior distribution. This is a feature that we will use a lot, especially with more complex LCM, to minimize the amount of work and the risk of typographic errors..

The `write.table()` function was simply used to save, as a .txt file, the `R` object containing the text we just created.

Both the `paste0()` and `write.table()` functions are part of basic `R`, no need to install or call a given library.

3.3 Generating initial values

The initial value is the first value where each Markov chain will start.
- We have to specify one initial value for each unknown parameter in the model.
- If we decide to run three Markov chains, we will need three different sets of initial values.
- We can also let the software choose a random value to start each chain, but it will then be impossible to **exactly** reproduce a given result. For instance, if we have obtained a very awkward Markov chain, we may not be able to reproduce this result to investigate what went wrong. Note, though, that it would be quite uncommon for a Markov chain to be extremely "sensitive" to the chosen starting value. But we will see during the workshop that it will sometimes happen.

In our preceding example we have one unknown parameter (*Prev*). If we were to run three Markov chains, we could generate three sets of initial values for this parameter as follows. Of course, the chosen values could be changed. In the case of a proportion, we can specify any value between 0 and 1.

```
#Initializing values for the parameter "Prev" for the 3 chains
inits <- list(
  list(Prev=0.10),
  list(Prev=0.50),
  list(Prev=0.90)
  )
```

We now have an `R` object called *inits* that contains the three values where each Markov chains will start.

3.4 Running a model in JAGS

We now have the three elements that we needed: the data, the model, and a set of initial values. We can now use the `R2jags` library to run our Bayesian analysis. The `R2jags` package provides an interface from `R` to the `JAGS` software for Bayesian data analysis. `JAGS` uses Markov Chain Monte Carlo (**MCMC**) simulation to generate a sample from the posterior distribution of the parameters.

The `R2jags` function that we will use for that purpose is the `jags()` function. Here are the arguments that we need to specify:

- The dataset using `data=`
- The text file containing the model written in `JAGS` code using `model.file=`
- The names of the parameters which should be monitored using `parameters.to.save=`
- The number of Markov chains using `n.chains=`. The default is 3
- The sets of initial values using `inits=`. If inits is NULL, `JAGS` will randomly generate values
- The number of total iterations per chain (including burn-in; default: 2000) using `n.iter=`
- The length of burn-in (i.e., number of iterations to discard at the beginning) using `n.burnin=`
- The thinning rate, which represent how many of the values sampled by the MCMC will be retained. The default is 5 (1 out of 5). Since there is no *a priori* need for thinning, we can specify `n.thin`=1 in most situations
- The last argument `DIC=TRUE` or `DIC=FALSE` will determine whether to compute the deviance, pD, and deviance information criteria (**DIC**). These could sometimes be useful for model selection/comparison. We could leave that to `DIC=FALSE` since we will not be using these values during the workshop

With the following code, we will create an `R` object named *bug.out*. This object will contain the results of running our model with the specified priors using the available data, and the sets of initial values that we have created. For this analysis, we have asked to run 3 Markov chains for 6000 iterations each and we discarded the first 1000 iterations of each chain. Finally, we will monitor our only unknown parameter: *Prev*.

```
library(R2jags)
library(coda)

#Run the Bayesian model
bug.out <- jags(data=datalist,                                  #Specifying the R object co
                model.file="model_single_prop.txt",             #Specifying the .txt file c
                parameters.to.save=c("Prev"),                   #Specifying the parameters
                n.chains=3,                                     #Number of chains
                inits=inits,                                    #Specifying the R object co
                n.iter=6000,                                    #Specifying the number of i
                n.burnin=1000,                                  #Specifying the number of i
                n.thin=1,                                       #Specifying the thinning ra
                DIC=FALSE)                                      #Indicating that we do not
```

```
## Compiling model graph
##    Resolving undeclared variables
##    Allocating nodes
## Graph information:
##    Observed stochastic nodes: 1
##    Unobserved stochastic nodes: 1
##    Total graph size: 5
##
## Initializing model
```

As we can see, we received a few messages. It looks good, don't you think?

3.5 Model diagnostic

Before jumping to the results, we should first verify that:
- The chains have converged,
- The burn-in period was long enough,
- We have a sample of values large enough to describe the posterior distribution with sufficient precision,
- The Markov chains behaved well (mainly their auto-correlation).

There are many diagnostic methods available for Bayesian analyses, but, for this workshop, we will mainly use basic methods relying on plots.

Diagnostics are available when we convert our model output into a MCMC object using the `as.mcmc()` function of the `mcmcplots` library.

```
library(mcmcplots)
bug.mcmc <- as.mcmc(bug.out)          #Converting the model output into a MCMC object
```

Then, we can apply the `mcmcplot()` function on this newly created MCMC object.

```
mcmcplot(bug.mcmc, title="Diagnostic plots")        #Asking for the diagnostic plots
```

When we do that, a HTML document will be created and our web browser will automatically open it. Here is a snapshot of the HTML file in our browser:

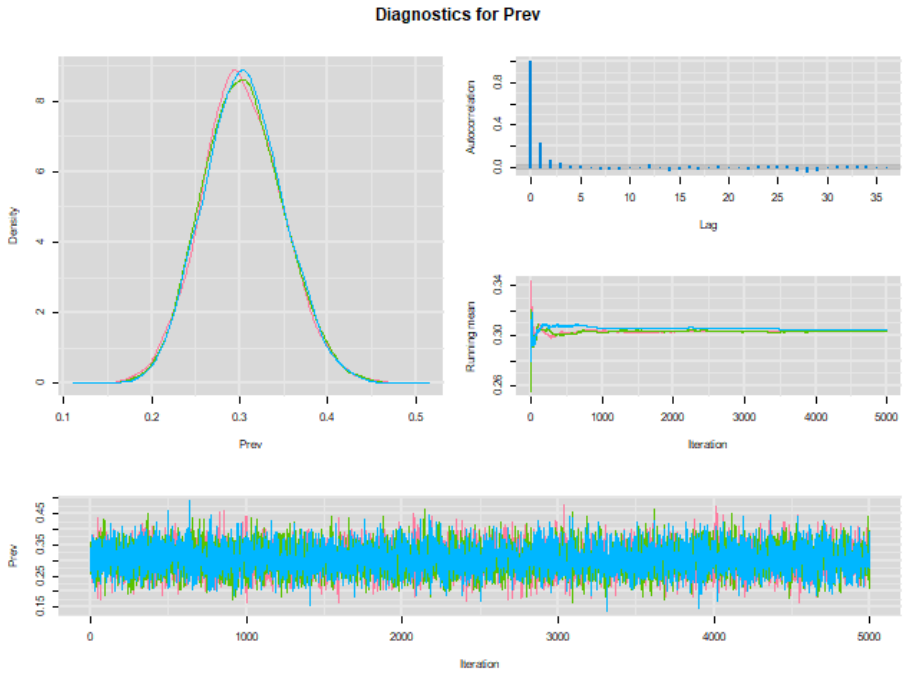


Figure 3.2: **Diagnostic plots.**

3.5.1 Convergence and burn-in

To check whether convergence of the Markov chains was achieved we can inspect the "trace plot" (the bottom figure) presenting the values that were picked by each chain on each iteration. The different chains that we used (three in the preceding example) will be overlaid on the trace plot. We should inspect the trace plot to **evaluate whether the different chains all converged in the same area**. If so, after a number of iterations they should be moving in a relatively restricted area and this area should be the same for all chains.

The plots above would be a good example of the trace plot of three "nice" Markov chains (lucky us!). In this preceding example, we can see on the trace

plot that the chains had already converged (probably way before the end of the first 1000 iterations that were excluded) toward the same area. On the trace plot, we can only see the values that were stored after the burn-in period. The chains were then moving freely within this limited area (thus sampling from the posterior distribution).

We could conclude based on the trace plot that:
- The Markov chains did converge
- A short burn-in period of less than a 1000 iterations would have been sufficient. Note that we will often use a longer burn-in (e.g., 5,000 or even more) just to be sure and because it only costs a few more seconds on our computer…

Here is another example where two chains (the red and green) converged in the same area, but a third one (the blue) also converged, but in a different area. **We should be worried in such case**. We will see later the reason for such behavior.

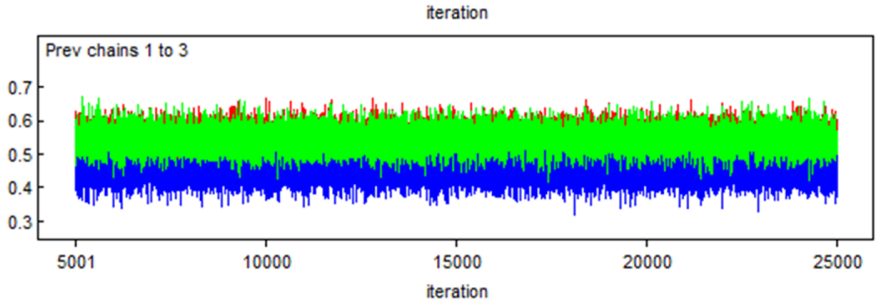


Figure 3.3: **Trace plot - chains converging in different areas.**

Finally, here is a last example with just one Markov chain which clearly did not converge, even after 25,000 iterations. **We should be terrified by that!!! ;-)**.

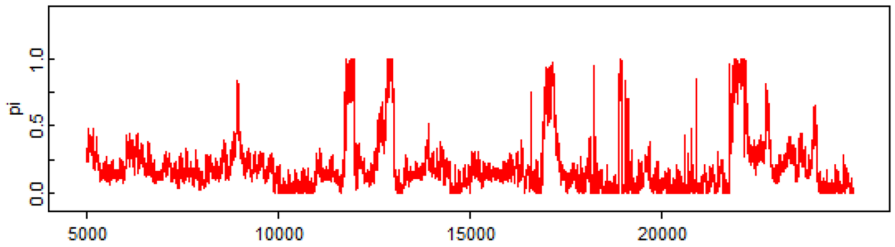


Figure 3.4: **Trace plot - a non-converging chain.**

3.5.2 Number of iterations

Next, we can inspect the posterior distributions to determine if we have a large enough sample of independent values to describe with precision the median estimate and the 2.5th and 97.5th percentiles. The effective sample size (**EES**) takes into account the number of values generated by the Markov chains **AND** whether these values were auto-correlated, to compute the number of "effective" independent values that are used to describe the posterior distribution. Chains that are auto-correlated will generate a smaller number of "effective values" than chains that are not auto-correlated.

The `effectiveSize()` function of the `coda` library provide a way to appraise whether we had enough iterations. In the current example, we already created a `mcmc` object named `bug.mcmc`. We can ask for the effective sample size as follows:

```
effectiveSize(bug.mcmc)          #Asking for the ESS
```

```
##     Prev
## 9577.497
```

So we see here that, despite having stored 3 times 5,000 values (15,000 values), we have the equivalent of roughly 10,000 independent values. What to do with this information? Well, we could, for instance, decide on an arbitrary rule for ESS (say 10,000 values) and then adapt the length of the chains to achieve such an ESS **for each of the selected parameters**, because the effective sample sizes will not be the same for all parameters.

3.5.3 Auto-correlation

Remember that a feature of Markov chains is to have some auto-correlation between two immediately subsequent iterations, and then a correlation that quickly goes down to zero (i.e., they have a short memory). The auto-correlation plot can be used to assess if this feature is respected.

From our previous example (check the upper right figure in the plot below), it seems that it is the case.

Here is another example where we do have an auto-correlation that is persisting.

When such a behavior is observed, running the chains for more iterations will help achieve the desire ESS. In some situations, specifying different priors (i.e., increasing the amount of information in priors) may also help. In the past, thinning of the Markov chains (i.e., discarding all but every k^{th} sampled value) was

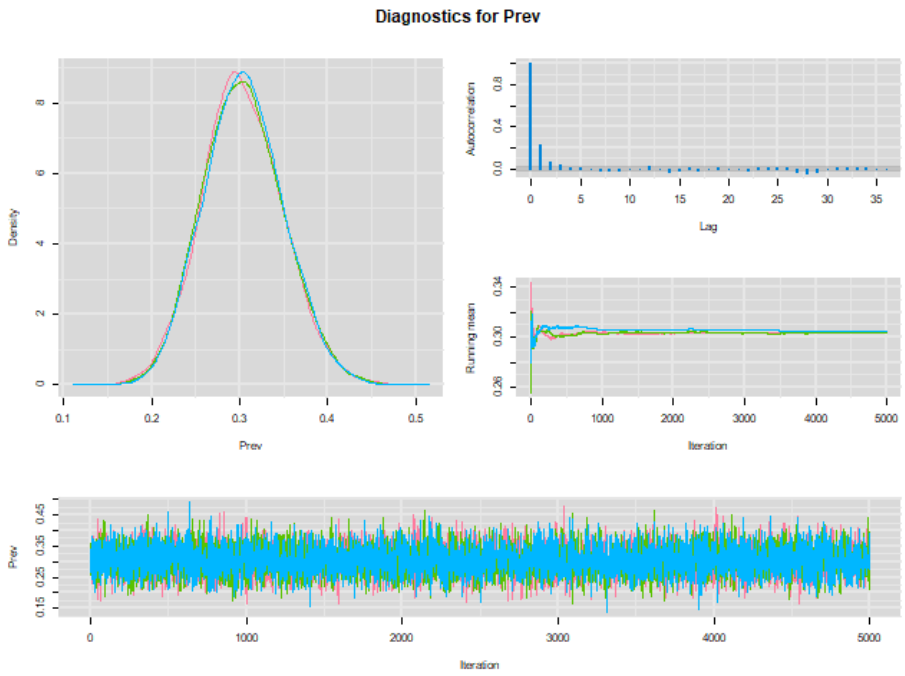


Figure 3.5: **Autocorrelation plot with Markov chains exhibiting a short memory.**

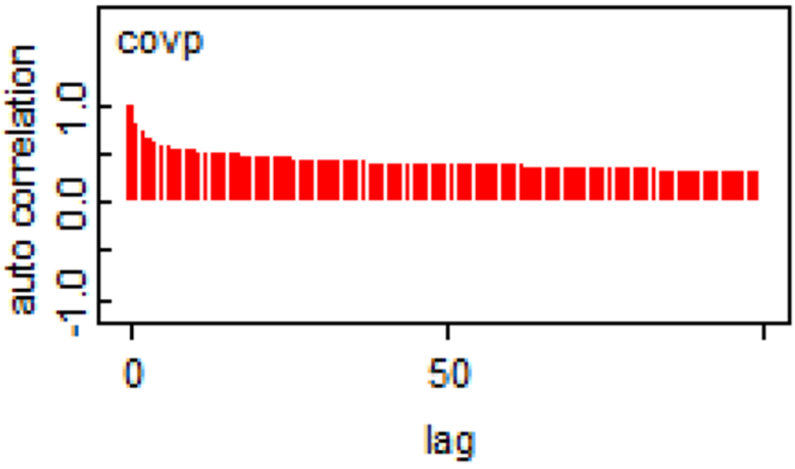


Figure 3.6: **Autocorrelation plot with Markov chains exhibiting a long memory.**

often used when autocorrelation was detected, but this method is not recommended anymore, except for rare instances where computer memory limitations would be a problem Link et Eaton, 2012.

3.6 Getting our estimates

Now, if we are happy with the behavior of our Markov chains, the number of iterations, and burn-in period, we can summarize our results (i.e., report the median, 2.5th and 97.5th percentiles) very easily with the `print()` function. Below, we also indicated `digits.summary=3` to adjust the precision of the estimates that are reported (the default is 1).

```
print(bug.out,                    #Asking to see the mean, median, etc of the unknown parameters th
      digits.summary=3)           #Requiring a precision of 3 digits
```

```
## Inference for Bugs model at "model_single_prop.txt", fit using jags,
##  3 chains, each with 6000 iterations (first 1000 discarded)
##  n.sims = 15000 iterations saved. Running time = 0.03 secs
##      mu.vect sd.vect  2.5%   25%   50%   75% 97.5%  Rhat n.eff
## Prev   0.262   0.021 0.223 0.248 0.262 0.277 0.304 1.001 15000
##
## For each parameter, n.eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor (at convergence, Rhat=1).
```

Here we see that the median *Prev* estimate (95BCI) was 26% (22, 31).

We can also plot the *bug.mcmc* object that we have created using the `plot()` function to produce the trace plot and a density plot presenting the posterior distributions of each unknown parameter.

```
plot(bug.mcmc)                #Asking for trace plot and plots of prior and posterior distributions
```

If we prefer to work on our own plots, note that all the values sampled from our posterior distributions were stored in the *bug.out* object that we have created. If we inspect the *bug.out* object, we will notice an element named *BUGSoutput*, inside this element, there is another object named *sims.list*, within this *sims.list*, there is an element named *Prev*. As we can see below, this element is made of a list of 15,000 values. This correspond to the 5,000 values sampled (1/iteration) by each Markov chain (3 chains) for the unknown parameter *Prev*. **So this element is simply all the sampled values assembled together**.

We could use this element, for instance, to plot together the prior and posterior distributions on a same figure. On the figure below, the prior distribution is not

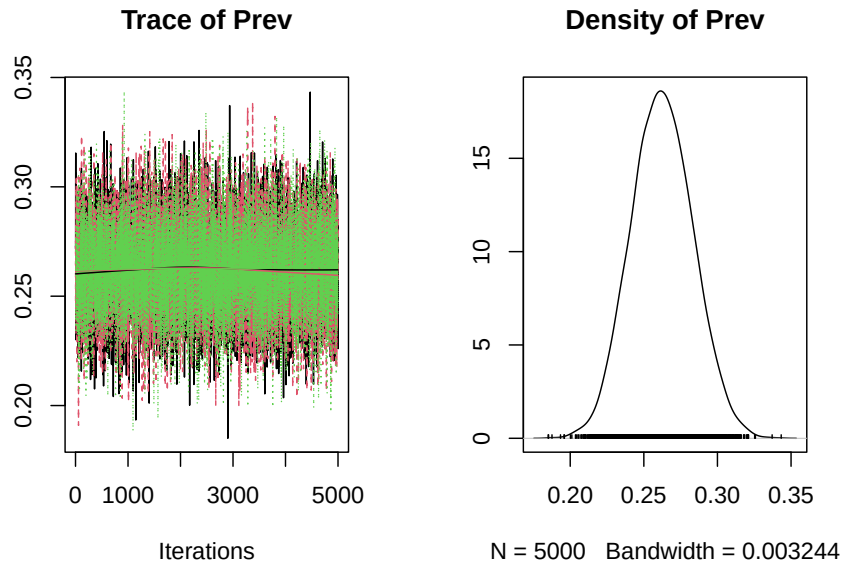


Figure 3.7: Trace plot and density plot produced using the plot() function.

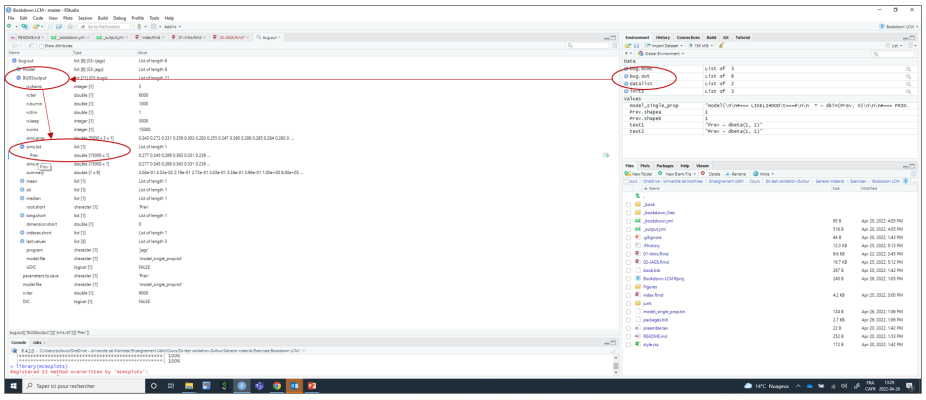


Figure 3.8: **sims.list object located in the *bug.out* output.**

obvious, it is the flat dashed red line just above the x axis (remember, we used a flat prior for *Prev*).

```
#Plot the posterior distribution
plot(density(x=bug.out$BUGSoutput$sims.list$Prev),      #Indicating variable to plot
     main="Disease prevalence",                         #Title for the plot
     xlab="Prevalence", ylab="Density",                 #x and y axis titles
     )

#plot the prior distribution
curve(dbeta(x,
            shape1=Prev.shapea,
            shape2=Prev.shapeb),
      from=0, to=1,
      lty=2,                      #Asking for a dashed line pattern
      col="red",                  #Asking for a red line
      add=TRUE)                  #Asking to add this plot to the preceding one
```

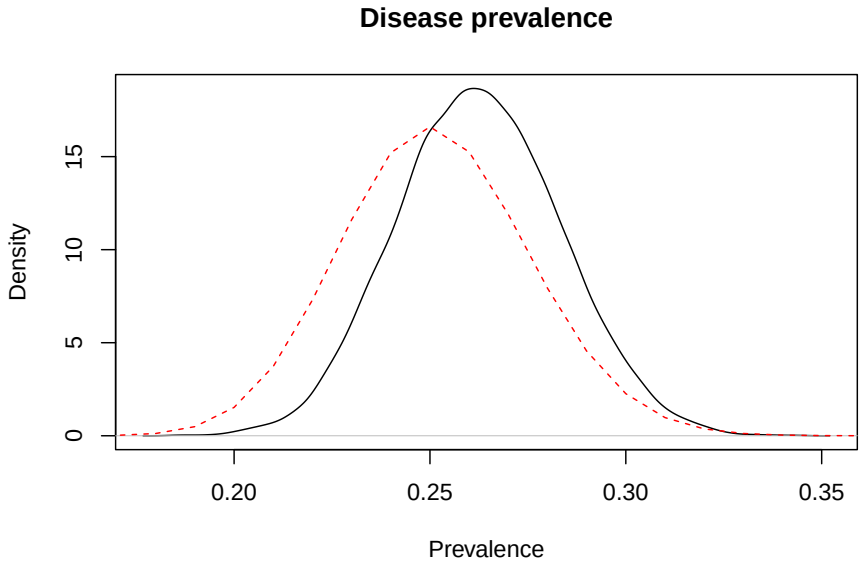


Figure 3.9: Prior (dashed red line) and posterior (full black line) distribution of the prevalence of disease.

Chapter 4

Exercise 1 - Proportions

4.1 Data

The data used for the following exercises came from a study aiming at estimating diagnostic accuracy of a milk pregnancy associated glycoprotein (**PAG**) ELISA and transrectal ultrasonographic (**US**) exam when used for determining pregnancy status of individual dairy cows 28 to 45 days post-insemination. In that study, the new test under investigation was the PAG test and US was used for comparison, but was considered as an imperfect test.

In the original study, data from 519 cows from 18 commercial dairy herds were used. For the following exercises, the dataset was modified so that we have data from 519 cows from 3 herds (i.e., the data from 16 herds were collapsed together so they appear to be from one large herd). Note that there are some cows with missing values for one of the tests, so we will not always work with 519 cows. The complete original publication can be found here: Dufour et al., 2017.

The dataset *Preg.xlsx* contains the results from the study. However, **for the exercises we will always simply use the cross-tabulated data and these will be already organized for you and presented at the beginning of each exercise**. The dataset is still provided so you can further explore additional comparisons. The list of variables in the *Preg.xlsx* dataset are described in the table below.

Table. List of variables in the *Preg.xlsx* dataset.

Variable	Description	Range
Obs	An observation unique ID number	1 to 519
Herd	Herd ID number	1 to 3
Cow	Cow ID number	NA

Variable	Description	Range
T1_DOP	Number of days since insemination at testing	28 to 45
PAG_DX	Results from the PAG test	0 (not pregnant); 1 (pregnant)
US_DX	Results from the ultrasound test	0 (not pregnant); 1 (pregnant)
TRE_DX	Results from the transrectal exam (TRE) test	0 (not pregnant); 1 (pregnant)

4.2 Questions

We will use data from one population (herd #1). In this herd, we had the following proportion of apparently pregnant cows (based on the US exam).

139 US+/262 exams

Open up the `R` project for Exercise 1 (i.e., the R project file named *Exercise 1.Rproj*).

In the Exercise 1 folder, you will find partially completed `R` scripts. To answer the questions, try to complete the missing parts (they are highlighted by a **#TO COMPLETE#** comment). We also provided complete `R` scripts, but try to work it out on your own first!

1. Start from the partially completed *Question 1.R* script located in Exercise 1 folder. What would be the proportion of pregnant cows? Use a Bayesian approach to compute that proportion and a credible interval (**CI**). For this first computation, assume that we have no prior knowledge on the expected proportion of pregnant cows in a Canadian herd. Run three Markov chains with different sets of initial values. Look at the trace plot. Do you think convergence was achieved? Do you need a longer burn-in period? Are all 3 chains converging in the same space? Compute the effective sample size (**ESS**), do you feel that number of iterations was sufficient to describe the posterior distribution with enough precision? What about auto-correlation of the Markov chains, any issues there?

2. If we were to compute a Frequentist estimate with 95% CI, would it differ a lot from our Bayesian estimates? Why? As a refresher, the formula for a Frequentist 95%CI for a proportion is below (where P is the actual observed proportion and n is the denominator for that proportion):

$$95CI = P \pm 1.96 * \sqrt{\frac{P*(1-P)}{n}}$$

3. Start from the partially completed *Question 3.R* script located in Exercise 1 folder. We just found that Romano et al. (2006) reported, for a

US exam conducted between 28-45 days post-insemination:
-Sensitivity (**Se**) = 0.90 (95CI: 0.85, 0.95)
-Specificity (**Sp**) = 0.95 (95CI: 0.90, 0.97)

We now realized that, since a US exam is not a gold-standard test, in the previous exercises we rather computed an apparent prevalence (**AP**), not the true prevalence (**P**). From the literature, we know that these two parameters are linked as follow:

$AP = Se * P + (1 - Sp) * (1 - P)$

Knowing that we had the following proportion of apparently pregnant cows (based on the US exam).

139 US+/262 exams

And assuming, again, that we have no prior knowledge on the expected proportion of pregnant cows in a Canadian herd.

Can you think of a way to estimate the true prevalence, rather than the apparent prevalence, using that formulae?

4. Start from the `R` script that you completed in Question 1. In the literature, we saw a recent study conducted on 30 Canadian herds and reporting an expected pregnancy prevalence of 42% with 97.5% percentile of 74%. What kind of distribution could we use to represent this information? Use these information as a pregnancy prevalence prior distribution to compute an apparent prevalence. Are your results very different from those of Question 1?

5. Start from the partially completed *Question 5.R* script located in Exercise 1 folder. When comparing PAG to US results in herd #1, we got the following 2x2 table.

Table. Cross-classified results of the PAG and US tests in herd #1.

	US+	**US-**	**Total**
PAG+	138	10	148
PAG-	1	113	114
Total	139	123	262

Assuming that US is a gold-standard test could we compute PAG sensitivity (**Se**) and specificity (**Sp**)? Use vague priors for PAG *Se* and *Sp* since it is the first ever study on this topic (i.e., we do not have any prior knowledge on these unknown parameters).

4.3 Answers

1. What would be the proportion of pregnant cows? Use a Bayesian approach to compute that proportion and a Bayesian credible interval (**BCI**). For this first computation, assume that we have no prior knowledge on the expected proportion of pregnant cows in a Canadian herd. Run three Markov chains with different sets of initial values. Look at the trace plot. Do you think convergence was achieved? Do you need a longer burn-in period? Are all 3 chains converging in the same space? Compute the effective sample size (**ESS**), do you feel that number of iterations was sufficient to describe the posterior distribution with enough precision? What about auto-correlation of the Markov chains, any issues there?

Answer: I chose to run a model with 3 chains of 10,000 iterations each (11,000 iterations minus a burn-in of 1,000). I have obtained the following diagnostic plots:

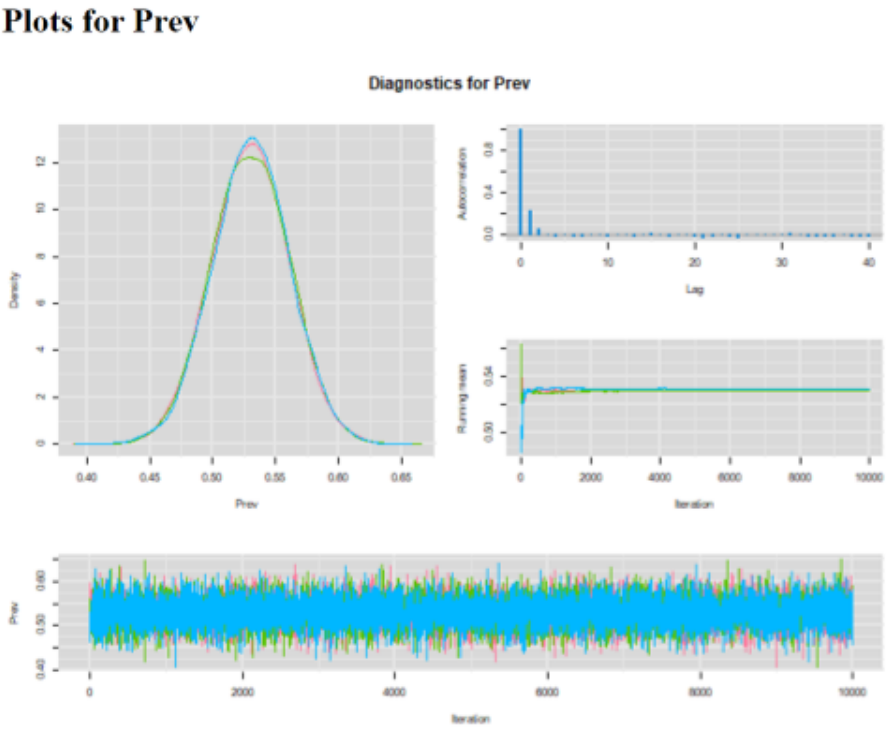


Figure 4.1: **Diagnostic plot**

Convergence of the 3 chains was achieved, all three chains appear to be moving in

the same space (see trace plot). The 1,000 iterations burn-in period is probably more than needed for this very simple problem. Autocorrelation plot is just perfect with correlation declining very rapidly close to zero at lag of 1! The effective sample size for the *Prev* parameter is >18,000 values. So, plenty of precision to report the median and 2.5th and 97.5th percentiles.

The median pregnancy prevalence estimate (95% CI) was 53.0% (46.9, 59.0).

2. If we were to compute a Frequentist estimate with 95% CI, would it differ a lot from our Bayesian estimates? Why?

Answer: It should not differ much from the Bayesian median estimate and 95CI because these latter estimates were generated using vague priors. In such cases, Bayesian and Frequentist estimates should be quite similar. Actually, if we use the Frequentist formula for computing 95CI and the observed data (i.e., 139/262) we get:

$P = 139/262 = 0.531$

$95CI = 0.531 \pm 1.96 * \sqrt{\frac{0.531*(1-0.531)}{262}} = 0.531 \pm 0.060$

Thus, we have a Frequentist estimated proportion of 53.1% with a Frequentist 95 CI of 47.1 to 59.1 (virtually unchanged compared to the Bayesian estimates).

3. Can we think of a way to estimate the true prevalence, rather than the apparent prevalence, using that formulae?

Answer: That was a tricky question don't we think? Below is the model we used for that:

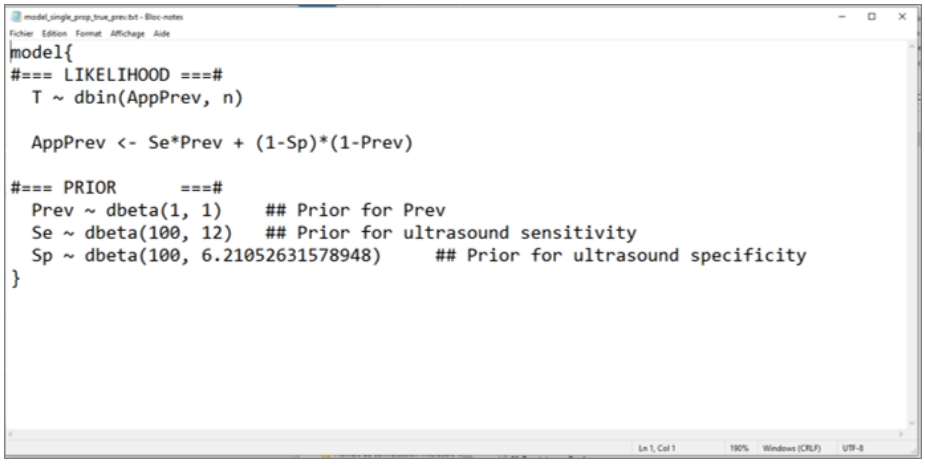


Figure 4.2: **Model used to estimate true prevalence**

It may a bit disturbing at first sight, but we did not try to isolate the true

prevalence (*Prev*) to the left side of the equation relating the apparent (*AppPrev*) and true prevalence (*Prev*). This is because Bayesian models are not necessarily read from left to right, but rather as a big "spaghetti bowl". In short, there is no need for a left to right form of mathematical writing. As long as we explain how the two parameters *AppPrev* and *Prev* are linked, the model will make sense of this.

Then, you may also have noticed that we provided prior distributions for *Prev*, an unknown parameter, and also for US sensitivity and specificity (*Se* and *Sp*). But we did not specify any prior distribution for the apparent prevalence (*AppPrev*). Indeed, *AppPrev* is also an unknown parameter, but it was directly defined by the equation linking the true (*Prev*) and apparent prevalence (*AppPrev*). Therefore, there is no need for specifying a prior distribution for *AppPrev*.

Using this model, we were able to estimate that the **median (95% BCI) true prevalence was 56.7% (48.4, 65.4)**. As a comparison, the **median (95% BCI) apparent prevalence was estimated at 53.1% (47.1, 59.1)**. You may have noted that the width of the 95% BCI for the true prevalence (17.0 percentage-points) is larger than that of the apparent prevalence (12 percentage-points). It is because we combined the uncertainty regarding the prevalence with the uncertainty regarding ultrasound sensitivity and specificity, when computing the true prevalence. For computing the apparent prevalence, we only considered the uncertainty regarding this latter proportion.

4. In the literature, we saw a recent study conducted on 30 Canadian herds and reporting an expected pregnancy prevalence of 42% with 97.5% percentile of 74%. What kind of distribution could we use to represent this information? Use these information as a pregnancy prevalence prior distribution to compute an apparent prevalence. Are our results very different from those of Question 1?

Answer: A beta(4.2, 5.4) distribution would have a mode of 0.42 and a 97.5th percentile of 0.74. Using this information as prior, I get a prevalence of pregnancy of 52.7% (95 CI: 46.8, 58.6). Actually, we observe very little difference between the models using vague *vs.* informative priors. This is because this informative prior contains only the equivalent of 10 observations ($4.2 + 5.4$). In the dataset, there are 262 cows. Therefore, the estimation process is still mainly driven by our dataset.

5. Assuming that US is a gold-standard test could you compute PAG sensitivity (**Se**) and specificity (**Sp**)? Use vague priors for PAG *Se* and *Sp* since it is the first ever study on this topic (i.e., we do not have any prior knowledge on these unknown parameters).

Answer: The sensitivity and specificity are simple proportions. Sensitivity is simply the number of test positive among the number of truly diseased individuals. We have data for these in the 2x2 table. If we assume that *US* is a gold-standard test, then the number of truly diseased cows is 139. Of them, 138 tested positive to the *PAG* test. Similarly, the specificity is simply the number

of test negative among the number of truly healthy individuals. From the 2x2 table, the number of true negative cows was 123 and 113 of them tested negative to the *PAG* test. The likelihood functions (one for each parameter in this case) linking the unknown parameters (*Se* and *Sp*) to these observed data would be:

$Test+ \sim dbin(Se, True+)$

$Test- \sim dbin(Sp, True-)$

Using vague beta(1, 1) priors on the *Se* and *Sp*, I got an estimated *Se* of 98.8% (95 CI: 96.1, 99.8) and a *Sp* of 91.5% (95CI: 85.9, 95.5). **But be cautious with these numbers.** We know very well that an *US* exam is not a perfect diagnostic test for pregnancy in dairy cows. With the current approach, we are attributing all disagreements between tests to a failure of the *PAG* test… But it could be the *US* test that was wrong in many instances. Moreover, animals for which the two tests agreed could still be misdiagnosed as pregnant or open, but by both tests (i.e., a failure of the *PAG* **AND** the *US* tests). We wil see in the next sections of the course how to account for the fact that neither of the tests are gold-standards.

Chapter 5

Two imperfect tests

When using two imperfect diagnostic tests on the same population, we will have to use a Bayesian latent class model (**LCM**) to estimate the true disease prevalence, and the sensitivity and specificity of both diagnostic tests. Two different approaches could be used. The simplest approach would be to consider that the two diagnostic tests are independent conditionally on the true health status of the individuals. Conditional independence can be defined as:

Two tests are conditionally independent when the sensitivity (or specificity) of one test does not depend on whether the results of another test are positive or negative among infected (or non-infected) individuals. Cheung et al., 2021

In many situations, however, we will use two diagnostic tests that make use of the same biological process. For instance, an ELISA test could measure antibodies against a given pathogen in blood and another test could also measure antibodies, but in milk. In such case, it would be difficult to pretend that:

The sensitivity of the milk test does not depend on whether the results of the blood test are positive or negative among infected individuals.

Therefore, it would be better to consider a LCM where conditional dependence between tests is possible. Note that, even for tests that make use of very different biological processes, it is rather difficult (or, actually, often impossible) to "demonstrate" conditional independence between them. So, if doable, allowing for a potential conditional dependence between tests would be a more robust strategy.

5.1 Assuming conditional independence

When two tests are conditionally independent, then obtaining a given combination of results to Test A and Test B, as function of the true disease prevalence

Table 5.1: Cross-classified results from two diagnostic tests

	Test A+	Test A-
Test B+	n1	n3
Test B-	n2	n4

(P) and of the accuracy of each test (SeA, SpA, SeB, SpB) can be described as illustrated below:

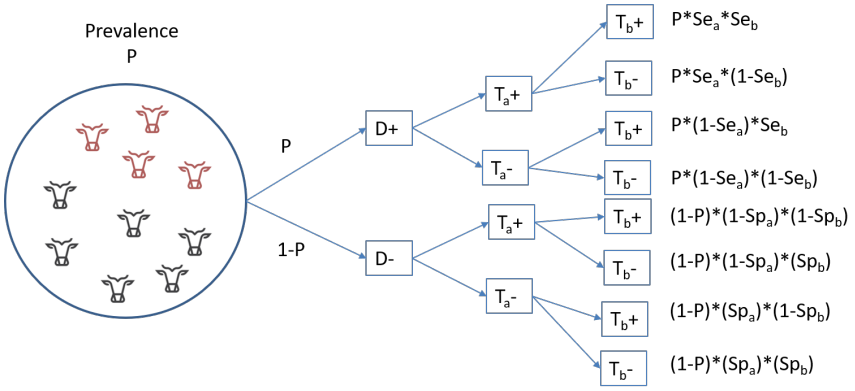


Figure 5.1: **Illustration of the different potential results when applying two conditionally independent tests in a population.**

From that illustration, we can deduce that the probability of testing positive to both tests is the sum of the first and fifth rows:

$$P(TestA + TestB+) = P * SeA * SeB + (1 - P) * (1 - SpA) * (1 - SpB)$$

Or, putted in words, it is the probability of being a truly diseased individual and of being identified as such by both tests ($P * SeA * SeB$), plus the probability of being a healthy individual and being misdiagnosed as diseased by both tests ($(1 - P) * (1 - SpA) * (1 - SpB)$). If we extend this reasoning to all tests combination possibilities, we get:

$$P(TestA + TestB+) = P * SeA * SeB + (1 - P) * (1 - SpA) * (1 - SpB)$$
$$P(TestA + TestB-) = P * SeA * (1 - SeB) + (1 - P) * (1 - SpA) * SpB$$
$$P(TestA - TestB+) = P * (1 - SeA) * SeB + (1 - P) * SpA * (1 - SpB)$$
$$P(TestA - TestB-) = P * (1 - SeA) * (1 - SeB) + (1 - P) * SpA * SpB$$

When applying two diagnostic tests on a given population, the data generated can be presented very simply as the number of individuals in each cell of the 2x2 table presenting the cross-classified results of the two tests:

Thus, the likelihood function that could link the observed data (*n1*, *n2*, *n3*,

n4) to the unknown parameters (P, SeA, SeB, SpA, SpB) could be defined as follows, using a multinomial distribution:

$n[1:4] \sim dmulti(P[1:4], n)$

$P1 = P * SeA * SeB + (1 - P) * (1 - SpA) * (1 - SpB)$
$P2 = P * SeA * (1 - SeB) + (1 - P) * (1 - SpA) * SpB$
$P3 = P * (1 - SeA) * SeB + (1 - P) * SpA * (1 - SpB)$
$P4 = P * (1 - SeA) * (1 - SeB) + (1 - P) * SpA * SpB$

Again, if we put that in words, the number of individuals in cell *n1* (i.e., TestA+ and TestB+) is determined by the number of individuals tested (n) and the probability (*P1*) of falling in that cell. The latter itself depends on the true prevalence of disease in the population (P), and on each of the two tests sensitivity (SeA and SeB) and specificity (SpA and SpB). We can use similar reasoning for *n2*, *n3*, and *n4*. Finally, the multinomial distribution would further impose that the four probabilities (*P1*, *P2*, *P3*, and *P4*) have to sum up to 1.0 (i.e., 100%).

In such a model, we will have five unknown parameters to estimate (P, SeA, SeB, SpA, SpB), but only four equations. Thus, the model will be "non-identifiable", and we will need to provide informative priors on, at least, two parameters. Often, we will provide priors on the Se and Sp of one of the tests, and/or on disease prevalence.

To run such a model, we simply need to provide a dataset where *n1*, *n2*, *n3*, and *n4* are listed (in that order).

```
#n is of the form : (TestA pos and TestB pos), (TestA pos and TestB neg), (TestA neg and TestB po
datalist <- list(n=c(138,1,10,113))
```

Moreover, we will need the following information to generate the a and b shape parameters of the various distributions. These values will be included in the text file presenting the `JAGS` model. Note that we used below the `paste0()` function to produce a model in which we could modify the tests "labels". We could thus use the `JAGS` model that will be described next as a generic model for various diagnostic tests, and still obtain "meaningful" outputs.

```
#We could first create labels for TestA and TestB
TestA <- "US"
TestB <- "PAG"

#Provide information for the prior distributions (all beta distributions) for the 5 unknown paran
#Below, we have used informative priors for Prev, and Se and Sp of Test A. The values used are no
Prev.shapea <- 4.2          #a shape parameter for Prev
Prev.shapeb <- 5.4          #b shape parameter for Prev

Se.TestA.shapea <- 131      #a shape parameter for Se of TestA
Se.TestA.shapeb <- 15       #b shape parameter for Se of TestA
```

```
Sp.TestA.shapea <- 100      #a shape parameter for Sp of TestA
Sp.TestA.shapeb <- 6        #b shape parameter for Sp of TestA

Se.TestB.shapea <- 1        #a shape parameter for Se of TestB
Se.TestB.shapeb <- 1        #b shape parameter for Se of TestB
Sp.TestB.shapea <- 1        #a shape parameter for Sp of TestB
Sp.TestB.shapeb <- 1        #b shape parameter for Sp of TestB

#We will also need the total number of individuals tested (n)
n <- sapply(datalist, sum)
```

With that, we have everything that is needed to write the JAGS model.

```
#Create the JAGS text file
model_2tests_1pop_indep <- paste0("model{

#=== LIKELIHOOD ===#

  n[1:4] ~ dmulti(p[1:4], ",n,")
  p[1] <- Prev*Se_", TestA, "*Se_", TestB, " + (1-Prev)*(1-Sp_", TestA, ")*(1-Sp_", Tes
  p[2] <- Prev*Se_", TestA, "*(1-Se_", TestB, ") + (1-Prev)*(1-Sp_", TestA, ")*Sp_", Te
  p[3] <- Prev*(1-Se_", TestA, ")*Se_", TestB, " + (1-Prev)*Sp_", TestA, "*(1-Sp_", Tes
  p[4] <- Prev*(1-Se_", TestA, ")*(1-Se_", TestB, ") + (1-Prev)*Sp_", TestA, "*Sp_", Te

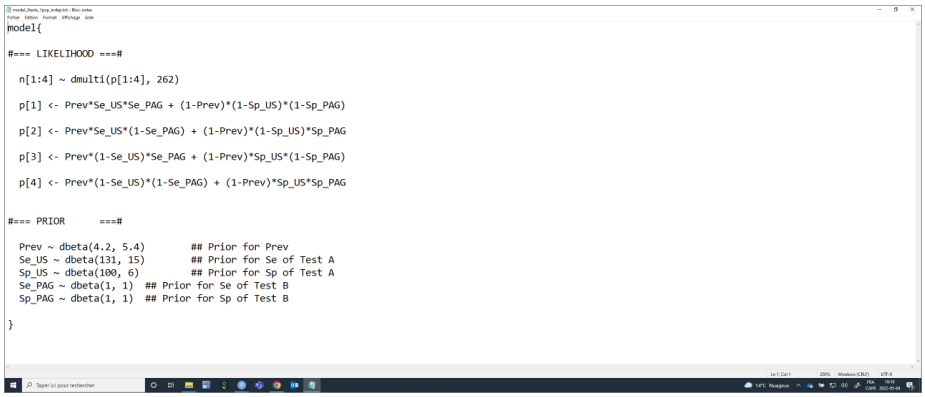
#=== PRIOR  ===#

  Prev ~ dbeta(",Prev.shapea,", ",Prev.shapeb,")    ## Prior for Prev
  Se_", TestA, " ~ dbeta(",Se.TestA.shapea,", ",Se.TestA.shapeb,")  ## Prior for Se of
  Sp_", TestA, " ~ dbeta(",Sp.TestA.shapea,", ",Sp.TestA.shapeb,")  ## Prior for Sp of
  Se_", TestB, " ~ dbeta(",Se.TestB.shapea,", ",Se.TestB.shapeb,")  ## Prior for Se of
  Sp_", TestB, " ~ dbeta(",Sp.TestB.shapea,", ",Sp.TestB.shapeb,")  ## Prior for Sp of

}")

#write as a text (.txt) file
write.table(model_2tests_1pop_indep,
            file="model_2tests_1pop_indep.txt",
            quote=FALSE,
            sep="",
            row.names=FALSE,
            col.names=FALSE)
```

With this code, we could simply modify the labels for Test A and Test B, and the shape parameters for the prior distributions and the text file with the JAGS model will automatically be updated. Currently, it looks like this:



```
model{

#=== LIKELIHOOD ===#

  n[1:4] ~ dmulti(p[1:4], 262)

  p[1] <- Prev*Se_US*Se_PAG + (1-Prev)*(1-Sp_US)*(1-Sp_PAG)

  p[2] <- Prev*Se_US*(1-Se_PAG) + (1-Prev)*(1-Sp_US)*Sp_PAG

  p[3] <- Prev*(1-Se_US)*Se_PAG + (1-Prev)*Sp_US*(1-Sp_PAG)

  p[4] <- Prev*(1-Se_US)*(1-Se_PAG) + (1-Prev)*Sp_US*Sp_PAG


#=== PRIOR      ===#

  Prev ~ dbeta(4.2, 5.4)       ## Prior for Prev
  Se_US ~ dbeta(131, 15)       ## Prior for Se of Test A
  Sp_US ~ dbeta(100, 6)        ## Prior for Sp of Test A
  Se_PAG ~ dbeta(1, 1)  ## Prior for Se of Test B
  Sp_PAG ~ dbeta(1, 1)  ## Prior for Sp of Test B

}
```

Figure 5.2: **Text file with the model for 2 independent diagnostic tests in 1 single population.**

We will also need to provide a list of initial values (one per Markov chain) for all unknown parameters.

```
#Initializing values for the parameters Prev, and the Ses and Sps of the two tests for the 3 chai
inits <- list(list(Prev=0.50,
                   Se_US=0.90,
                   Sp_US=0.90,
                   Se_PAG=0.90,
                   Sp_PAG=0.90),

                   list(Prev=0.70,
                   Se_US=0.10,
                   Sp_US=0.10,
                   Se_PAG=0.10,
                   Sp_PAG=0.10),

                   list(Prev=0.50,
                   Se_US=0.50,
                   Sp_US=0.50,
                   Se_PAG=0.50,
                   Sp_PAG=0.50)
                   )
```

Now we can run the model using `jags()` function.

```
library(R2jags)
library(coda)
```

```
#Run the Bayesian model
bug.out <- jags(data=datalist,
                model.file="model_2tests_1pop_indep.txt",
                parameters.to.save=c("Prev", "Se_US", "Sp_US", "Se_PAG", "Sp_PAG"),
                n.chains=3,
                inits=inits,
                n.iter=11000,
                n.burnin=1000,
                n.thin=1,
                DIC=FALSE)
```

```
## Compiling model graph
##    Resolving undeclared variables
##    Allocating nodes
## Graph information:
##    Observed stochastic nodes: 1
##    Unobserved stochastic nodes: 5
##    Total graph size: 32
##
## Initializing model
```

Then we could produce the diagnostic plots, compute the ESS, and print out our results as we did previously (results not shown).

```
library(mcmcplots)
bug.mcmc <- as.mcmc(bug.out)
mcmcplot(bug.mcmc, title="Diagnostic plots")
effectiveSize(bug.mcmc)
print(bug.out, digits.summary=3)
```

Or we could ask to plot the prior and posterior distributions of one (or many) of the parameters. Here, we asked for the prior and posterior distributions of *US* sensitivity.

```
#Plot the posterior distribution
plot(density(x=bug.out$BUGSoutput$sims.list$Se_US),
     main="US sensitivity",
     xlab="Sensitivity", ylab="Density",
     )

#plot the prior distribution
curve(dbeta(x, shape1=Se.TestA.shapea, shape2=Se.TestA.shapeb), from=0, to=1,
      lty=2,
      col="red",
      add=TRUE)
```

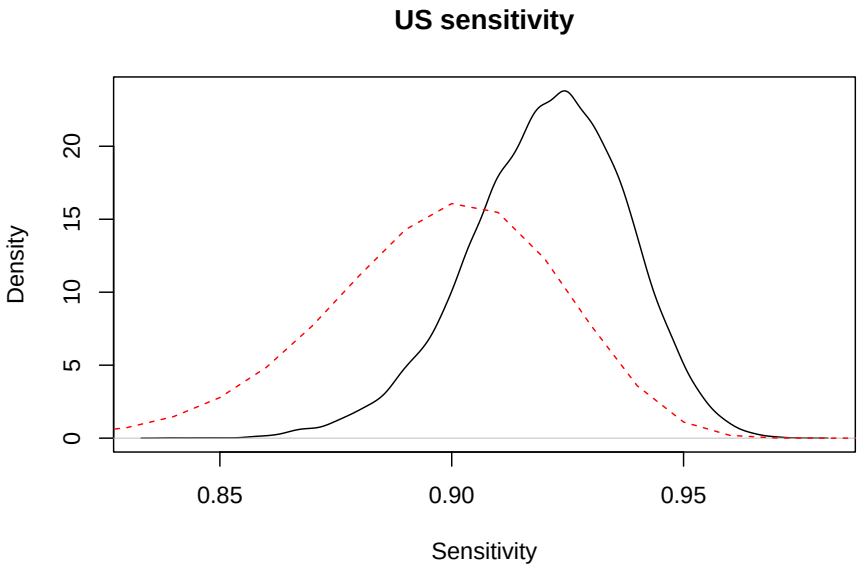


Figure 5.3: Prior (dashed red line) and posterior (full black line) distribution of US sensitivity.

5.2 Allowing for conditional dependence

We will not go into the exact details, but Dendukuri and Joseph (2001) developed a highly generalisable model structure that can account for conditional dependence between diagnostic tests. To allow for conditional dependence between tests, the likelihood function would be modified as follows, by adding covariance terms representing the conditional dependence between tests:

$n[1:4] \sim dmulti(P[1:4], n)$

$P1 = P * (SeA * SeB + covp) + (1 - P) * ((1 - SpA) * (1 - SpB) + covn)$
$P2 = P * (SeA * (1 - SeB) - covp) + (1 - P) * ((1 - SpA) * SpB - covn)$
$P3 = P * ((1 - SeA) * SeB - covp) + (1 - P) * (SpA * (1 - SpB) - covn)$
$P3 = P * ((1 - SeA) * (1 - SeB) + covp) + (1 - P) * (SpA * SpB + covn)$

Thus, a small quantity ($covp$) is added to the sensitivities of the tests when they agree on a positive individual, and that same quantity is substracted when the tests disagree. $covp$ thus represents the positive covariance between tests. Similarly, $covn$ represents the negative covariance between tests. The effect of these covariances, if greater than zero, will be to penalize our estimates of the tests sensitivities and specificities to account for their conditional dependence. Note that having covariance terms equal to 0.0 (i.e., conditional independence)

is a possibility. This model, therefore, **is not imposing a conditional dependence between tests, it rather allows for conditional dependence**.

Moreover, Dendukuri and Joseph (2001) proposed constraints on these covariance terms based on the inferred test sensitivities and specificities. This latter aspect allows to specify "natural limits" for the covariance terms prior distributions. Briefly, they demonstrated that:

- $covp > (1 - SeA) * (SeB - 1)$
- $covp <$ (the minimum between SeA and SeB) minus $SeA * SeB$
- $covn > (SpA - 1) * (1 - SpB)$
- $covn <$ (the minimum between SpA and SpB) minus $SpA * SpB$

Thus, we can use these as the minimum and maximum for *covp* and *covn*. For instance, we could use uniform distributions with these limits to describe the prior distributions for *covp* and *covn*. Our LCM would thus become:

```
#Create the JAGS text file
model_2tests_1pop_dep <- paste0("model{

#=== LIKELIHOOD ===#

  n[1:4] ~ dmulti(p[1:4], ",n,")
  p[1] <- Prev*(Se_", TestA, "*Se_", TestB, " + covp) + (1-Prev)*((1-Sp_", TestA, ")*(
  p[2] <- Prev*(Se_", TestA, "*(1-Se_", TestB, ") - covp) + (1-Prev)*((1-Sp_", TestA, "
  p[3] <- Prev*((1-Se_", TestA, ")*Se_", TestB, " - covp) + (1-Prev)*(Sp_", TestA, "*(
  p[4] <- Prev*((1-Se_", TestA, ")*(1-Se_", TestB, ") + covp) + (1-Prev)*(Sp_", TestA,

#=== PRIOR  ===#

  Prev ~ dbeta(",Prev.shapea,", ",Prev.shapeb,")    ## Prior for Prev
  Se_", TestA, " ~ dbeta(",Se.TestA.shapea,", ",Se.TestA.shapeb,")  ## Prior for Se of
  Sp_", TestA, " ~ dbeta(",Sp.TestA.shapea,", ",Sp.TestA.shapeb,")  ## Prior for Sp of
  Se_", TestB, " ~ dbeta(",Se.TestB.shapea,", ",Se.TestB.shapeb,")  ## Prior for Se of
  Sp_", TestB, " ~ dbeta(",Sp.TestB.shapea,", ",Sp.TestB.shapeb,")  ## Prior for Sp of

#=== CONDITIONAL DEPENDENCE STRUCTURE ===#

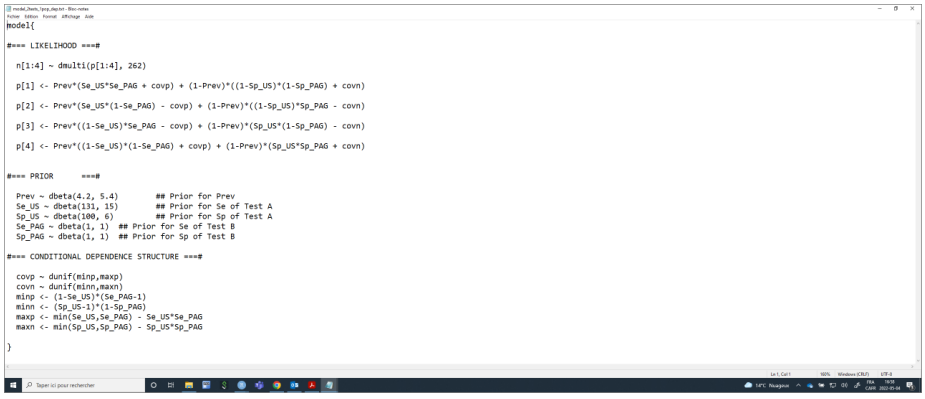
  covp ~ dunif(minp,maxp)
  covn ~ dunif(minn,maxn)
  minp <- (1-Se_", TestA, ")*(Se_", TestB, "-1)
  minn <- (Sp_", TestA, "-1)*(1-Sp_", TestB, ")
  maxp <- min(Se_", TestA, ",Se_", TestB, ") - Se_", TestA, "*Se_", TestB, "
```

```
  maxn <- min(Sp_", TestA, ",Sp_", TestB, ") - Sp_", TestA, "*Sp_", TestB, "

}")

#write as a text (.txt) file
write.table(model_2tests_1pop_dep,
            file="model_2tests_1pop_dep.txt",
            quote=FALSE,
            sep="",
            row.names=FALSE,
             col.names=FALSE)
```

Again, with this code, we could simply modify the labels for Test A and Test B, and the shape parameters for the prior distributions and the text file with the `JAGS` model will automatically be updated. Currently, it looks like this:



```
model{

#=== LIKELIHOOD ===#

  n[1:4] ~ dmulti(p[1:4], 262)

  p[1] <- Prev*(Se_US*Se_PAG + covp) + (1-Prev)*((1-Sp_US)*(1-Sp_PAG) + covn)

  p[2] <- Prev*(Se_US*(1-Se_PAG) - covp) + (1-Prev)*((1-Sp_US)*Sp_PAG - covn)

  p[3] <- Prev*((1-Se_US)*Se_PAG - covp) + (1-Prev)*(Sp_US*(1-Sp_PAG) - covn)

  p[4] <- Prev*((1-Se_US)*(1-Se_PAG) + covp) + (1-Prev)*(Sp_US*Sp_PAG + covn)


#=== PRIOR      ===#

  Prev ~ dbeta(4.2, 5.4)       ## Prior for Prev
  Se_US ~ dbeta(131, 15)       ## Prior for Se of Test A
  Sp_US ~ dbeta(100, 6)        ## Prior for Sp of Test A
  Se_PAG ~ dbeta(1, 1)  ## Prior for Se of Test B
  Sp_PAG ~ dbeta(1, 1)  ## Prior for Sp of Test B

#=== CONDITIONAL DEPENDENCE STRUCTURE ===#

  covp ~ dunif(minp,maxp)
  covn ~ dunif(minn,maxn)
  minp <- (1-Se_US)*(Se_PAG-1)
  minn <- (Sp_US-1)*(1-Sp_PAG)
  maxp <- min(Se_US,Se_PAG) - Se_US*Se_PAG
  maxn <- min(Sp_US,Sp_PAG) - Sp_US*Sp_PAG

}
```

Figure 5.4: **Text file with the model for 2 possibly conditionally dependent diagnostic tests in 1 single population.**

If we do not modify the other priors nor the dataset used previously we could run this model using the `jags()` function. However, we now have to add initial values for *covp* and *covn*. Also, note that we have added *covp* and *covn* to the list of parameters that we would like to monitor and we have increased substantially the number of iterations. The model allowing for conditional dependence between tests tends to be more difficult to solve. We will thus need a substantial number of iterations to get a decent ESS.

```
#Initializing values for the parameters Prev, and the Ses and Sps of the two tests for the 3 cha
inits <- list(list(Prev=0.50,
                   Se_US=0.90,
                   Sp_US=0.90,
```

```
              Se_PAG=0.90,
              Sp_PAG=0.90,
              covp=0,
              covn=0),

              list(Prev=0.70,
              Se_US=0.10,
              Sp_US=0.10,
              Se_PAG=0.10,
              Sp_PAG=0.10,
              covp=0.01,
              covn=0.01),

              list(Prev=0.50,
              Se_US=0.50,
              Sp_US=0.50,
              Se_PAG=0.50,
              Sp_PAG=0.50,
              covp=0.05,
              covn=0.05)
              )


library(R2jags)
library(coda)

#Run the Bayesian model
bug.out <- jags(data=datalist,
               model.file="model_2tests_1pop_dep.txt",
               parameters.to.save=c("Prev", "Se_US", "Sp_US", "Se_PAG", "Sp_PAG", "covp
               n.chains=3,
               inits=inits,
               n.iter=101000,
               n.burnin=1000,
               n.thin=1,
               DIC=FALSE)
```

```
## Compiling model graph
##    Resolving undeclared variables
##    Allocating nodes
## Graph information:
##    Observed stochastic nodes: 1
##    Unobserved stochastic nodes: 7
##    Total graph size: 58
##
```

```
## Initializing model
```

Then we could produce the diagnostic plots, compute the ESS, and print out our results as we did previously (the diagnostic plots are not shown).

```
library(mcmcplots)
bug.mcmc <- as.mcmc(bug.out)
mcmcplot(bug.mcmc, title="Diagnostic plots")
effectiveSize(bug.mcmc)
print(bug.out, digits.summary=3)
```

```
effectiveSize(bug.mcmc)
```

```
##      Prev    Se_PAG     Se_US    Sp_PAG     Sp_US      covn      covp
## 15812.487  2167.916  6168.379  7883.295  6853.168  6822.977  2149.342
```

```
print(bug.out, digits.summary=3)
```

```
## Inference for Bugs model at "model_2tests_1pop_dep.txt", fit using jags,
##  3 chains, each with 101000 iterations (first 1000 discarded)
##  n.sims = 300000 iterations saved. Running time = 6.46 secs
##        mu.vect sd.vect  2.5%   25%   50%   75% 97.5%  Rhat n.eff
## Prev     0.555   0.039 0.480 0.529 0.555 0.581 0.632 1.001 18000
## Se_PAG   0.949   0.035 0.872 0.925 0.954 0.979 0.998 1.002  2100
## Se_US    0.906   0.023 0.856 0.892 0.908 0.922 0.944 1.001  9700
## Sp_PAG   0.923   0.040 0.836 0.897 0.927 0.953 0.990 1.001 16000
## Sp_US    0.950   0.021 0.902 0.938 0.953 0.965 0.982 1.001 14000
## covn     0.033   0.020 0.002 0.018 0.031 0.045 0.077 1.001 22000
## covp     0.037   0.029 0.000 0.012 0.033 0.058 0.099 1.002  2000
##
## For each parameter, n.eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor (at convergence, Rhat=1).
```

The ESS is still just around 2,000 for some parameters (e.g., *Se_PAG*), if it was for publication, we would possibly let it run for more iterations.

The model proposed by Dendukuri and Joseph (2001) can be used to model conditional dependence between two tests only. In Chapter 9 we will see that we can also used models where > 2 tests are compared (e.g., 3, 4, 5 tests). In such cases, the method described above could be used to evaluate pairwise dependence between tests (i.e., *TestA vs. TestB*, then *TestA vs. TestC*, and, finally, *TestB vs. TestC*).

Chapter 6

Exercise 2 - Two tests

6.1 Questions

Open up the R project for Exercise 2 (i.e., the R project file named *Exercise 2.Rproj*).

In the Exercise 2 folder, you will find partially completed R scripts. To answer the questions, try to complete the missing parts (they are highlighted by a **#TO COMPLETE#** comment). We also provided complete R scripts, but try to work it out on your own first!

1. Start from the partially completed *Question 1.R* script located in Exercise 2 folder. We could try to estimate the Se and Sp of the PAG-based ELISA test when compared to US. We could, however, explicitly acknowledge that the US exam is not a gold-standard test. At first, we could assume that these tests are conditionally independent. We will use data from one population (herd #1). In herd #1, the cross-tabulated PAG by US results were as follows:

Table. Cross-classified results of the PAG and US tests in herd #1.

	US+	**US-**	**Total**
PAG+	138	10	148
PAG-	1	113	114
Total	139	123	262

How many unknown parameters do you have to estimate and how many degrees of freedom do you have? Therefore, how many informative priors will you need? Estimate PAG's Se and Sp, and true pregnancy prevalence using informative priors on US' Se and Sp, and on pregnancy's prevalence. Compare your results

to those of Exercise 1 (question 4) where you estimated PAG's Se and Sp, but assuming that US was a gold standard test.

Some literature to help you with Informative priors:
In 2006, Romano et al. reported, for a US exam conducted between 28-45 days post-insemination:
-Se = 0.90 (95CI: 0.85, 0.95)
-Sp = 0.95 (95CI: 0.90, 0.97)

Moreover, in a recent study conducted on 30 Canadian herds they reported an expected pregnancy prevalence of 42% with 97.5% percentile of 74%.

2. Start from the `R` script that you completed in Question 1. Now run the same model, but use vague priors on all unknown parameters. Do you think it will work? What do you observe regarding the Markov chains?

3. Start from the partially completed *Question 3.R* script located in Exercise 2 folder. Now, we could consider that some conditional dependency exist between the two tests. For instance, it was reported that PAG concentration is higher in first parity cows (Ricci et al., 2015, Mercadante et al., 2016). US exams may also be easier to conduct on first parity cows and, thus, Se of both tests may be higher in first parity cows and lower in older cows. Build a model where conditional dependency is a possibility. How many unknown parameters and degrees of freedom do you have? Are your Markov chains behaving OK? What do you observe? Try to play with the number of iterations to reach an effective sample size (**ESS**) > 3,000. How are your results differing from that of question 1 (where you assumed conditional independence)?

6.2 Answers

1. We could try to estimate the Se and Sp of the PAG-based ELISA test using US as comparison. We could, however, explicitly acknowledge that the US exam is not a gold-standard test. At first, we could assume that these tests are conditionally independent. How many unknown parameters do you have to estimate and how many degrees of freedom do you have? Therefore, how many informative priors will you need? Estimate PAG's Se and Sp, and true pregnancy prevalence using informative priors on US' Se and Sp, and on pregnancy's prevalence. Compare your results to those of Exercise 1 (question 4) where you estimated PAG's Se and Sp, but assuming that US was a gold standard test.

Answer: We now have 5 unknown parameters (PAG's *Se* and *Sp*, US' *Se* and *Sp*, and pregnancy prevalence, *P*) and we have 3 degrees of freedom (i.e., one 2x2 table where three cells are providing valuable information and the content of the 4th cell can be deduced from the total number of observations minus the content of the other three cells). Thus, we need informative priors for at least

2 parameters. As suggested, we will use informative priors for 3 parameters (US' *Se* and *Sp*, and *P*), so the model is identifiable.

Table. Results assuming **US is not perfect**

Parameters	Median	95CI
PAG Se	0.993	0.968, 1.000
PAG Sp	0.979	0.922, 0.999
US Se	0.923	0.885, 0.955
US Sp	0.970	0.941, 0.987
Prevalence	0.554	0.493, 0.614

As a comparison, in question 4 of Exercise 1, where we considered that US was a perfect test, we assumed that US *Se* and *Sp* was 1.0 and we estimated accuracy of PAG as:

Table. Results assuming **US is perfect**

Parameters	Median	95CI
PAG Se	0.988	0.961, 0.998
PAG Sp	0.915	0.859, 0.955

Thus, the 10 cows that were PAG+ and US- where all considered to be a failure of the PAG test (i.e., False+ on PAG). This resulted in a lower estimated PAG specificity. Now that we indicated that US is not perfect, these disagreements are not exclusively attributed to the PAG test, hence the higher estimated PAG specificity.

2. Now run the same model, but use vague priors on all unknown parameters. Do you think it will work? What do you observe regarding the Markov chains?

Answer: Beforehand, we would say no, it will not work! We have 5 unknown parameters and only 3 degrees of freedom. The model is non-identifiable. With some data sets, though, it will sometimes work (if the number of potential solutions is very limited). This is the case here, there is so little disagreement between tests that the Markov chains were able to converge. However, they have a strange behaviour, for all parameters, we always have two chains converging in one area and another one converging at the exact opposite (see figure below). This is a case where we have a mirror solution. When such situation arise, we could consider adding informative priors for at least one other parameter (to make the model identifiable). However, remember that **mirror solutions may arise even in LCM that are identifiable**. In this latter case, we would have to exclude the Markov chain producing the "biologically impossible" results or to modify the set of initial values of the problematic Markov chain.

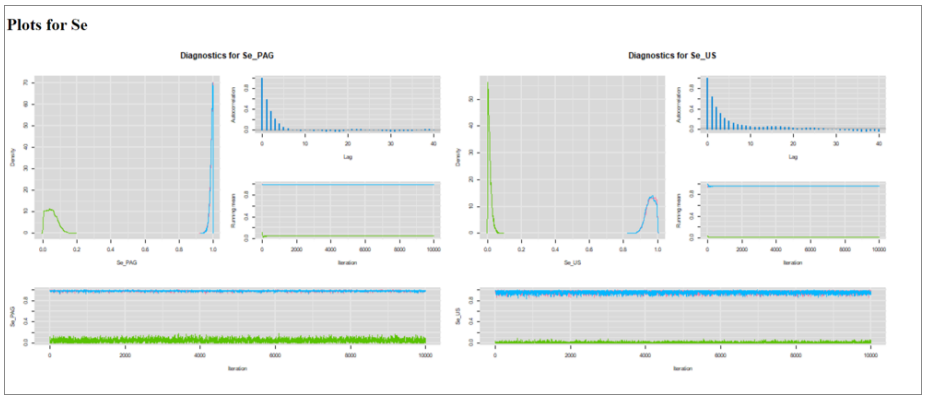


Figure 6.1: **Diagnostic plot - Mirror solution**

3. Build a model where conditional dependency is a possibility. How many unknown parameters and degrees of freedom do you have? Are your Markov chains behaving OK? What do you observe? Try to play with the number of iterations to reach an effective sample size (**ESS**) > 3,000. How are your results differing from that of question 1 (where you assumed conditional independence)?

Answer: We have 7 unkown parameters (PAG's *Se* and *Sp*, US' *Se* and *Sp*, *P*, *covp*, and *covn*). We still only have 3 degrees of freedom, so we need informative priors on 4 parameters. We can use the Romano et al. (2006) derived priors for US' *Se* and *Sp*, and use our pregnancy prevalence informative prior. We can use the natural bounds of *covp* and *covn* proposed by Dendukuri and Joseph (2001) as priors for these parameters.

We initially ran the LCM for 11,000 iterations (with a burn-in of 1,000). The Markov chains were still cycling up and down for some of the parameters (*Se_PAG* and *covp* were the worst; see *Se_PAG* below). But it is still cycling in a very tight space (i.e., check the Y-axis scale). Autocorrelation seems to be a problem (especially for *Se_pag* and *covp*, again).

The ESS for these two parameters were both around 170. So, despite the 30,000 values (3 chains times 11,000 iterations, minus the 1,000 burn-in of each chain) stored during the Monte Carlo simulation, there is just the equivalent of 170 independent values. This would be quite a small sample to report with precision the median, 2.5th, and 97.5th estimates. Using the exact same model, but with a number of iteration of 201,000, we obtained ESS >3,500 for all parameters.

Table. Results of the LCM assuming or not conditional independence between diagnostic tests.

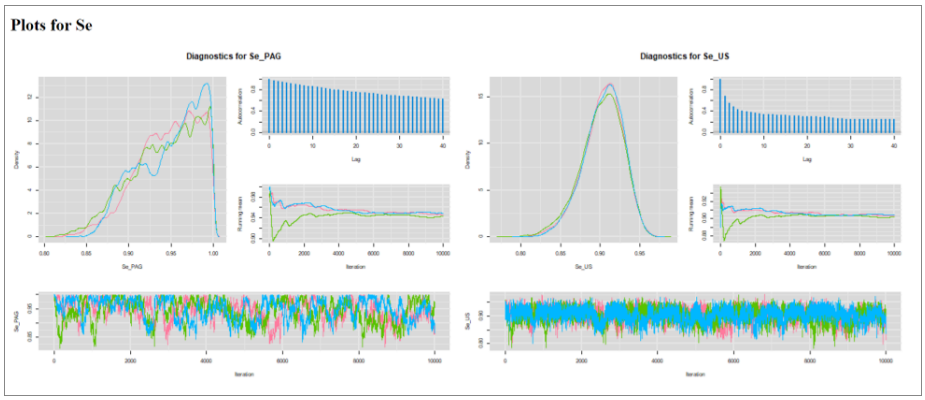


Figure 6.2: **Diagnostic plot - 11,000 iterations**

	Conditional independence LCM		Conditional dependence LCM	
Parameters	**Median**	**95CI**	**Median**	**95CI**
PAG Se	0.993	0.968, 1.00	0.952	0.866, 0.998
PAG Sp	0.979	0.922, 0.999	0.924	0.833, 0.988
US Se	0.923	0.885, 0.955	0.906	0.847, 0.947
US Sp	0.970	0.941, 0.987	0.951	0.899, 0.981
covp	NA	NA	0.035	0.000, 0.104
covn	NA	NA	0.032	0.002, 0.079
Prevalence	0.554	0.493, 0.614	0.555	0.479, 0.634

When we compare the LCM assuming conditional independence *vs.* the one allowing conditional dependence, we can see that the tests' Se and Sp estimates are slightly lower in the model allowing conditional dependence. Indeed, in that model the covariance terms, *covp* and *covn*, where small, but positive. Remember, in the model allowing for conditional dependence, the agreement between tests is now due to the PAG and US accuracy parameters (e.g., their Se) **plus a small value** due to their covariance (e.g., *covp*). Therefore, the tests accuracy parameters estimates will be reduced as their covariance is increased.

Chapter 7

Multiple populations

In some situations, the diagnostic tests will be apply to two or more populations with different disease prevalence. In such cases, it could be reasonable to assume that:

- The prevalence varies from one population to another
- The tests Se and Sp are constant across populations

It could then be advantageous to model the different populations simultaneously in the same LCM as described by Hui and Walter (1980). With two populations, we would have six unknown parameters (*SeA* and *SpA*, *SeB* and *SpB*, and each population's prevalence, *P1* and *P2*). When conducting two diagnostic tests in two populations, the data generated can be presented as two 2x2 tables presenting the cross-classified results of the two tests.

Table. Cross-classified results from two diagnostic tests in two populations.

Population 1			**Population 2**		
	Test A+	**Test A-**		**Test A+**	**Test A-**
Test B+	Pop1 n1	Pop1 n3	**Test B+**	Pop2 n1	Pop2 n3
Test B-	Pop1 n2	Pop1 n4	**Test B-**	Pop2 n2	Pop2 n4

We can see from these 2x2 tables that we have 6 degrees of freedom available. Indeed, in each of the 2x2 table, we have three cells that do contribute a valuable

information and the information in the 4th cell can be "guessed" using the total number of observations minus the content of the other three cells. The model proposed by Hui and Walter (1980) is, therefore, barely identifiable.

Cheung et al. (2021) have described how the number of unknown parameters and degrees of freedom vary as function of the number of populations studied when comparing two conditionally independent diagnostic tests (see below).

Table. Number of degrees of freedom and of unknown parameters when comparing two conditionally independent diagnostic tests across various number of populations (adapted from Cheung et al. (2021)).

Number of Populations	**Number of unknown parameters (np)**	**Number of degrees of freedom (df)**	**Minimum number of informative priors needed**
1	5	3	2
2	6	6	0
3	7	9	0
4	8	12	0
p	4+p	3*p	if df < np, then np-df

7.1 Likelihood function with 2 populations

With two populations, the likelihood function that could be used to link the observed data (*Pop1 n1*, *Pop1 n2*, ..., *Pop2 n4*) to the unknown parameters (*SeA* and *SpA*, *SeB* and *SpB*, and *P1* and *P2*) can be described as follows:

$$Ppop1[1:4] \sim dmulti(Pop1[1:4], nPop1)$$
$$Ppop1_1 = P1 * SeA * SeB + (1 - P1) * (1 - SpA) * (1 - SpB)$$
$$Ppop1_2 = P1 * SeA * (1 - SeB) + (1 - P1) * (1 - SpA) * SpB$$
$$Ppop1_3 = P1 * (1 - SeA) * SeB + (1 - P1) * SpA * (1 - SpB)$$
$$Ppop1_4 = P1 * (1 - SeA) * (1 - SeB) + (1 - P1) * SpA * SpB$$

$$Ppop2[1:4] \sim dmulti(Pop2[1:4], nPop2)$$
$$Ppop2_1 = P2 * SeA * SeB + (1 - P2) * (1 - SpA) * (1 - SpB)$$
$$Ppop2_2 = P2 * SeA * (1 - SeB) + (1 - P2) * (1 - SpA) * SpB$$
$$Ppop2_3 = P2 * (1 - SeA) * SeB + (1 - P2) * SpA * (1 - SpB)$$
$$Ppop2_4 = P2 * (1 - SeA) * (1 - SeB) + (1 - P2) * SpA * SpB$$

Where $Ppop1_1$ to $Ppop1_4$ and $Ppop2_1$ to $Ppop2_4$ are the probabilities of falling in a given cell of the 2x2 table in population 1 and population 2, respectively; and $nPop1$ and $nPop2$ are the number of individuals tested in population 1 and 2, respectively. Thus, if you look carefully, **we simply duplicated the likelihood function used before and we added subscripts to differentiate the cells, probabilities, and disease prevalence of the two populations.**

Pretty cool right? If you have three, four, etc populations, you can then extend the likelihood function to include them.

7.2 Organizing the data with 2 populations

To run such a model, we simply need to provide, for each population, a dataset where *n1*, *n2*, *n3*, and *n4* are listed (in that order). For instance, if we use herd #1 and herd #2 from the PAG *vs.* US study, we will have the following 2x2 tables:

Table. Cross-classified results from the PAG and US diagnostic tests in herd #1 and herd #2.

Herd 1			**Total**	**Herd 2**			**Total**
	US+	**US-**			**US+**	**US-**	
PAG+	138	10		**PAG+**	71	8	
PAG-	1	113		**PAG-**	0	69	
Total			262	**Total**			148

The dataset could, thus be created as follows:

```
#n is of the form : (TestA pos and TestB pos), (TestA pos and TestB neg), (TestA neg and TestB po
datalist <- list(Pop1=c(138,1,10,113),
                 Pop2=c(71, 0, 8, 69)
                 )
```

7.3 The JAGS model with 2 populations

We could provide the values that will be used to described the prior distributions as we did before (so they are included in the model text file). The only difference is that we now have two prevalence parameters (*P1* and *P2*) to describe. In the example below, I will use vague priors for all parameters (just because we can!).

```
#We could first create labels for TestA and TestB
TestA <- "US"
TestB <- "PAG"

#Provide information for the prior distributions (all beta distributions) for the 6 unknown param
Prev1.shapea <- 1          #a shape parameter for Prev in population 1
Prev1.shapeb <- 1          #b shape parameter for Prev in population 1
```

```
Prev2.shapea <- 1          #a shape parameter for Prev in population 2
Prev2.shapeb <- 1          #b shape parameter for Prev in population 2

Se.TestA.shapea <- 1      #a shape parameter for Se of TestA
Se.TestA.shapeb <- 1      #b shape parameter for Se of TestA
Sp.TestA.shapea <- 1      #a shape parameter for Sp of TestA
Sp.TestA.shapeb <- 1      #b shape parameter for Sp of TestA

Se.TestB.shapea <- 1      #a shape parameter for Se of TestB
Se.TestB.shapeb <- 1      #b shape parameter for Se of TestB
Sp.TestB.shapea <- 1      #a shape parameter for Sp of TestB
Sp.TestB.shapeb <- 1      #b shape parameter for Sp of TestB

#We will also need the total number of individuals tested in each population (nPop1 an
n <- sapply(datalist, sum)
nPop1 <- n[1]
nPop2 <- n[2]
```

With that, we have everything that is needed to write the JAGS model.

```
#Create the JAGS text file
model_2tests_2pop_indep <- paste0("model{

#=== LIKELIHOOD ===#

  #=== POPULATION 1 ===#
  Pop1[1:4] ~ dmulti(p1[1:4], ",nPop1,")
  p1[1] <- Prev1*Se_", TestA, "*Se_", TestB, " + (1-Prev1)*(1-Sp_", TestA, ")*(1-Sp_",
  p1[2] <- Prev1*Se_", TestA, "*(1-Se_", TestB, ") + (1-Prev1)*(1-Sp_", TestA, ")*Sp_"
  p1[3] <- Prev1*(1-Se_", TestA, ")*Se_", TestB, " + (1-Prev1)*Sp_", TestA, "*(1-Sp_",
  p1[4] <- Prev1*(1-Se_", TestA, ")*(1-Se_", TestB, ") + (1-Prev1)*Sp_", TestA, "*Sp_"

  #=== POPULATION 2 ===#
  Pop2[1:4] ~ dmulti(p2[1:4], ",nPop2,")
  p2[1] <- Prev2*Se_", TestA, "*Se_", TestB, " + (1-Prev2)*(1-Sp_", TestA, ")*(1-Sp_",
  p2[2] <- Prev2*Se_", TestA, "*(1-Se_", TestB, ") + (1-Prev2)*(1-Sp_", TestA, ")*Sp_"
  p2[3] <- Prev2*(1-Se_", TestA, ")*Se_", TestB, " + (1-Prev2)*Sp_", TestA, "*(1-Sp_",
  p2[4] <- Prev2*(1-Se_", TestA, ")*(1-Se_", TestB, ") + (1-Prev2)*Sp_", TestA, "*Sp_"


#=== PRIOR  ===#

  Prev1 ~ dbeta(",Prev1.shapea,", ",Prev1.shapeb,")     ## Prior for Prevalence in popu
  Prev2 ~ dbeta(",Prev2.shapea,", ",Prev2.shapeb,")     ## Prior for Prevalence in popu
  Se_", TestA, " ~ dbeta(",Se.TestA.shapea,", ",Se.TestA.shapeb,")  ## Prior for Se of
```

```
  Sp_", TestA, " ~ dbeta(",Sp.TestA.shapea,", ",Sp.TestA.shapeb,")  ## Prior for Sp of Test A
  Se_", TestB, " ~ dbeta(",Se.TestB.shapea,", ",Se.TestB.shapeb,")  ## Prior for Se of Test B
  Sp_", TestB, " ~ dbeta(",Sp.TestB.shapea,", ",Sp.TestB.shapeb,")  ## Prior for Sp of Test B

}")

#write as a text (.txt) file
write.table(model_2tests_2pop_indep,
            file="model_2tests_2pop_indep.txt",
            quote=FALSE,
            sep="",
            row.names=FALSE,
            col.names=FALSE)
```

With this code, you could, again, simply modify the labels for Test A and Test B, and the shape parameters for the prior distributions. The text file with the `JAGS` model will automatically be updated. Currently, it looks like this:

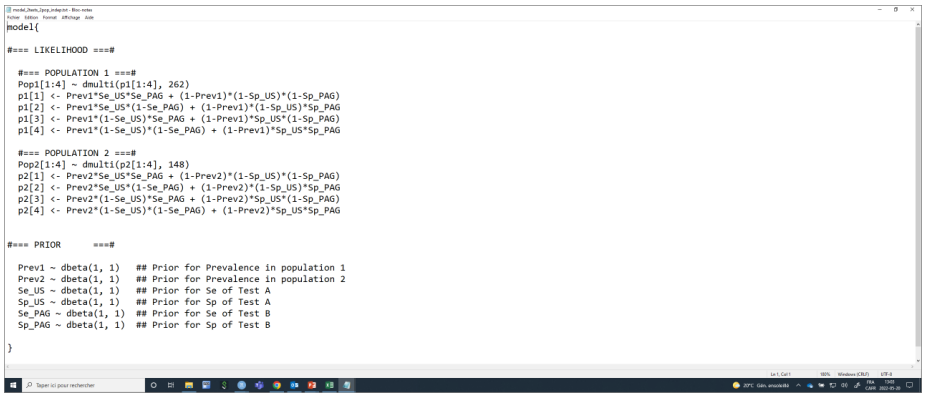


Figure 7.1: **Text file with the model for 2 independent diagnostic tests applied to two populations.**

Again, we will need to provide a list of initial values (one per Markov chain) for all unknown parameters. Careful again, we now have two prevalence (*Prev1* and *Prev2*).

```
#Initializing values for the parameters Prev, and the Ses and Sps of the two tests for the 3 chai
inits <- list(list(Prev1=0.50,
                   Prev2=0.50,
                   Se_US=0.90,
                   Sp_US=0.90,
                   Se_PAG=0.90,
```

```
              Sp_PAG=0.90),

              list(Prev1=0.60,
              Prev2=0.60,
              Se_US=0.80,
              Sp_US=0.80,
              Se_PAG=0.80,
              Sp_PAG=0.80),

              list(Prev1=0.40,
              Prev2=0.40,
              Se_US=0.70,
              Sp_US=0.70,
              Se_PAG=0.70,
              Sp_PAG=0.70)
              )
```

We can run the model using `jags()` function as seen before.

```
library(R2jags)
library(coda)

#Run the Bayesian model
bug.out <- jags(data=datalist,
                model.file="model_2tests_2pop_indep.txt",
                parameters.to.save=c("Prev1", "Prev2", "Se_US", "Sp_US", "Se_PAG", "Sp_
                n.chains=3,
                inits=inits,
                n.iter=11000,
                n.burnin=1000,
                n.thin=1,
                DIC=FALSE)
```

Then we could produce the diagnostic plots, compute the ESS, and print out our results as we did previously (results not shown).

```
library(mcmcplots)
bug.mcmc <- as.mcmc(bug.out)
mcmcplot(bug.mcmc, title="Diagnostic plots")
effectiveSize(bug.mcmc)
print(bug.out, digits.summary=3)
```

7.4 Conditional dependence

As discussed before, an important assumption of the LCM described in the previous section is that diagnostic tests are conditionally independent. This assumption, however, can be relaxed using the method proposed by Dendukuri and Joseph (2001). Note that, we are now increasing the number of unknown parameters and we will, therefore, have to provide additional informative distributions.

Table. Number of degrees of freedom and of unknown parameters when comparing two **conditionally dependent** diagnostic tests across various number of populations (adapted from Cheung et al. (2021)).

Number of Populations	Number of unknown parameters (np)	Number of degrees of freedom (df)	Minimum number of informative priors needed
1	7	3	4
2	8	6	2
3	9	9	0
4	10	12	0
p	4+2+p	3*p	if df < np, then np-df

For this example, in addition to specifying priors for *covp* and *covn*, we will add informative priors on US *Se* and *Sp*.

```
library(epiR)
# US sensitivity ------------------------------------------------------------
# Sensitivity of US:  Mode=0.90, and we are 97.5% sure >0.85
Se.US <- epi.betabuster(mode=0.90, conf=0.975, imsure="greater than", x=0.85)

# US specificity ------------------------------------------------------------
# Specificity of US:  Mode=0.95, and we are 97.5% sure >0.90
Sp.US <- epi.betabuster(mode=0.95, conf=0.975, imsure="greater than", x=0.90)

Se.TestA.shapea <- Se.US$shape1     #a shape parameter for Se of TestA
Se.TestA.shapeb <- Se.US$shape2     #b shape parameter for Se of TestA
Sp.TestA.shapea <- Sp.US$shape1     #a shape parameter for Sp of TestA
Sp.TestA.shapeb <- Sp.US$shape2     #b shape parameter for Sp of TestA
```

Now, we could add the *covp* and *covn* parameters in the likelihood function, and specified prior distributions for these as we did before:

```
#Create the JAGS text file
model_2tests_2pop_dep <- paste0("model{

#=== LIKELIHOOD ===#

  #=== POPULATION 1 ===#
  Pop1[1:4] ~ dmulti(p1[1:4], ",nPop1,")
  p1[1] <- Prev1*(Se_", TestA, "*Se_", TestB, " + covp) + (1-Prev1)*((1-Sp_", TestA, "
  p1[2] <- Prev1*(Se_", TestA, "*(1-Se_", TestB, ") - covp) + (1-Prev1)*((1-Sp_", TestA
  p1[3] <- Prev1*((1-Se_", TestA, ")*Se_", TestB, " - covp) + (1-Prev1)*(Sp_", TestA, "
  p1[4] <- Prev1*((1-Se_", TestA, ")*(1-Se_", TestB, ") + covp) + (1-Prev1)*(Sp_", Test

  #=== POPULATION 2 ===#
  Pop2[1:4] ~ dmulti(p2[1:4], ",nPop2,")
  p2[1] <- Prev2*(Se_", TestA, "*Se_", TestB, " + covp) + (1-Prev2)*((1-Sp_", TestA, "
  p2[2] <- Prev2*(Se_", TestA, "*(1-Se_", TestB, ") - covp) + (1-Prev2)*((1-Sp_", TestA
  p2[3] <- Prev2*((1-Se_", TestA, ")*Se_", TestB, " - covp) + (1-Prev2)*(Sp_", TestA, "
  p2[4] <- Prev2*((1-Se_", TestA, ")*(1-Se_", TestB, ") + covp) + (1-Prev2)*(Sp_", Test



#=== PRIOR  ===#

  Prev1 ~ dbeta(",Prev1.shapea,", ",Prev1.shapeb,")     ## Prior for Prevalence in popu
  Prev2 ~ dbeta(",Prev2.shapea,", ",Prev2.shapeb,")     ## Prior for Prevalence in popu
  Se_", TestA, " ~ dbeta(",Se.TestA.shapea,", ",Se.TestA.shapeb,")  ## Prior for Se of
  Sp_", TestA, " ~ dbeta(",Sp.TestA.shapea,", ",Sp.TestA.shapeb,")  ## Prior for Sp of
  Se_", TestB, " ~ dbeta(",Se.TestB.shapea,", ",Se.TestB.shapeb,")  ## Prior for Se of
  Sp_", TestB, " ~ dbeta(",Sp.TestB.shapea,", ",Sp.TestB.shapeb,")  ## Prior for Sp of

  #=== CONDITIONAL DEPENDENCE STRUCTURE ===#

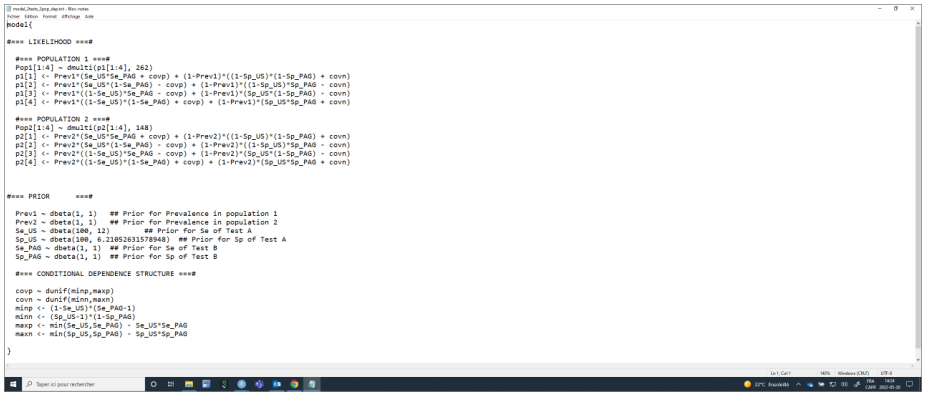
  covp ~ dunif(minp,maxp)
  covn ~ dunif(minn,maxn)
  minp <- (1-Se_", TestA, ")*(Se_", TestB, "-1)
  minn <- (Sp_", TestA, "-1)*(1-Sp_", TestB, ")
  maxp <- min(Se_", TestA, ",Se_", TestB, ") - Se_", TestA, "*Se_", TestB, "
  maxn <- min(Sp_", TestA, ",Sp_", TestB, ") - Sp_", TestA, "*Sp_", TestB, "

}")

#write as a text (.txt) file
write.table(model_2tests_2pop_dep,
            file="model_2tests_2pop_dep.txt",
            quote=FALSE,
```

```
            sep="",
            row.names=FALSE,
            col.names=FALSE)
```

This text file looks like this:



```
model{

#=== LIKELIHOOD ===#

  #=== POPULATION 1 ===#
  Pop1[1:4] ~ dmulti(p1[1:4], 262)
  p1[1] <- Prev1*(Se_US*Se_PAG + covp) + (1-Prev1)*((1-Sp_US)*(1-Sp_PAG) + covn)
  p1[2] <- Prev1*(Se_US*(1-Se_PAG) - covp) + (1-Prev1)*((1-Sp_US)*Sp_PAG - covn)
  p1[3] <- Prev1*((1-Se_US)*Se_PAG - covp) + (1-Prev1)*(Sp_US*(1-Sp_PAG) - covn)
  p1[4] <- Prev1*((1-Se_US)*(1-Se_PAG) + covp) + (1-Prev1)*(Sp_US*Sp_PAG + covn)

  #=== POPULATION 2 ===#
  Pop2[1:4] ~ dmulti(p2[1:4], 148)
  p2[1] <- Prev2*(Se_US*Se_PAG + covp) + (1-Prev2)*((1-Sp_US)*(1-Sp_PAG) + covn)
  p2[2] <- Prev2*(Se_US*(1-Se_PAG) - covp) + (1-Prev2)*((1-Sp_US)*Sp_PAG - covn)
  p2[3] <- Prev2*((1-Se_US)*Se_PAG - covp) + (1-Prev2)*(Sp_US*(1-Sp_PAG) - covn)
  p2[4] <- Prev2*((1-Se_US)*(1-Se_PAG) + covp) + (1-Prev2)*(Sp_US*Sp_PAG + covn)


#=== PRIOR      ===#

  Prev1 ~ dbeta(1, 1)   ## Prior for Prevalence in population 1
  Prev2 ~ dbeta(1, 1)   ## Prior for Prevalence in population 2
  Se_US ~ dbeta(100, 12)        ## Prior for Se of Test A
  Sp_US ~ dbeta(100, 6.210526315789948)  ## Prior for Sp of Test A
  Se_PAG ~ dbeta(1, 1)  ## Prior for Se of Test B
  Sp_PAG ~ dbeta(1, 1)  ## Prior for Sp of Test B

  #=== CONDITIONAL DEPENDENCE STRUCTURE ===#

  covp ~ dunif(minp,maxp)
  covn ~ dunif(minn,maxn)
  minp <- (1-Se_US)*(Se_PAG-1)
  minn <- (Sp_US-1)*(1-Sp_PAG)
  maxp <- min(Se_US,Se_PAG) - Se_US*Se_PAG
  maxn <- min(Sp_US,Sp_PAG) - Sp_US*Sp_PAG

}
```

Figure 7.2: **Text file with the model for 2 DEPENDENT diagnostic tests applied to two populations.**

Again, the list of initial values will have to be modified to include (*covp* and *covn*).

```
#Initializing values for the parameters Prev, and the Ses and Sps of the two tests for the 3 chai
inits <- list(list(Prev1=0.50,
                   Prev2=0.50,
                   Se_US=0.90,
                   Sp_US=0.90,
                   Se_PAG=0.90,
                   Sp_PAG=0.90,
                   covp=0.0,
                   covn=0.0),

                   list(Prev1=0.60,
                   Prev2=0.60,
                   Se_US=0.80,
                   Sp_US=0.80,
                   Se_PAG=0.80,
                   Sp_PAG=0.80,
                   covp=0.01,
                   covn=0.01),
```

```
              list(Prev1=0.40,
              Prev2=0.40,
              Se_US=0.70,
              Sp_US=0.70,
              Se_PAG=0.70,
              Sp_PAG=0.70,
              covp=0.05,
              covn=0.05)
              )
```

And, when running the model using `jags()`, we would also ask to monitor *covp* and *covn*. Note that we have increased the number of iterations / burn-in period (remember the autocorrelation problem seen with LCM with a covariance between tests?).

```
library(R2jags)
library(coda)

#Run the Bayesian model
bug.out <- jags(data=datalist,
                model.file="model_2tests_2pop_dep.txt",
                parameters.to.save=c("Prev1", "Prev2", "Se_US", "Sp_US", "Se_PAG", "Sp_
                n.chains=3,
                inits=inits,
                n.iter=210000,
                n.burnin=10000,
                n.thin=1,
                DIC=FALSE)
```

```
## Compiling model graph
##    Resolving undeclared variables
##    Allocating nodes
## Graph information:
##    Observed stochastic nodes: 2
##    Unobserved stochastic nodes: 8
##    Total graph size: 72
##
## Initializing model
```

Then we could produce the diagnostic plots, compute the ESS, and print out our results as we did previously (results not shown).

```
library(mcmcplots)
bug.mcmc <- as.mcmc(bug.out)
```

```
mcmcplot(bug.mcmc, title="Diagnostic plots")
effectiveSize(bug.mcmc)
print(bug.out, digits.summary=3)
```

7.5 Hierarchical latent class models

7.5.1 The general idea

In a given study, you may have measured individuals from a substantial number of sub-populations. For instance, in the Dufour et al. (2017) original study comparing the PAG-ELISA and ultrasound tests for diagnosing pregnancy in dairy cows, cows from 18 commercial dairy herds were studied. In such case, we could, for instance, split the data in 18 sub-populations and use the models described in the preceding sections, with 18 likelihood functions, and where we would estimate 18 prevalence (one for each herd). This approach is totally fine, but it may not be the most elegant, nor efficient model for that type of study. If we were to use this approach, and assuming conditional independence between tests, we would have:

- 22 unknown parameters to estimate (*SeA* and *SpA*, *SeB* and *SpB*, and each sub-population's prevalence, *P1* to *P18* in this case);
- 54 degrees of freedom (3 degrees of freedom for each 2x2 table, and 18 2x2 tables).

Nevertheless, this would be an identifiable model and we would not necessarily require informative priors.

On the other hand, Hanson et al. (2003) have proposed a slightly different, and possibly more elegant and efficient, modelling approach named the hierarchical latent class model. Actually, in most cases, **estimating the disease prevalence in each of the sub-populations (e.g., in each of the 18 herds in the PAG study) is not a primary objective as compared to estimating** test accuracy and/or **the distribution of within-herd prevalence in the population of herds from which these herds were sampled (i.e., the source population)**. In such case, we could use the model described in the figure below.

Briefly, in this model we would assume that each of the recruited herds originated from a large population of herds (the source population). We would then only need to describe the distribution of within-herd prevalence of the source population (rather than the prevalence of sick individuals of each herd, as we

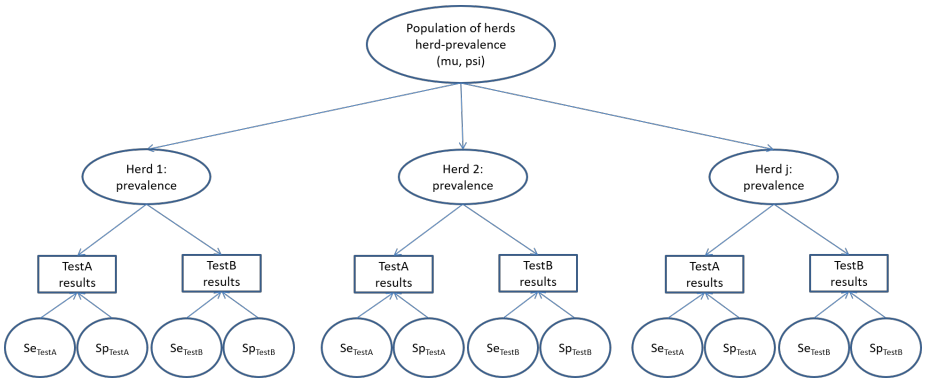


Figure 7.3: **Diagrammatic representation of Hanson et al. (2005) hierarchical latent class model. Rectangles represent observed variables (results of the two tests), ovals represent latent variable (individual herd's disease prevalence and population's distribution of within-herd prevalence), and circles represent the unknown accuracy parameters (Se and Sp of the two tests).**

did so far). We could propose a *beta(a, b)* distribution as prior for this distribution of within-herd prevalence. Given the hierarchical nature of the priors (i.e., the priors are on the source population, not on the studied sub-populations), we will rather refer to them as **hyperpriors**. In the hierarchical model, instead of providing hyperpriors for a and b, we will instead provide hyperpriors for *mu* and *psi*. Remember from Chapter 2 that:

- *mu* represents how the mean within-herd prevalence varies in the source population of herds;
- *psi* describes the spread of the distribution around the mean within-herd prevalence in the source population of herds.

And also, in a *beta(a,b)* distribution:

$a = mu * psi$
$b = psi * (1 - mu)$

For *mu*, it would be quite natural to use a beta distribution as hyperprior, given that the mean within-herd prevalence need to be bounded by 0.0 and 1.0. For *psi*, which represent a variance, a gamma distribution would be a better option for this hyperprior.

We have not discussed gamma distributions in Chapter 2, since they are not used in the other BLCM presented in this document. Briefly, the gamma distribution is also defined by two parameters and . The parameter is also known as the

shape parameter and the parameter is known as the *rate* parameter. Gamma distributions can take different forms, but are bounded to values between 0 and ∞. Moreover, a **Gamma(0.001, 0.001) would correspond to a vague gamma distribution.**

7.5.2 The likelihood function

We could described the likelihood function for the hierarchical model as follow:

for i in $1 : NumberOfHerds$
$n[i, 1:4] \sim dmulti(p[i, 1:4], NumAnimInHerd[i])$

$p[i,1] = prevInHerd[i]*SeA*SeB+(1-prevInHerd[i])*(1-SpA)*(1-SpB)$
$p[i,2] = prevInHerd[i]*SeA*(1-SeB)+(1-prevInHerd[i])*(1-SpA)*SpB$
$p[i,3] = prevInHerd[i]*(1-SeA)*SeB+(1-prevInHerd[i])*SpA*(1-SpB)$
$p[i,4] = prevInHerd[i]*(1-SeA)*(1-SeB)+(1-prevInHerd[i])*SpA*SpB$

$prevInHerd[i] \sim dbeta(alpha, beta)$
$alpha = mu * psi$
$beta = psi * (1 - mu)$

Where *NumberOfHerds* is the number of sub-populations in the study, *n[i,1:4]* is the content, for each sub-population, of the 4 cells (*n1*, *n2*, *n3*, and *n4*) from the 2x2 tables with cross-classified tests results (i.e., the observed data), *NumAnimInHerd[i]* is the number of individuals tested in each sub-population, and *prevInHerd[i]* is the prevalence of disease in each sub-population. *dbeta(alpha, beta)* is the distribution of within-herd prevalence in the source population (assumed to follow a beta distribution) and is defined by two unknown parameters, *mu* and *psi*, for which we will have to propose hyperpriors. We could use a beta hyperprior for *mu* and a gamma hyperprior for *psi*. These hyperpriors could be vague (e.g., beta(1.0, 1.0) and gamma(0.001, 0.001)) or informative (we will see in a following section how to propose informative hyperpriors for *mu* and *psi*).

If we examine the hierarchical model more carefully, we see one important difference between this model and the models we have used so far, the inclusion of a loop in the likelihood function. Briefly, we are asking to repeat this likelihood function for each sub-population (i.e., *for (i in 1:NumberOfHerds)*). Again, if we try to translate this likelihood function in words, and focusing on Herd #1 (to simplify) the number of individuals in cell *n1* (i.e., TestA+ and TestB+) is determined by the number of individuals tested in Herd #1 (*NumAnimInHerd[1]*) and the probability (*p[1]*) of falling in that cell. The latter itself depends on the true prevalence of disease in Herd #1 (*prevInHerd[1]*; which can be drawn from the source population distribution of within-herd prevalence, *dbeta(alpha, beta)*), and on each of the two tests sensitivity (*SeA* and *SeB*) and specificity (*SpA* and *SpB*). We can use similar reasoning for *n2*, *n3*, and *n4*. Finally, the multinomial distribution would further impose that the four probabilities (*p[1]*, *p[2]*, *p[3]*, and *p[4]*) have to sum up to 1.0 (i.e., 100%) within a given herd.

Note that we now have:

- 6 unknown parameters to estimate (*SeA* and *SpA*, *SeB* and *SpB*, *mu*, and *psi*), as compared to 4 parameters plus one parameter for each sub-population for the approach described in the preceding sections;
- 3 degrees of freedom for each sub-population.

For instance, with the 18 commercial herds available in the Dufour et al. (2017) original study, we would have 6 unknown parameters and 54 degrees of freedom. If we were to use 30 herds, we would still have 6 unknown parameters, but now 90 degrees of freedom, etc. This is the reason why we described the Hanson et al. (2003) hierarchical model as "more efficient".

7.5.3 The data

Data for the hierarchical model will have to be organized as follow. Briefly, we will first need a **dataset where we report, for each sub-population, the number of individuals with a given combination of tests results**. For instance, below we are importing an Excel sheet with the organized results. The example below is reproduced from Dufour et al. (2017; Table 1).

```
library(readxl)   #A library that can import .xlsx documents
obs <- read_excel("Data/Herd level results.xlsx")
#Check the data
obs
```

```
## # A tibble: 18 x 4
##    `TestA+TestB+` `TestA+TestB-` `TestA-TestB+` `TestA-TestB-`
##             <dbl>          <dbl>          <dbl>          <dbl>
##  1             20              0              2             14
##  2              7              0              0              4
##  3             10              0              0             10
##  4             12              0              1             21
##  5             17              0              2              9
##  6              2              0              0              3
##  7             46              1              3             37
##  8              0              0              0              1
##  9              7              0              1              5
## 10              7              0              0              3
## 11             12              1              2             12
## 12              8              0              1              7
## 13             11              0              0              9
## 14              5              0              1             10
```

```
## 15              71              0               8              69
## 16              10              0               0               3
## 17               5              0               0               1
## 18               5              0               0               1
```

You can see that we have used this order:

- *TestA* + and *TestB* +
- *TestA* + and *TestB* -
- *TestA* - and *TestB* +
- *TestA* - and *TestB* -

This will be important, because we will use the same order for the 4 probabilities in the likelihood function.

Then, we will need a **second dataset where we report the total number of individual tested by sub-population**. This can be computed from the above dataset as follow:

```
#Compute total number of animals by herd
totals <- rowSums(obs)
#Check the total number of animals per herd
totals
```

```
##  [1]  36  11  20  34  28   5  87   1  13  10  27  16  20  16 148  13   6   6
```

The previous two pieces of data will have to be reunited together. We can do this using:

```
datalist <- list(obs=obs, totals=totals)
#Check the datalist
datalist
```

```
## $obs
## # A tibble: 18 x 4
##    `TestA+TestB+` `TestA+TestB-` `TestA-TestB+` `TestA-TestB-`
##             <dbl>          <dbl>          <dbl>          <dbl>
##  1             20              0              2             14
##  2              7              0              0              4
##  3             10              0              0             10
##  4             12              0              1             21
```

```
##  5              17              0              2              9
##  6               2              0              0              3
##  7              46              1              3             37
##  8               0              0              0              1
##  9               7              0              1              5
## 10               7              0              0              3
## 11              12              1              2             12
## 12               8              0              1              7
## 13              11              0              0              9
## 14               5              0              1             10
## 15              71              0              8             69
## 16              10              0              0              3
## 17               5              0              0              1
## 18               5              0              0              1
##
## $totals
##  [1]  36  11  20  34  28   5  87   1  13  10  27  16  20  16 148  13   6   6
```

Finally, we will need to know the **number of sub-populations studied**. Again, this can be computed as follow:

```
#Compute number of herds
numherd <- dim(obs)[1]
#Check number of herd
numherd
```

```
## [1] 18
```

We could also assign labels to the tests for future usage, as we did before:

```
TestA <- "US"
TestB <- "PAG"
```

7.5.4 The priors

We will need to assign priors (or hyperpriors) for *mu*, *psi*, *SeA* and *SpA*, *SeB* and *SpB*. Below, we have used vague priors (or hyperpriors) for all these unknown parameters. If we were to use informative priors for *SeA*, *SpA*, *SeB*, or *SpB*, we could implement this as in the previous chapters. Informative priors for *mu* and *psi* are bit more tedious to generate; we will describe in a following section how to achieve this.

```
mu.shapea <- 1.0            # a shape parameter for mu
mu.shapeb <- 1.0            # b shape parameter for mu

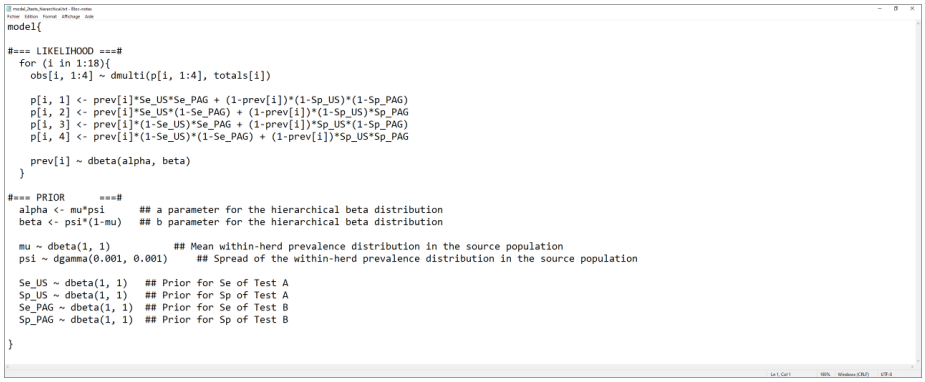
psi.shapea <- 0.001         #   (shape) parameter for gamma
psi.shapeb <- 0.001         #   (rate) parameter for gamma

Se.TestA.shapea <- 1        #a shape parameter for Se of TestA
Se.TestA.shapeb <- 1        #b shape parameter for Se of TestA
Sp.TestA.shapea <- 1        #a shape parameter for Sp of TestA
Sp.TestA.shapeb <- 1        #b shape parameter for Sp of TestA

Se.TestB.shapea <- 1        #a shape parameter for Se of TestB
Se.TestB.shapeb <- 1        #b shape parameter for Se of TestB
Sp.TestB.shapea <- 1        #a shape parameter for Sp of TestB
Sp.TestB.shapeb <- 1        #b shape parameter for Sp of TestB
```

7.5.5 Write the JAGS model

The text file we would like to generate would look as the screen capture below.



```
model{

#=== LIKELIHOOD ===#
  for (i in 1:18){
    obs[i, 1:4] ~ dmulti(p[i, 1:4], totals[i])

    p[i, 1] <- prev[i]*Se_US*Se_PAG + (1-prev[i])*(1-Sp_US)*(1-Sp_PAG)
    p[i, 2] <- prev[i]*Se_US*(1-Se_PAG) + (1-prev[i])*(1-Sp_US)*Sp_PAG
    p[i, 3] <- prev[i]*(1-Se_US)*Se_PAG + (1-prev[i])*Sp_US*(1-Sp_PAG)
    p[i, 4] <- prev[i]*(1-Se_US)*(1-Se_PAG) + (1-prev[i])*Sp_US*Sp_PAG

    prev[i] ~ dbeta(alpha, beta)
  }

#=== PRIOR      ===#
  alpha <- mu*psi      ## a parameter for the hierarchical beta distribution
  beta <- psi*(1-mu)   ## b parameter for the hierarchical beta distribution

  mu ~ dbeta(1, 1)          ## Mean within-herd prevalence distribution in the source population
  psi ~ dgamma(0.001, 0.001)     ## Spread of the within-herd prevalence distribution in the source population

  Se_US ~ dbeta(1, 1)   ## Prior for Se of Test A
  Sp_US ~ dbeta(1, 1)   ## Prior for Sp of Test A
  Se_PAG ~ dbeta(1, 1)  ## Prior for Se of Test B
  Sp_PAG ~ dbeta(1, 1)  ## Prior for Sp of Test B

}
```

Figure 7.4: **Text file for the hierarchical model.**

The script below can be used to generate this text file.

```
#Create the JAGS text file
model_2tests_hierarchical <- paste0("model{

#=== LIKELIHOOD ===#
  for (i in 1:", numherd,"){
    obs[i, 1:4] ~ dmulti(p[i, 1:4], totals[i])
```

```
    p[i, 1] <- prev[i]*Se_", TestA, "*Se_", TestB, " + (1-prev[i])*(1-Sp_", TestA, ")*
    p[i, 2] <- prev[i]*Se_", TestA, "*(1-Se_", TestB, ") + (1-prev[i])*(1-Sp_", TestA,
    p[i, 3] <- prev[i]*(1-Se_", TestA, ")*Se_", TestB, " + (1-prev[i])*Sp_", TestA, "*
    p[i, 4] <- prev[i]*(1-Se_", TestA, ")*(1-Se_", TestB, ") + (1-prev[i])*Sp_", TestA

    prev[i] ~ dbeta(alpha, beta)
  }

#=== PRIOR  ===#
  alpha <- mu*psi      ## a parameter for the hierarchical beta distribution
  beta <- psi*(1-mu)   ## b parameter for the hierarchical beta distribution

  mu ~ dbeta(", mu.shapea, ", ", mu.shapeb, ")       ## Mean within-herd prevalence dis
  psi ~ dgamma(", psi.shapea, ", ", psi.shapeb, ")   ## Spread of the within-herd preva

  Se_", TestA, " ~ dbeta(", Se.TestA.shapea, ", ", Se.TestA.shapeb, ")  ## Prior for Se
  Sp_", TestA, " ~ dbeta(", Sp.TestA.shapea, ", ", Sp.TestA.shapeb, ")  ## Prior for Sp
  Se_", TestB, " ~ dbeta(", Se.TestB.shapea, ", ", Se.TestB.shapeb, ")  ## Prior for Se
  Sp_", TestB, " ~ dbeta(", Sp.TestB.shapea, ", ", Sp.TestB.shapeb, ")  ## Prior for Sp

}")


#write as a text (.txt) file
write.table(model_2tests_hierarchical,
            file="model_2tests_hierarchical.txt",
            quote=FALSE,
            sep="",
            row.names=FALSE,
            col.names=FALSE)
```

7.5.6 Generating initial values

Remember, the initial value is the first value where each Markov chain will start.
- You have to specify one initial value for each unknown parameter in the model (here *mu*, *psi*, and the tests' accuracy parameters);
- If you decide to run three Markov chains, you will need three different sets of initial values;
- You can also let the software choose a random value to start each chain.

```
#Initializing values for the unknown parameters for the 3 chains.
inits <- list(list(mu=0.50,
                   psi=1,
                   Se_US=0.90,
```

```
                    Sp_US=0.90,
                    Se_PAG=0.90,
                    Sp_PAG=0.90),

               list(mu=0.40,
                    psi=2,
                    Se_US=0.80,
                    Sp_US=0.80,
                    Se_PAG=0.80,
                    Sp_PAG=0.80),

               list(mu=0.60,
                    psi=0.5,
                    Se_US=0.95,
                    Sp_US=0.95,
                    Se_PAG=0.95,
                    Sp_PAG=0.95)
)
```

7.5.7 Running the model

As we did before, we could use the following script to run the model. Note that the tests accuracy parameters, along with *mu* and *psi* are the main parameters that you could monitor for that model.

```
library(R2jags)
library(coda)

#Run the Bayesian model
bug.out <- jags(data=datalist,
                model.file="model_2tests_hierarchical.txt",
                parameters.to.save=c("Se_US", "Sp_US", "Se_PAG", "Sp_PAG",
                                     "mu", "psi"),
                n.chains=3,
                inits=inits,
                n.iter=25000,
                n.burnin=5000,
                n.thin=1,
                DIC=FALSE)
```

```
## Compiling model graph
##    Resolving undeclared variables
##    Allocating nodes
## Graph information:
```

```
##    Observed stochastic nodes: 18
##    Unobserved stochastic nodes: 24
##    Total graph size: 321
##
## Initializing model
```

7.5.8 Model diagnostics

Model's diagnostics can be generated as we did before (results not shown).

```
library(mcmcplots)
bug.mcmc <- as.mcmc(bug.out)
mcmcplot(bug.mcmc, title="Diagnostic plots")
effectiveSize(bug.mcmc)
```

7.5.9 Getting our estimates

Our estimates can be obtained as we did before.

```
print(bug.out, digits.summary=3)
```

```
## Inference for Bugs model at "model_2tests_hierarchical.txt", fit using jags,
##  3 chains, each with 25000 iterations (first 5000 discarded)
##  n.sims = 60000 iterations saved. Running time = 41.26 secs
##        mu.vect sd.vect   2.5%    25%     50%     75%    97.5%  Rhat n.eff
## Se_PAG   0.992   0.006  0.976  0.989   0.994   0.997    1.000 1.001 13000
## Se_US    0.958   0.025  0.907  0.940   0.960   0.979    0.998 1.002  2200
## Sp_PAG   0.950   0.029  0.892  0.928   0.951   0.974    0.997 1.002  2200
## Sp_US    0.991   0.007  0.972  0.987   0.993   0.997    1.000 1.001  8400
## mu       0.542   0.030  0.484  0.522   0.541   0.561    0.603 1.002  2000
## psi    285.298 362.739 17.848 66.868 154.262 346.208 1327.726 1.017   190
##
## For each parameter, n.eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor (at convergence, Rhat=1).
```

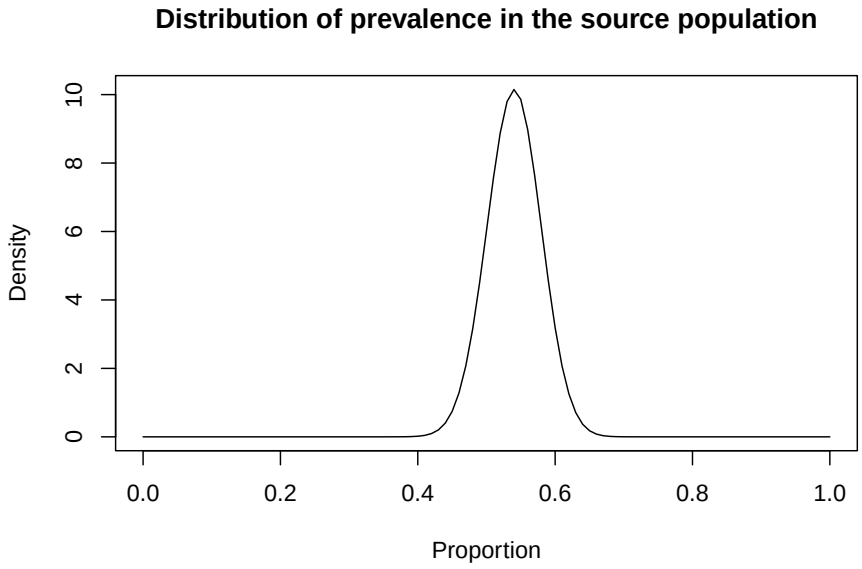
Finally, you may be interested in describing the distribution of within-herd prevalence of the source population. In the following script we used *mu* and *psi* median estimates to generate this plot.

```
mu <- 0.540      #Here used the median estimate for mu
psi <- 161.242   #Here used the median estimate for psi

alpha <- mu*psi
```

```
beta <- psi*(1-mu)

#plot the posterior distribution
curve(dbeta(x,
            shape1=alpha,
            shape2=beta),
      from=0, to=1,
      main="Distribution of prevalence in the source population",
      xlab = "Proportion",
      ylab = "Density")
```



7.5.10 Generating informative hyperpriors for mu and psi

Hanson et al. (2003) proposed to use a **beta distribution as a hyperprior for *mu* and a gamma distribution for *psi***. They also provided a method to elicit values for these hyperpriors. This method is, however, slightly more technical than what we have done so far.

7.5.10.1 mu

For *mu*, (the mean within-herd prevalence) we could use a method similar to the one we used before. For instance, we could have some literature, external data,

or experts opinion, indicating a most likely (mode) mean within-herd prevalence in the source population, and we could provide the 97.5th (or another) percentile for this mean within-herd prevalence. As an example, in the PAG *vs.* ultrasound study, Dufour et al. (2017) used reproductive data from 30 other herds from the same region as the 18 recruited herds. They computed the within-herd pregnancy prevalence at testing for these 30 herds and found a 50th and 95th percentiles of 42% and 74%, respectively. Using the `epiR` `epi.betabuster` function we could find the corresponding beta distribution as follow:

```
library(epiR)
# Prevalence as Mode=0.42, and we are 95% sure <0.74
rval <- epi.betabuster(mode=0.42,
                       conf=0.95,
                       imsure="less than",
                       x=0.74)

rval$shape1                 #View the a shape parameter in rval
```

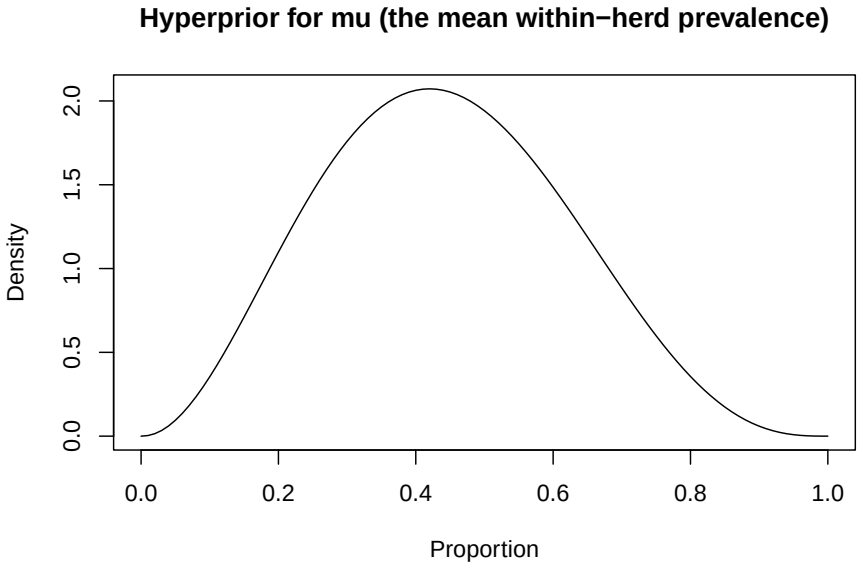
```
## [1] 3.132
```

```
rval$shape2                 #View the b shape parameter in rval
```

```
## [1] 3.94419
```

```
#Assign the generated values for later usage.
mu.shapea <- rval$shape1
mu.shapeb <- rval$shape2

#plot the prior distribution
curve(dbeta(x,
            shape1=mu.shapea,
            shape2=mu.shapeb),
      from=0, to=1,
      main="Hyperprior for mu (the mean within-herd prevalence)",
      xlab = "Proportion",
      ylab = "Density")
```



7.5.10.2 psi

For *psi*, which describe the spread of the distribution around the mean within-herd prevalence, Hanson et al. (2003) proposed to describe the uncertainty around the 90th percentile of the distribution of the mean within-herd prevalence. More specifically, they proposed to use the most likely value (mode) for that 90th percentile and a less likely and large value (for instance the 90th percentile) of this 90th percentile.

For instance, in Dufour et al. (2017), using the 30 herds from the same region, they found a 90th percentile of 65% of pregnancy (i.e., the top 10% of herds would achieve 65% pregnancy rate at pregnancy testing). Through experts opinion, they identified that this 90th percentile could hardly be higher than 75%, thus a 90th percentile of the 90th percentile of 75%. In other words, they proposed that the top 10% of herds would most probably have a pregnancy rate at testing of 65%, but this could possibly (90% sure) be up to 75%.

Then, as described by Hanson et al. (2003), the 50th percentile of the gamma distribution for *psi* can be computed by adding the *a* and *b* terms of the *beta(a, b)* distribution with mean of 42% (based on *mu*'s mode) and 90th percentile of 65% (based on the above assumption on the median value for the 90th percentile).

```
fifty.psi <- epi.betabuster(mode=0.42,
                            conf = 0.90,
                            imsure="less than",
```

```
                              x=0.65)

# Computing the value for the 50th percentile for the gamma distribution for psi
fifty <- fifty.psi$shape1 + fifty.psi$shape2
fifty
```

```
## [1] 8.904762
```

The 90th percentile of the gamma distribution is computed in the same manner, but using the a and b terms of the *beta(a, b)* distribution with mean of, again, 42%, but 90th percentile of 75%.

```
ninety.psi <- epi.betabuster(mode=0.42,
                              conf = 0.90,
                              imsure="less than",
                              x=0.75)

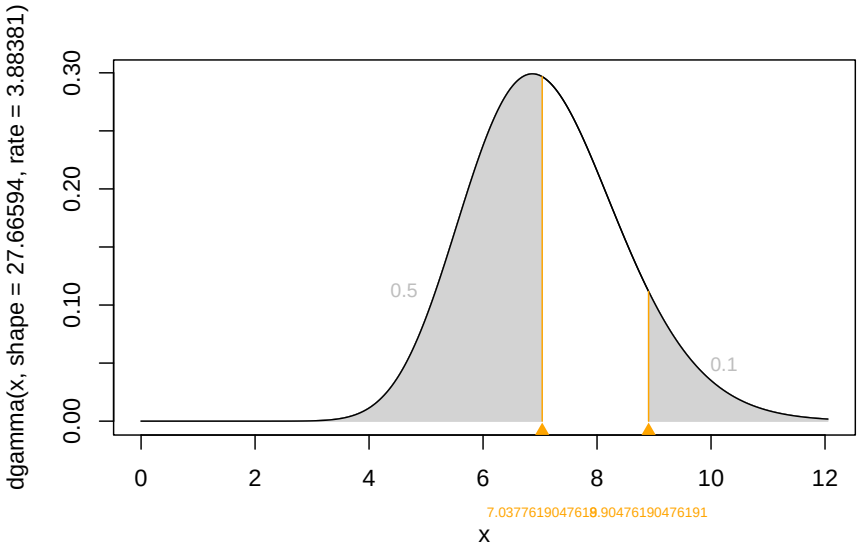
# Computing the value for the 50th percentile for the gamma distribution for psi
ninety <- ninety.psi$shape1 + fifty.psi$shape2
ninety
```

```
## [1] 7.037762
```

Using this methodology, we obtained the 50th and 90th percentiles for the gamma distribution for *psi* of 8.9047619 and 7.0377619, respectively. Finally, these percentiles can be translated to obtain the shape and rate parameters of the corresponding gamma distribution.

To achieve this in R, we can use the `GammaParmsFromQuantile.R` function that was developed by Patrick Brisle from McGill U. This function is not currently part of `R` package. It has to be downloaded from Patrick Brisle website. You can save the function on your computer, we did save it in a folder named "Functions". In the chunk below, the function is called and then used to identify the two parameters (shape and rate) for *psi*'s gamma distribution.

```
source('Functions/GammaParmsFromQuantiles.R')
psi.gamma <- gamma.parms.from.quantiles(q=c(fifty, ninety), p=c(0.50, 0.90), plot=T)
```




```
psi.shapea <- psi.gamma$shape    # a shape parameter for gamma
psi.shapea
```

```
## [1] 27.66594
```

```
psi.shapeb <- psi.gamma$rate    # b shape parameter for gamma
psi.shapeb
```

```
## [1] 3.88381
```

A gamma (27.7, 3.9) distribution can, therefore, be used to represent hyperprior information on *psi*.

7.5.11 Extensions

Of course, the hierarchical model presented could be extended as we have seen before to add, for instance, covariance terms between tests (thus, allowing for conditional dependence between tests). We could also extend the likelihood function to compare 3 (or 4, or 5, etc) tests as we will see in Chapter 9.

Chapter 8

Exercise 3 - Multiple populations

8.1 Questions

Open up the R project for Exercise 3 (i.e., the R project file named *Exercise 3.Rproj*).

In the Exercise 3 folder, you will find partially completed R scripts. To answer the questions, try to complete the missing parts (they are highlighted by a **#TO COMPLETE#** comment). We also provided complete R scripts, but try to work it out on your own first!

We know that US is not a gold-standard test for diagnosing pregnancy in dairy cows between 28-45 days post-AI. Romano et al. (2006) have reported estimates of Se and Sp for US that we have used in the preceding exercises. However, these US accuracy estimates were themselves computed without comparing to a gold-standard, and the authors did not used a LCM methodology. The reported US accuracy estimates are, thus, possibly biased to some extent.

It would be great if we could compute Se and Sp estimates for the PAG test, but without using these potentially biased US accuracy estimates. In the complete study, we have results for cows from three herds. Assuming that pregnancy prevalence is sufficiently different between herds, we could possibly use the Hui and Walter model with 3 populations. Cross-tabulated PAG and US data from the 3 herds are presented below.

Table. Cross-classified results from the PAG and US diagnostic tests in the three herds studied.

Herd 1			**Total**	**Herd 2**			**Total**	**Herd 3**			**Total**
	US+	**US-**			**US+**	**US-**			**US+**	**US-**	
PAG+	138	10		**PAG+**	71	8		**PAG+**	46	3	
PAG-	1	113		**PAG-**	0	69		**PAG-**	1	37	
Total			262	**Total**			148	**Total**			87

1. Using a Hui and Walter model with these three populations, how many unknown parameters and degrees of freedom would you have, and how many informative priors will you need?

2. Based on the observed results (do not compute a model yet, just ballpark estimates), do you think pregnancy prevalence differ between the 3 herds? Do you have any biological reason to believe that PAG or US Se and/or Sp should not be the same in herd #1 *vs.* herd #2 *vs.* herd #3?

3. Start from the partially completed *Question 3.R* script located in Exercise 3 folder. Try to apply a Hui and Walter model for three populations to these data using informative pregnancy prevalence (mode=0.42, 97.5th percentile=0.74) and informative US accuracy priors (Se: mode=0.90, 2.5th percentile=0.85; Sp: mode=0.95, 2.5th percentile=0.90). For this analysis, consider that the two tests are conditionally independent. Then use the same LCM, but with vague priors for all parameters and compare your results.

8.2 Answers

1. Using a Hui and Walter model with these three populations, how many unknown parameters and degrees of freedom would you have, and how many informative priors will you need?

Answer: We would still have 2 Se and 2 Sp and now 3 prevalence (one for each herd) to estimate, thus 7 unknown parameters. For each herd we have 3 degrees of freedom; in total 9 degrees of freedom. We have 7 unknown parameters and 9 degrees of freedom, so we do not need any informative prior to estimate these parameters.

2. Based on the observed results (do not compute a model yet, just ballpark estimates), do you think pregnancy prevalence differ between the 3 herds? Do you have any biological reason to believe that PAG or US Se and/or Sp should not be the same in herd #1 *vs.* herd #2 *vs.* herd #3?

Answer: We do not know the true pregnancy prevalence in these herds, but we could explore the apparent prevalence observed using our tests under investiga-

tion. Here are the apparent prevalence estimates computed using either PAG or US results by herd.

Table. Apparent prevalence of pregnancy in 3 herds using either a PAG or US test.

Herd	PAG-based prevalence	US-based prevalence
1	0.56	0.53
2	0.53	0.48
3	0.56	0.54

These prevalences are fairly similar… In a real-life situation, I would probably try to find another way to split the dataset in different populations so we have more striking outcome prevalence differences between populations (e.g., cows with < 3 AI *vs.* cows with ≥ 3 AI; 1st lactation cows *vs.* older cows). But, for the sake of the exercise, we could still try to apply the Hui and Walter model to these data.

Regarding PAG or US changing Se and/or Sp between the three herds, it is difficult to imagine any reason that would affect PAG or US accuracy from one herd to another. We would probably be OK considering that, no matter the herd, tests Se and Sp are the same.

3. Try to apply a Hui and Walter model for three populations to these data using informative pregnancy prevalence (mode=0.42, 97.5th percentile=0.74) and informative US accuracy priors (Se: mode=0.90, 2.5th percentile=0.85; Sp: mode=0.95, 2.5th percentile=0.90). For this analysis, consider that the two tests are conditionally independent. Then use the same LCM, but with vague priors for all parameters and compare your results.

Answer: The model with informative priors run very smoothly, as expected.

The model using vague priors for all parameters also converged very easily, autocorrelation is not a problem, etc. Isn't it a bit surprising? We would have expected that, with prevalence that are not overly different between the three herds, using the Hui and Walter model would not necessarily make the model identifiable. In this specific example, however, we have two very precise tests and, therefore, limited disagreement between tests (check the three 2x2 tables). Therefore, the number of potential solutions is very limited and the model is still identifiable even though we do not have striking prevalence differences between herds. It may not work as well in a different dataset. Below, I have summarize the results of the LCM with informative *vs.* vague priors.

Table. Results from a 2 tests, 3 populations LCM using different priors (informative on pregnancy prevalence and on US Se and Sp *vs.* vague priors on all parameters).

Parameter	Informative		Vague	
	Prior	**Posterior**	**Prior**	**Posterior**
Prev1	Beta(4.2, 5.4)	0.554 (0.493, 0.614)	Beta (1.0, 1.0)	0.547 (0.482, 0.612)
Prev2	Beta(4.2, 5.4)	0.517 (0.436, 0.595)	Beta (1.0, 1.0)	0.505 (0.418, 0.590)
Prev3	Beta(4.2, 5.4)	0.547 (0.444, 0.646)	Beta (1.0, 1.0)	0.551 (0.445, 0.654)
Se US	Beta (100, 12)	0.926 (0.893, 0.955)	Beta (1.0, 1.0)	0.961 (0.908, 0.998)
Sp US	Beta (100, 6.2)	0.977 (0.956, 0.990)	Beta (1.0, 1.0)	0.993 (0.972, 1.000)
Se PAG	Beta (1.0, 1.0)	0.996 (0.980, 1.000)	Beta (1.0, 1.0)	0.994 (0.976, 1.000)
Sp PAG	Beta (1.0, 1.0)	0.980 (0.956, 0.990)	Beta (1.0, 1.0)	0.949 (0.891, 0.997)

We have small difference between approaches. In the LCM with informative priors, we indicated, as prior knowledge, that US Se and Sp were around 0.90 and 0.95, respectively. Our posterior estimates for these parameters were 0.93 and 0.98. So, our data indicates that US Se and Sp is possibly better than what was reported by Romano et al.. This is confirmed in the model where we used vague priors. In that latter LCM, US Se and Sp were estimated at 0.96 and 0.99, respectively.

In the LCM with vague priors, Se of US was estimated to 0.96 (*vs.* 0.93), as a result, the PAG Sp was slightly reduced (0.95 *vs.* 0.98). In other words, in the LCM with informative priors, most of the US-/PAG+ results were attributed to errors of the US exam (because of the prior US Se with mode of 0.90). In the LCM with vague priors, the US-/PAG+ disagreements were distributed more equally between an error of the PAG *vs.* an error of the US test.

Chapter 9

Multiple diagnostic tests

In some situations, individuals will be tested using more than two diagnostic tests. For instance, if all individuals were tested using three different diagnostic tests (Test A, Test B, and Test C) we could present the cross-classified results of the three tests as follows:

Table. Cross-classified results from three diagnostic tests in one population.

Test C+			**Test C-**		
	Test A+	**Test A-**		**Test A+**	**Test A-**
Test B+	n1	n3	**Test B+**	n5	n7
Test B-	n2	n4	**Test B-**	n6	n8

Contrarily to the case where we had two tests in 2 populations, we can see from the preceding table that we have 7 degrees of freedom available. Indeed, we need to know the total number of individuals (n) and the content of seven of the eight cells to "guess" the value in the last cell. Moreover, with three diagnostic tests, we would have seven unknown parameters (SeA and SpA, SeB and SpB, SeC and SpC,and the population's prevalence, P) to estimate. The model is, therefore, identifiable. Again, we can refer to Cheung et al. (2021) to describe how the number of unknown parameters and degrees of freedom vary as function of the number of diagnostic tests used (see below).

Table. Number of degrees of freedom and of unknown parameters when comparing an increasing number of conditionally independent diagnostic tests in one single population (adapted from Cheung et al. (2021)).

Number of Tests	Number of unknown parameters (np)	Number of degrees of freedom (df)	Minimum number of informative priors needed
2	5	3	2
3	7	7	0
4	9	15	0
k	2*k+1	2^k -1	if df $<$ np, then np-df

As you can see, adding diagnostic tests could be an extremely valuable option to make a LCM identifiable. The gain in statistical power, however, will be maximized if:

-We used many **conditionally independent** diagnostic tests
-The sample size is sufficient so **all tests combinations are represented** (i.e., no empty cells in the table presenting the cross-classified results)

9.1 Likelihood function with 3 tests

With three diagnostic tests applied to a single population, the likelihood function that could be used to link the observed data (*n1*, …, *n8*) to the unknown parameters (*SeA* and *SpA*, *SeB* and *SpB*, *SeC* and *SpC*,and *P*) can be described as follows:

$n[1:8] \sim dmulti(P[1:8], n)$

$P1 = P * SeA * SeB * SeC + (1-P) * (1-SpA) * (1-SpB) * (1-SpC)$
$P2 = P * SeA * (1-SeB) * SeC + (1-P) * (1-SpA) * SpB * (1-SpC)$
$P3 = P * (1-SeA) * SeB * SeC + (1-P) * SpA * (1-SpB) * (1-SpC)$
$P4 = P * (1-SeA) * (1-SeB) * SeC + (1-P) * SpA * SpB * (1-SpC)$
$P5 = P * SeA * SeB * (1-SeC) + (1-P) * (1-SpA) * (1-SpB) * SpC$
$P6 = P * SeA * (1-SeB) * (1-SeC) + (1-P) * (1-SpA) * SpB * SpC$
$P7 = P * (1-SeA) * SeB * (1-Sec) + (1-P) * SpA * (1-SpB) * SpC$
$P8 = P * (1-SeA) * (1-SeB) * (1-SeC) + (1-P) * SpA * SpB * SpC$

Again, if we put that in words, the number of individuals in cell *n1* (i.e., TestA+ and TestB+ and TestC+) is determined by the number of individuals tested (n) and the probability (*P1*) of falling in that cell. The latter itself depends on the true prevalence of disease in the population (P), and on each of the tests sensitivity (*SeA*, *SeB*, and *SeC*) and specificity (*SpA*, *SpB*, and *SpC*). We can use similar reasoning for *n2* to *n8*. Finally, the multinomial distribution would further impose that the eight probabilities (*P1* to *P8*) have to sum up to 1.0 (i.e., 100%).

9.2 Organizing the data with 3 tests

To run such a model, we simply need to provide a dataset where *n1*, *n2*, *n3*, *n4*, *n5*, *n6*, *n7*, and *n8* are listed (in that order). For instance, in the PAG *vs.* US study, we have a number of cows that also had a conventional transrectal exam (**TRE**). The results of the three tests can be presented as follows:

Table. Cross-classified results from a PAG, US, and TRE diagnostic tests in one population.

TRE+			**TRE-**		
	US+	**US-**		**US+**	**US-**
PAG+	240	12	**PAG+**	15	9
PAG-	1	2	**PAG-**	1	216

The dataset could, thus be created as follows:

```
#n is of the form : (TestA+ TestB+ Test C+), (TestA+ TestB- TestC+), (TestA- TestB+ TestC+), (Tes
datalist <- list(n=c(240, 1, 12, 2, 15, 1, 9, 216))
```

9.3 The JAGS model with 3 tests

Again, we could provide the values that will be used to describe the prior distributions as we did before (they will then become included in the LCM text file). The only difference is that we now have three Se and Sp to describe. In the example below, I will use vague priors for all parameters.

```
#We could first create labels for TestA, TestB, and TestC
TestA <- "US"
TestB <- "PAG"
TestC <- "TRE"

#Provide information for the prior distributions (all beta distributions) for the 7 unknown param
Prev.shapea <- 1          #a shape parameter for Prev
Prev.shapeb <- 1          #b shape parameter for Prev

Se.TestA.shapea <- 1      #a shape parameter for Se of TestA
Se.TestA.shapeb <- 1      #b shape parameter for Se of TestA
Sp.TestA.shapea <- 1      #a shape parameter for Sp of TestA
Sp.TestA.shapeb <- 1      #b shape parameter for Sp of TestA

Se.TestB.shapea <- 1      #a shape parameter for Se of TestB
```

```
Se.TestB.shapeb <- 1     #b shape parameter for Se of TestB
Sp.TestB.shapea <- 1     #a shape parameter for Sp of TestB
Sp.TestB.shapeb <- 1     #b shape parameter for Sp of TestB

Se.TestC.shapea <- 1     #a shape parameter for Se of TestC
Se.TestC.shapeb <- 1     #b shape parameter for Se of TestC
Sp.TestC.shapea <- 1     #a shape parameter for Sp of TestC
Sp.TestC.shapeb <- 1     #b shape parameter for Sp of TestC

#I will also need the total number of individuals tested
n <- sapply(datalist, sum)
```

We now have everything that is needed to write the JAGS model.

```
#Create the JAGS text file
model_3tests_1pop_indep <- paste0("model{

#=== LIKELIHOOD ===#

  n[1:8] ~ dmulti(p[1:8], ",n,")
  p[1] <- Prev*Se_", TestA, "*Se_", TestB, "*Se_", TestC, " + (1-Prev)*(1-Sp_", TestA,
  p[2] <- Prev*Se_", TestA, "*(1-Se_", TestB, ")*Se_", TestC, " + (1-Prev)*(1-Sp_", Tes
  p[3] <- Prev*(1-Se_", TestA, ")*Se_", TestB, "*Se_", TestC, " + (1-Prev)*Sp_", TestA
  p[4] <- Prev*(1-Se_", TestA, ")*(1-Se_", TestB, ")*Se_", TestC, " + (1-Prev)*Sp_", Te

  p[5] <- Prev*Se_", TestA, "*Se_", TestB, "*(1-Se_", TestC,") + (1-Prev)*(1-Sp_", Test
  p[6] <- Prev*Se_", TestA, "*(1-Se_", TestB, ")*(1-Se_", TestC,") + (1-Prev)*(1-Sp_",
  p[7] <- Prev*(1-Se_", TestA, ")*Se_", TestB, "*(1-Se_", TestC,") + (1-Prev)*Sp_", Tes
  p[8] <- Prev*(1-Se_", TestA, ")*(1-Se_", TestB, ")*(1-Se_", TestC,") + (1-Prev)*Sp_"


#=== PRIOR  ===#

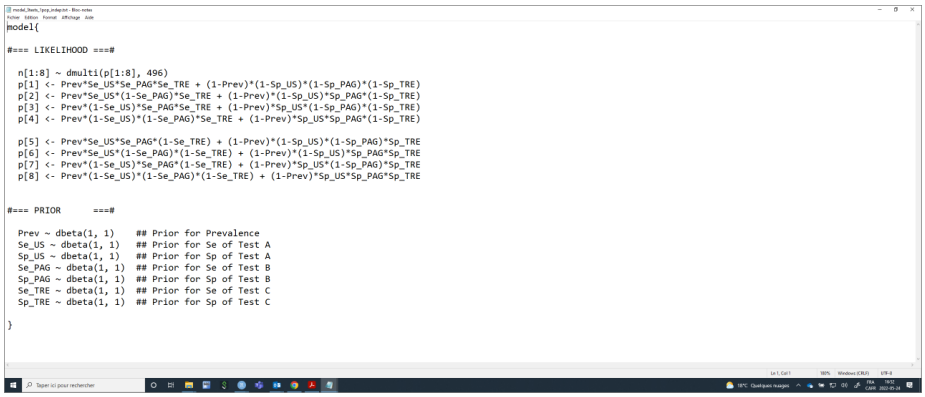
  Prev ~ dbeta(",Prev.shapea,", ",Prev.shapeb,")    ## Prior for Prevalence
  Se_", TestA, " ~ dbeta(",Se.TestA.shapea,", ",Se.TestA.shapeb,")  ## Prior for Se of
  Sp_", TestA, " ~ dbeta(",Sp.TestA.shapea,", ",Sp.TestA.shapeb,")  ## Prior for Sp of
  Se_", TestB, " ~ dbeta(",Se.TestB.shapea,", ",Se.TestB.shapeb,")  ## Prior for Se of
  Sp_", TestB, " ~ dbeta(",Sp.TestB.shapea,", ",Sp.TestB.shapeb,")  ## Prior for Sp of
  Se_", TestC, " ~ dbeta(",Se.TestC.shapea,", ",Se.TestC.shapeb,")  ## Prior for Se of
  Sp_", TestC, " ~ dbeta(",Sp.TestC.shapea,", ",Sp.TestC.shapeb,")  ## Prior for Sp of

}")

#write as a text (.txt) file
write.table(model_3tests_1pop_indep,
```

```
              file="model_3tests_1pop_indep.txt",
              quote=FALSE,
              sep="",
              row.names=FALSE,
              col.names=FALSE)
```

With this code, you could, again, simply modify the labels for Test A, Test B, and Test C, and the shape parameters for the prior distributions. The text file with the JAGS model will automatically be updated. Currently, it looks like this:



```
model{

#=== LIKELIHOOD ===#

  n[1:8] ~ dmulti(p[1:8], 496)
  p[1] <- Prev*Se_US*Se_PAG*Se_TRE + (1-Prev)*(1-Sp_US)*(1-Sp_PAG)*(1-Sp_TRE)
  p[2] <- Prev*Se_US*(1-Se_PAG)*Se_TRE + (1-Prev)*(1-Sp_US)*Sp_PAG*(1-Sp_TRE)
  p[3] <- Prev*(1-Se_US)*Se_PAG*Se_TRE + (1-Prev)*Sp_US*(1-Sp_PAG)*(1-Sp_TRE)
  p[4] <- Prev*(1-Se_US)*(1-Se_PAG)*Se_TRE + (1-Prev)*Sp_US*Sp_PAG*(1-Sp_TRE)

  p[5] <- Prev*Se_US*Se_PAG*(1-Se_TRE) + (1-Prev)*(1-Sp_US)*(1-Sp_PAG)*Sp_TRE
  p[6] <- Prev*Se_US*(1-Se_PAG)*(1-Se_TRE) + (1-Prev)*(1-Sp_US)*Sp_PAG*Sp_TRE
  p[7] <- Prev*(1-Se_US)*Se_PAG*(1-Se_TRE) + (1-Prev)*Sp_US*(1-Sp_PAG)*Sp_TRE
  p[8] <- Prev*(1-Se_US)*(1-Se_PAG)*(1-Se_TRE) + (1-Prev)*Sp_US*Sp_PAG*Sp_TRE


#=== PRIOR      ===#

  Prev ~ dbeta(1, 1)    ## Prior for Prevalence
  Se_US ~ dbeta(1, 1)   ## Prior for Se of Test A
  Sp_US ~ dbeta(1, 1)   ## Prior for Sp of Test A
  Se_PAG ~ dbeta(1, 1)  ## Prior for Se of Test B
  Sp_PAG ~ dbeta(1, 1)  ## Prior for Sp of Test B
  Se_TRE ~ dbeta(1, 1)  ## Prior for Se of Test C
  Sp_TRE ~ dbeta(1, 1)  ## Prior for Sp of Test C

}
```

Figure 9.1: **Text file with the LCM for 3 independent diagnostic tests applied to one population.**

Again, we will need to provide a list of initial values (one per Markov chain) for all unknown parameters. Careful again, we now have three tests.

```
#Initializing values for the parameters Prev, and the Ses and Sps of the two tests for the 3 chai
inits <- list(list(Prev=0.50,
              Se_US=0.90,
              Sp_US=0.90,
              Se_PAG=0.90,
              Sp_PAG=0.90,
              Se_TRE=0.90,
              Sp_TRE=0.90),

              list(Prev=0.60,
              Se_US=0.80,
              Sp_US=0.80,
              Se_PAG=0.80,
              Sp_PAG=0.80,
```

```
              Se_TRE=0.80,
              Sp_TRE=0.80),

              list(Prev=0.40,
              Se_US=0.70,
              Sp_US=0.70,
              Se_PAG=0.70,
              Sp_PAG=0.70,
              Se_TRE=0.70,
              Sp_TRE=0.70)
              )
```

We can run the model using `jags()` function as seen before.

```
library(R2jags)
library(coda)

#Run the Bayesian model
bug.out <- jags(data=datalist,
                model.file="model_3tests_1pop_indep.txt",
                parameters.to.save=c("Prev", "Se_US", "Sp_US", "Se_PAG", "Sp_PAG", "Se_
                n.chains=3,
                inits=inits,
                n.iter=11000,
                n.burnin=1000,
                n.thin=1,
                DIC=FALSE)
```

Then we could produce the diagnostic plots, compute the ESS, and print out our results as we did previously (results not shown).

```
library(mcmcplots)
bug.mcmc <- as.mcmc(bug.out)
mcmcplot(bug.mcmc, title="Diagnostic plots")
effectiveSize(bug.mcmc)
print(bug.out, digits.summary=3)
```

9.4 Conditional dependence

With LCM where >2 diagnostic tests are used, we could, again, relax the assumption of conditional independence by adding covariance terms representing the conditional dependence between tests (Dendukuri and Joseph, 2001). However, a LCM including every covariance term between every test is not always

feasible. A more pragmatic approach would be to consider pair-wise covariance terms between two of the three tests.

For instance, using the example from the preceding section, we could assume that US and TRE, which are both based on recognition (tactile or visual) of the amniotic vesicle or embryo, are possibly conditionally **dependent** (as compared to the PAG test, which measure a glycoprotein associated with the placenta). We could thus include covariance terms between these two specific tests. Remember, we had set up US as Test A and TRE as Test C. It will be easier, however, to manage the covariance terms if the dependent tests are next to each other within the likelihood function. I will, therefore, re-assign the tests as follows, and reorganized the dataset with that new tests configuration.
-Test A: US
-Test B: TRE
-Test C: PAG

```
TestA <- "US"
TestB <- "TRE"
TestC <- "PAG"

#n is of the form : (TestA+ TestB+ Test C+), (TestA+ TestB- TestC+), (TestA- TestB+ TestC+), (Tes
datalist <- list(n=c(240, 15, 12, 9, 1, 1, 2, 216))
n <- sapply(datalist, sum)

#Create the JAGS text file
model_3tests_1pop_dep <- paste0("model{

#=== LIKELIHOOD ===#

  n[1:8] ~ dmulti(p[1:8], ",n,")
  p[1] <- Prev*(Se_", TestA, "*Se_", TestB, " + covp)*Se_", TestC, " + (1-Prev)*((1-Sp_", TestA,
  p[2] <- Prev*(Se_", TestA, "*(1-Se_", TestB, ") - covp)*Se_", TestC, " + (1-Prev)*((1-Sp_", Tes
  p[3] <- Prev*((1-Se_", TestA, ")*Se_", TestB, " - covp)*Se_", TestC, " + (1-Prev)*(Sp_", TestA,
  p[4] <- Prev*((1-Se_", TestA, ")*(1-Se_", TestB, ") + covp)*Se_", TestC, " + (1-Prev)*(Sp_", Te

  p[5] <- Prev*(Se_", TestA, "*Se_", TestB, " + covp)*(1-Se_", TestC,") + (1-Prev)*((1-Sp_", Test
  p[6] <- Prev*(Se_", TestA, "*(1-Se_", TestB, ") - covp)*(1-Se_", TestC,") + (1-Prev)*((1-Sp_",
  p[7] <- Prev*((1-Se_", TestA, ")*Se_", TestB, " - covp)*(1-Se_", TestC,") + (1-Prev)*(Sp_", Tes
  p[8] <- Prev*((1-Se_", TestA, ")*(1-Se_", TestB, ") + covp)*(1-Se_", TestC,") + (1-Prev)*(Sp_",


#=== PRIOR  ===#

  Prev ~ dbeta(",Prev.shapea,", ",Prev.shapeb,")    ## Prior for Prevalence
  Se_", TestA, " ~ dbeta(",Se.TestA.shapea,", ",Se.TestA.shapeb,")  ## Prior for Se of Test A
  Sp_", TestA, " ~ dbeta(",Sp.TestA.shapea,", ",Sp.TestA.shapeb,")  ## Prior for Sp of Test A
```

```
  Se_", TestB, " ~ dbeta(",Se.TestB.shapea,", ",Se.TestB.shapeb,")  ## Prior for Se of
  Sp_", TestB, " ~ dbeta(",Sp.TestB.shapea,", ",Sp.TestB.shapeb,")  ## Prior for Sp of
  Se_", TestC, " ~ dbeta(",Se.TestC.shapea,", ",Se.TestC.shapeb,")  ## Prior for Se of
  Sp_", TestC, " ~ dbeta(",Sp.TestC.shapea,", ",Sp.TestC.shapeb,")  ## Prior for Sp of

  #=== CONDITIONAL DEPENDENCE STRUCTURE ===#

  covp ~ dunif(minp,maxp)
  covn ~ dunif(minn,maxn)
  minp <- (1-Se_", TestA, ")*(Se_", TestB, "-1)
  minn <- (Sp_", TestA, "-1)*(1-Sp_", TestB, ")
  maxp <- min(Se_", TestA, ",Se_", TestB, ") - Se_", TestA, "*Se_", TestB, "
  maxn <- min(Sp_", TestA, ",Sp_", TestB, ") - Sp_", TestA, "*Sp_", TestB, "

}")

#write as a text (.txt) file
write.table(model_3tests_1pop_dep,
            file="model_3tests_1pop_dep.txt",
            quote=FALSE,
            sep="",
            row.names=FALSE,
            col.names=FALSE)
```

The text file would look as follows:

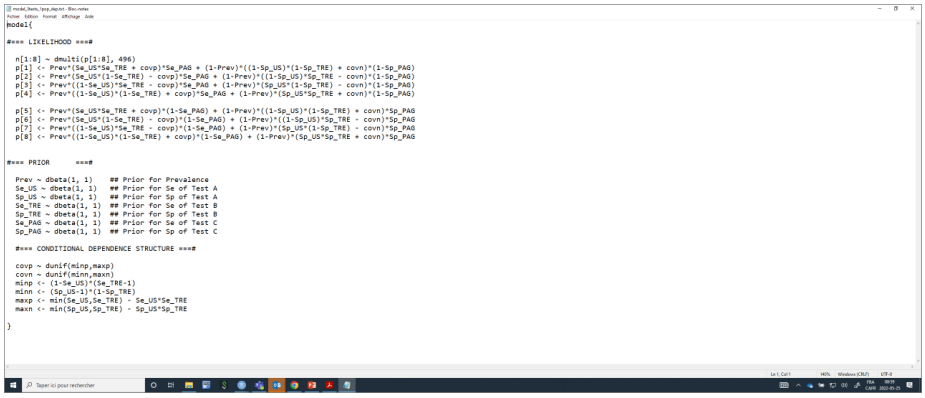


Figure 9.2: **Text file with the LCM for 3 diagnostic tests applied to one population and allowing for conditional dependence between two of the tests (US and TRE).**

We could run the model using `jags()` function and then produce the diagnostic

plots, compute the ESS, and print out our results as we did previously (results not shown).

```
library(R2jags)
library(coda)

#Run the Bayesian model
bug.out <- jags(data=datalist,
                model.file="model_3tests_1pop_dep.txt",
                parameters.to.save=c("Prev", "Se_US", "Sp_US", "Se_PAG", "Sp_PAG", "Se_TRE", "Sp_T
                n.chains=3,
                inits=inits,
                n.iter=11000,
                n.burnin=1000,
                n.thin=1,
                DIC=FALSE)

library(mcmcplots)
bug.mcmc <- as.mcmc(bug.out)
mcmcplot(bug.mcmc, title="Diagnostic plots")
effectiveSize(bug.mcmc)
print(bug.out, digits.summary=3)
```

In cases where you would like to consider potential conditional dependence between all tests, you could either:

- Run a number of models where you include, each time, one pairwise conditional dependence. For instance, with three tests: 1) model with *TestA vs. TestB*; 2) model with *TestA vs. TestC*; and 3) model with *TestB vs. TestC*. If you are "lucky", perhaps just one (or none) pair of tests will yield covariance terms that are substantially different from 0.0. Then, a model with just one (or no) set of covariance terms could be used.
- Include, in a single model, all potential covariance terms between 3 tests (or $>$ 3 tests). For this, a different modelling approach will have to be used. As an example, Denis-Robichaud et al. (2024) published a study where conditional dependence between 3 tests was allowed. The `R` script used for their analyses, along with the original dataset, was also published in this Dataverse.

Chapter 10

Exercise 4 - Multiple tests

10.1 Questions

Open up the `R` project for Exercise 4 (i.e., the R project file named *Exercise 4.Rproj*).

In the Exercise 4 folder, you will find partially completed `R` scripts. To answer the questions, try to complete the missing parts (they are highlighted by a **#TO COMPLETE#** comment). We also provided complete `R` scripts, but try to work it out on your own first!

During the study comparing PAG to US, a third test (a trans-rectal manual exam; **TRE**) was conducted on most cows. To be honest, this third exam was conducted later (>50 days post-AI). But, for the sake of the exercise, you could consider that three diagnostic tests (PAG, US, and TRE) were all conducted at the same time point and develop a LCM for 3 diagnostic tests applied to one single population (i.e., herd #1).

Table. Cross-classified results from a PAG, US, and TRE diagnostic tests in herd #1.

TRE+			**TRE-**		
	US+	**US-**		**US+**	**US-**
PAG+	126	3	**PAG+**	12	7
PAG-	0	0	**PAG-**	1	112

1. How many degrees of freedom and unknown parameters do you have? How many informative priors will be needed?

2. Start from the partially completed *Question 2.R* script located in Exercise 4 folder. Run a model using informative priors for US' Se and

Sp and pregnancy prevalence and with vague priors on all other parameters. Then run a second model with vague priors on all parameters. We will assume conditional independence between tests and we will also assume that this is one single population. Do you expect any problems with the model using vague priors on all parameters?

10.2 Answers

1. How many degrees of freedom and unknown parameters do you have? How many informative priors will be needed?

Answer: We have 7 degrees of freedom and 7 unknown parameters, so we do not need any informative priors.

2. Run a model using informative priors for US and pregnancy prevalence and with vague priors on all other parameters. Then run a second model with vague priors on all parameters. We will assume conditional independence between tests and we will also assume that this is one single population. Do you expect any problem with the model using vague priors on all parameters?

Answer: The first model with informative priors on US' Se, Sp and on pregnancy prevalence ran well. The Markov chains were nice. With 30,000 iterations, I do get a decent ESS (around 5,000 for the worst parameter, Se_PAG). The median estimates and 95 CI are reported in the table below.

A priori, I would also expect that the LCM with vague priors on all parameters will work well, since it is identifiable. This was the case, the Markov chains were also nice. The lowest ESS was again around 5,000 (with 3 times 10,000 iterations). If it was for publication, we could run these LCM for more iterations to reach an ESS $>$ 10,000 for all parameters. Results for the LCM with vague priors are also reported in the table below.

There were very little differences between the two LCM. Only the US' Se and Sp estimates were slightly affected. The LCM with informative priors generated posterior estimates that were somewhere in between the prior estimates and the estimates from the LCM with vague priors (i.e., between the priors and the data).

Table. Results from a 3 tests, 1 population LCM using different priors (informative on pregnancy prevalence and on US Se and Sp *vs.* vague priors on all parameters).

Parameter	**Informative**		**Vague**	
	Prior	**Posterior**	**Prior**	**Posterior**
Prev	Beta(4.2, 5.4)	0.539 (0.479, 0.599)	Beta(1.0, 1.0)	0.541 (0.481, 0.602)

Parameter	**Informative**		**Vague**	
Se PAG	Beta(1.0, 1.0)	0.995 (0.973, 1.00)	Beta(1.0, 1.0)	0.995 (0.972, 1.00)
Sp PAG	Beta(1.0, 1.0)	0.942 (0.887, 0.978)	Beta(1.0, 1.0)	0.938 (0.885, 0.974)
Se US	Beta(100, 12)	0.939 (0.904, 0.964)	Beta(1.0, 1.0)	0.972 (0.933, 0.992)
Sp US	Beta(100, 6.2)	0.969 (0.940, 0.986)	Beta(1.0, 1.0)	0.986 (0.953, 0.999)
Se TRE	Beta(1.0, 1.0)	0.907 (0.850, 0.949)	Beta(1.0, 1.0)	0.909 (0.852, 0.950)
Sp TRE	Beta(1.0, 1.0)	0.994 (0.969, 1.00)	Beta(1.0, 1.0)	0.994 (0.967, 1.00)

Chapter 11

Accuracy varying with covariate

So far we have (implicitly) assumed that tests' accuracy (i.e., their Se and Sp) are the same for every individuals tested. Some diagnostic tests, however, may behave differently whether they are applied to individuals of a certain age, breed, etc. For instance, we could hypothesize that many of the diagnostic tests aiming at quantifying antibodies against a given pathogen will behave differently in neonates (which typically have an immature immune system) compared to juvenile or adult animals.

The situation where test accuracy varies as function of a characteristic of the individual tested (i.e., as function of a covariate) can be implemented relatively easily in the different LCM we have used so far. For instance, if we hypothesize that the accuracy of the PAG-based pregnancy test varies as function of whether the cow is a first lactation *vs.* and older cow, we would simply specify a LCM where we have two different PAG Se and Sp, one for first lactation cows (*PAG_Se_first* and *PAG_Sp_first*) and one for older cows (*PAG_Se_old* and *PAG_Sp_old*). We would also have to provide two sets of data (one for first lactation cows and one for older cows). Of course, in that specific example, we would now have three additional unknown parameters to estimate (one additional Se, one additional Sp, and one additional prevalence), with three additional degrees of freedom (one additional 2x2 table). So investigating whether test accuracy varies as function of a covariate will not help us make the LCM identifiable. We will still need to use another trick to make the model identifiable (e.g., informative priors, > 1 population, or >2 diagnostic tests).

11.1 Likelihood function when accuracy varies as function of covariate

If we have two diagnostic tests (Test A and Test B) applied to a single population, and if we assumed that the Se and Sp of one of the test (Test A) varies as function of a characteristic (Z) of the individual tested, then we would specify one likelihood function for the individuals without the characteristic (Z=0) and one likelihood function for the individual having the characteristic (Z=1). Moreover, we would estimate two Se and two Sp for the test which is hypothesized to vary as function of the covariate (*Se_Z0*, *Sp_Z0*, *Se_Z1*, *Sp_Z1*). Note that this could be extended to covariates with more than 2 levels. Thus, the likelihood function would become:

With Z=0:
$n_0[1:4] \sim dmulti(P[1:4]_0, n_0)$
$P1_0 = P_0 * SeA_0 * SeB + (1 - P_0) * (1 - SpA_0) * (1 - SpB)$
$P2_0 = P_0 * SeA_0 * (1 - SeB) + (1 - P_0) * (1 - SpA_0) * SpB$
$P3_0 = P_0 * (1 - SeA_0) * SeB + (1 - P_0) * SpA_0 * (1 - SpB)$
$P4_0 = P_0 * (1 - SeA_0) * (1 - SeB) + (1 - P_0) * SpA_0 * SpB$

With Z=1:
$n_1[1:4] \sim dmulti(P[1:4]_1, n_1)$
$P1_1 = P_1 * SeA_1 * SeB + (1 - P_1) * (1 - SpA_1) * (1 - SpB)$
$P2_1 = P_1 * SeA_1 * (1 - SeB) + (1 - P_1) * (1 - SpA_1) * SpB$
$P3_1 = P_1 * (1 - SeA_1) * SeB + (1 - P_1) * SpA_1 * (1 - SpB)$
$P4_1 = P_1 * (1 - SeA_1) * (1 - SeB) + (1 - P_1) * SpA_1 * SpB$

11.2 Organizing the data when accuracy varies as function of covariate

To run such a model, we simply need to provide a dataset presenting the cross tabulated results of the two tests for individuals with Z=0 and a second dataset for the individuals with Z=1. For instance, in the PAG *vs.* US study, the results of the PAG and US tests can be presented as function of the cow's age as follows:

Table. Cross-classified results from a PAG and US in first lactation *vs.* older cows.

First (Z=0)			**Old (Z=1)**		
	US+	**US-**		**US+**	**US-**
PAG+	76	10	**PAG+**	179	11
PAG-	1	65	**PAG-**	1	153

The two datasets could be created as follows:

```
#n is of the form : (TestA pos and TestB pos), (TestA pos and TestB neg), (TestA neg and TestB po
datalist <- list(Z0=c(76,1 ,10, 65),
                 Z1=c(179, 1, 11, 153)
                 )
```

11.3 The JAGS model when accuracy varies as function of covariate

We could provide the values that will be used to described the prior distributions as we did before (so they are included in the model text file). The difference is that we now have, for one of the test, two Se and Sp and we also have one prevalence for Z=0 and one prevalence for Z=1. In the example below, I will use informative priors for these two prevalence (I will use the same priors for Z=0 and Z=1, but they could be different) and for US' Se and Sp.

```
#I could first create labels for TestA and TestB
TestA <- "US"
TestB <- "PAG"

#I could create labels for Z=0 and Z=1
Z0 <- "First"
Z1 <- "Old"

#Provide information for the prior distributions (all beta distributions) for the unknown paramet
Prev0.shapea <- 4.2          #a shape parameter for Prev in Z=0
Prev0.shapeb <- 5.4          #b shape parameter for Prev in Z=0

Prev1.shapea <- 4.2          #a shape parameter for Prev in Z=1
Prev1.shapeb <- 5.4          #b shape parameter for Prev in Z=1

Se.TestA.shapea <- 131     #a shape parameter for Se of TestA
Se.TestA.shapeb <- 15     #b shape parameter for Se of TestA
Sp.TestA.shapea <- 100     #a shape parameter for Sp of TestA
Sp.TestA.shapeb <- 6     #b shape parameter for Sp of TestA

Se.TestB.Z0.shapea <- 1     #a shape parameter for Se of TestB in Z=0
Se.TestB.Z0.shapeb <- 1     #b shape parameter for Se of TestB in Z=0
Sp.TestB.Z0.shapea <- 1     #a shape parameter for Sp of TestB in Z=0
Sp.TestB.Z0.shapeb <- 1     #b shape parameter for Sp of TestB in Z=0

Se.TestB.Z1.shapea <- 1     #a shape parameter for Se of TestB in Z=1
```

```
Se.TestB.Z1.shapeb <- 1     #b shape parameter for Se of TestB in Z=1
Sp.TestB.Z1.shapea <- 1     #a shape parameter for Sp of TestB in Z=1
Sp.TestB.Z1.shapeb <- 1     #b shape parameter for Sp of TestB in Z=1

#I will also need the number of individuals tested in each populations (n0 and n1)
n <- sapply(datalist, sum)
n0 <- n[1]
n1 <- n[2]
```

With that, we have everything that is needed to write the JAGS model.

```
#Create the JAGS text file
model_2tests_covariate <- paste0("model{

#=== LIKELIHOOD ===#
  #For Z=0
  Z0[1:4] ~ dmulti(p0[1:4], ",n0,")
  p0[1] <- Prev_", Z0,"*Se_", TestA, "*Se_", TestB, "_", Z0," + (1-Prev_", Z0,")*(1-Sp_
  p0[2] <- Prev_", Z0,"*Se_", TestA, "*(1-Se_", TestB, "_", Z0,") + (1-Prev_", Z0,")*(
  p0[3] <- Prev_", Z0,"*(1-Se_", TestA, ")*Se_", TestB, "_", Z0," + (1-Prev_", Z0,")*Sp
  p0[4] <- Prev_", Z0,"*(1-Se_", TestA, ")*(1-Se_", TestB, "_", Z0,") + (1-Prev_", Z0,"

  #For Z=1
  Z1[1:4] ~ dmulti(p1[1:4], ",n1,")
  p1[1] <- Prev_", Z1,"*Se_", TestA, "*Se_", TestB, "_", Z1," + (1-Prev_", Z1,")*(1-Sp_
  p1[2] <- Prev_", Z1,"*Se_", TestA, "*(1-Se_", TestB, "_", Z1,") + (1-Prev_", Z1,")*(
  p1[3] <- Prev_", Z1,"*(1-Se_", TestA, ")*Se_", TestB, "_", Z1," + (1-Prev_", Z1,")*Sp
  p1[4] <- Prev_", Z1,"*(1-Se_", TestA, ")*(1-Se_", TestB, "_", Z1,") + (1-Prev_", Z1,"


#=== PRIOR  ===#

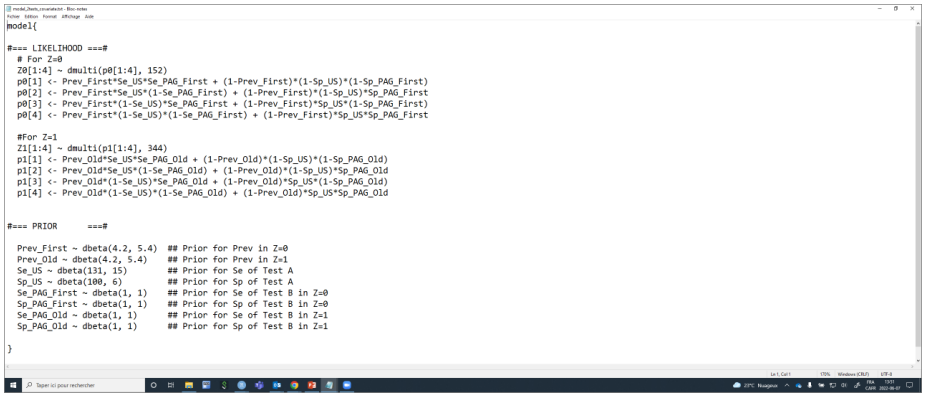
  Prev_", Z0," ~ dbeta(",Prev0.shapea,", ",Prev0.shapeb,")  ## Prior for Prev in Z=0
  Prev_", Z1," ~ dbeta(",Prev1.shapea,", ",Prev1.shapeb,")  ## Prior for Prev in Z=1
  Se_", TestA, " ~ dbeta(",Se.TestA.shapea,", ",Se.TestA.shapeb,")  ## Prior for Se of
  Sp_", TestA, " ~ dbeta(",Sp.TestA.shapea,", ",Sp.TestA.shapeb,")  ## Prior for Sp of
  Se_", TestB, "_", Z0, " ~ dbeta(",Se.TestB.Z0.shapea,", ",Se.TestB.Z0.shapeb,")   ##
  Sp_", TestB, "_", Z0, " ~ dbeta(",Sp.TestB.Z0.shapea,", ",Sp.TestB.Z0.shapeb,")   ##
  Se_", TestB, "_", Z1, " ~ dbeta(",Se.TestB.Z1.shapea,", ",Se.TestB.Z1.shapeb,")   ##
  Sp_", TestB, "_", Z1, " ~ dbeta(",Sp.TestB.Z1.shapea,", ",Sp.TestB.Z1.shapeb,")   ##

}")

#write as a text (.txt) file
write.table(model_2tests_covariate,
```

```
            file="model_2tests_covariate.txt",
            quote=FALSE,
            sep="",
            row.names=FALSE,
            col.names=FALSE)
```

With this code, you could, again, simply modify the labels for Test A and Test B, and for Z0 and Z1, and the shape parameters for the prior distributions. The text file with the JAGS model will automatically be updated. Currently, it looks like this:



```
model{

#=== LIKELIHOOD ===#
  # For Z=0
  Z0[1:4] ~ dmulti(p0[1:4], 152)
  p0[1] <- Prev_First*Se_US*Se_PAG_First + (1-Prev_First)*(1-Sp_US)*(1-Sp_PAG_First)
  p0[2] <- Prev_First*Se_US*(1-Se_PAG_First) + (1-Prev_First)*(1-Sp_US)*Sp_PAG_First
  p0[3] <- Prev_First*(1-Se_US)*Se_PAG_First + (1-Prev_First)*Sp_US*(1-Sp_PAG_First)
  p0[4] <- Prev_First*(1-Se_US)*(1-Se_PAG_First) + (1-Prev_First)*Sp_US*Sp_PAG_First

  #For Z=1
  Z1[1:4] ~ dmulti(p1[1:4], 344)
  p1[1] <- Prev_Old*Se_US*Se_PAG_Old + (1-Prev_Old)*(1-Sp_US)*(1-Sp_PAG_Old)
  p1[2] <- Prev_Old*Se_US*(1-Se_PAG_Old) + (1-Prev_Old)*(1-Sp_US)*Sp_PAG_Old
  p1[3] <- Prev_Old*(1-Se_US)*Se_PAG_Old + (1-Prev_Old)*Sp_US*(1-Sp_PAG_Old)
  p1[4] <- Prev_Old*(1-Se_US)*(1-Se_PAG_Old) + (1-Prev_Old)*Sp_US*Sp_PAG_Old


#=== PRIOR      ===#

  Prev_First ~ dbeta(4.2, 5.4)  ## Prior for Prev in Z=0
  Prev_Old ~ dbeta(4.2, 5.4)    ## Prior for Prev in Z=1
  Se_US ~ dbeta(131, 15)        ## Prior for Se of Test A
  Sp_US ~ dbeta(100, 6)         ## Prior for Sp of Test A
  Se_PAG_First ~ dbeta(1, 1)    ## Prior for Se of Test B in Z=0
  Sp_PAG_First ~ dbeta(1, 1)    ## Prior for Sp of Test B in Z=0
  Se_PAG_Old ~ dbeta(1, 1)      ## Prior for Se of Test B in Z=1
  Sp_PAG_Old ~ dbeta(1, 1)      ## Prior for Sp of Test B in Z=1

}
```

Figure 11.1: **Text file with the LCM for 2 independent diagnostic tests applied to one population but with one of the test accuracy varying as function of a covariate.**

Again, we will need to provide a list of initial values (one per Markov chain) for all unknown parameters. Careful, we now have two tests, but two Se and Sp for one of the test, and two prevalence.

```
#Initializing values for the parameters Prev, and the Ses and Sps of the two tests for the 3 cha
inits <- list(list(Prev_First=0.50,
                   Prev_Old=0.50,
                   Se_US=0.90,
                   Sp_US=0.90,
                   Se_PAG_First=0.90,
                   Sp_PAG_First=0.90,
                   Se_PAG_Old=0.90,
                   Sp_PAG_Old=0.90),

                   list(Prev_First=0.60,
                   Prev_Old=0.60,
```

```
                Se_US=0.80,
                Sp_US=0.80,
                Se_PAG_First=0.80,
                Sp_PAG_First=0.80,
                Se_PAG_Old=0.80,
                Sp_PAG_Old=0.80),

                list(Prev_First=0.40,
                Prev_Old=0.40,
                Se_US=0.70,
                Sp_US=0.70,
                Se_PAG_First=0.70,
                Sp_PAG_First=0.70,
                Se_PAG_Old=0.70,
                Sp_PAG_Old=0.70)
                )
```

We can run the model using `jags()` function as seen before.

```
library(R2jags)
library(coda)

#Run the Bayesian model
bug.out <- jags(data=datalist,
                model.file="model_2tests_covariate.txt",
                parameters.to.save=c("Prev_First", "Prev_Old","Se_US", "Sp_US", "Se_PAG
                n.chains=3,
                inits=inits,
                n.iter=11000,
                n.burnin=1000,
                n.thin=1,
                DIC=FALSE)
```

Then we could produce the diagnostic plots, compute the ESS, and print out our results as we did previously (results not shown).

```
library(mcmcplots)
bug.mcmc <- as.mcmc(bug.out)
mcmcplot(bug.mcmc, title="Diagnostic plots")
effectiveSize(bug.mcmc)
print(bug.out, digits.summary=3)
```

Chapter 12

Exercise 5 - Accuracy varying with covariate

12.1 Questions

Open up the R project for Exercise 5 (i.e., the R project file named *Exercise 5.Rproj*).

In the Exercise 5 folder, you will find partially completed R scripts. To answer the questions, try to complete the missing parts (they are highlighted by a **#TO COMPLETE#** comment). We also provided complete R scripts, but try to work it out on your own first!

Some anecdotic reports suggest that expression of PAG may vary as function of number of days of pregnancy. We could, therefore, hypothesize that PAG's Se and Sp may vary if the test is conducted early (35 days post-breeding) compared to later (>35 days post-breeding). When cross-tabulating the studies results for the PAG and US tests as function of days since breeding, we get:

Table. Cross-classified results from a PAG and US test as function of number of days since breeding.

35 days			**>35 days**		
	US+	**US-**		**US+**	**US-**
PAG+	143	11	**PAG+**	112	10
PAG-	2	136	**PAG-**	0	82

1.Start from the partially completed *Question 1.R* script located in Exercise 5 folder. Build a model for two diagnostic tests, one single population, but with Se and Sp of the PAG test varying as function of days since

breeding. Use the Romano et al. (2006) results, to inform the priors for the Se and Sp of a US exam (Se: 0.90, 95CI: 0.85, 0.95; Sp: 0.95, 95CI: 0.90, 0.97). Assume conditional independence between tests. How many degrees of freedom do you have and how many unknown parameters do you need to estimate? Based on your results, would you conclude that PAG accuracy is influenced by number of days since breeding?

12.2 Answers

1. How many degrees of freedom do you have and how many unknown parameters do you need to estimate? Based on your results, would you conclude that PAG accuracy is influenced by number of days since breeding?

Answer: We have 6 degrees of freedom and 8 unknown parameters to estimate (one Se and one Sp for US, 2 Se and 2 Sp for PAG, and two prevalence). We need two informative priors to make the LCM identifiable.

The results from this LCM are below.

Table. Results for the LCM allowing for varying accuracy of PAG as function of days since breeding.

Parameter	Median	95CI
Prevalence 28-35 days	0.521	0.462, 0.580
Prevalence 36-45 days	0.584	0.512, 0.653
Se US	0.928	0.895, 0.956
Sp US	0.977	0.957, 0.990
Se PAG 28-35 days	0.991	0.963, 1.00
Se PAG 36-45 days	0.994	0.966, 1.00
Sp PAG 28-35 days	0.980	0.927, 0.999
Sp PAG 36-45 days	0.962	0.879, 0.998

There was little difference between the 28-35 days since breeding *vs.* 36-45 days since breeding accuracy parameters. Below, we have plotted the two PAG Sp posterior distributions to illustrate how similar they are. However, since we are not in a Frequentist framework, "testing" whether the 28-35 days *vs.* 36-45 days accuracy parameters are statistically different is not straightforward. Note that Bayesian methods can be used to compute the probability that a given parameter is larger (or smaller) than another parameter, or than an estimate previously reported in the literature (see Arango-Sabogal et al., 2021 for an applied example.

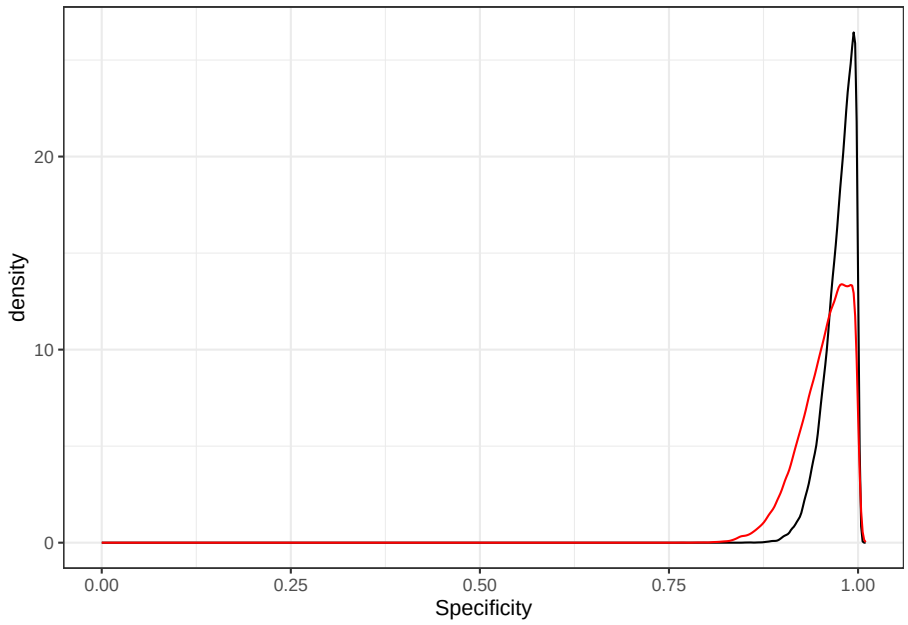


Figure 12.1: Posterior distributions for PAG specificity at 28-35 days (black) and 36-45 days (red) post-breeding.

Chapter 13

Extensions

In this document we have described some of the "main" LCM. More specifically we described LCMs that can be used with:

-**Two** conditionally **independent** diagnostic tests applied in **one single population**;
-**Two** conditionally **dependent** diagnostic tests applied in **one single population**;
-**Two** conditionally **independent** diagnostic tests applied in **more than one population** (using a conventional or hierarchical model);
-**Two** conditionally **dependent** diagnostic tests applied in **more than one population** (using a conventional or hierarchical model);
-**More than two** conditionally **independent** diagnostic tests applied in **one single population**;
-**More than two** diagnostic tests applied in **one single population** and with conditional **dependence** between two of the tests;
-**Two** conditionally **independent** diagnostic tests, but with **accuracy of one or both tests varying as function of a characteristic of the individual tested**;

From here, it is up to you to adapt these LCMs to fit your needs. For instance, you could have a study where three tests were applied to one population, but two of these tests were also applied to a second population. For two of the tests, you may want to relax the assumption of conditional dependence by including covariance terms between them. Finally, you may want to allow the Se and Sp of one of the test to vary as function of a characteristic of the individual tested.

When working with a "customized" LCM, the principles that we have discussed still apply. It will always be a good starting point to:

-Count the number of unknown parameters to estimate;
-Organize the dataset(s);
-Evaluate whether some tests combinations are not observed;

-Count the number of degrees of freedom available;
-Identify the number of informative priors that will be needed;
-Consider whether scientific literature will be available to develop the needed informative priors;

Based on this initial assessment, you will be able to decide whether running this LCM is doable, or if it would be better to revise your plans.

Chapter 14

Sensitivity analyses

An important step of any Bayesian analysis is to evaluate whether the results are strongly affected by the model's assumptions and/or the choice of priors.

14.1 Model's assumptions

Regarding the sensitivity of our results to the assumptions made, this could be assessed by comparing the results from the LCM where a given assumption was made *vs.* the ones from a LCM where the assumption was relaxed. Of course, **to fully "isolate" the effect of a given assumption, the same priors would have to be used in both LCM**. This is, however, not always possible since, sometimes, the "relaxed" LCM may require a larger number of informative priors to make it identifiable.

In the end, providing the results from an "alternative" LCM could help the readers understand what was the influence of the model chosen on the presented results. As author, you would probably want to use the LCM that is the most "biologically relevant" as your main model, but present the results of these alternative models to illustrate how the choice of model may have impacted the results.

Below are some of the main assumptions that you may want to evaluate.

14.1.1 Conditional independence between tests

Even if two tests rely on very different biological processes, it would be a good practice to evaluate whether allowing for conditional dependence between tests would produce different results. You could do this using the model described in Chapter 5.

14.1.2 Constant test accuracy across populations

To evaluate whether test's accuracy is constant across populations, the following options could be used:

- Check if the accuracy vary as function of a characteristic of the tested individuals (e.g., age, breed, etc);
- If a substantial number of populations were studied:
  - We could run the model after removing one of the population and repeat this process for each population. If the results (e.g., the test's accuracy parameters) differ substantially for one of the model, this could indicate that the test accuracy is different in this population;
  - We could allow for different accuracy in one or many of the populations. For instance, we may define *TestA* accuracy as *SeA-2* and *SpA-2* in one (or some) of the populations studied and as *SeA* and *SpA* for all other populations. If the *SeA* and *SeA-2* (or *SpA* and *SpA-2*) parameters differ substantially, this may, again, indicate that the test accuracy is different in this population.

14.1.3 Prevalence varies from one population to another

This latter assumption is important for the Hui and Walter (1980) model and other similar models. One thing that can be done is to "eyeball" the initial tests' results and/or the estimated prevalence to confirm some differences between populations in, respectively, the apparent and/or true prevalence.

Besides, Gustafson et al. (2005) have reviewed the impacts of having two populations with very similar prevalence and the only impact was a lack of identifiability of the model. This latter issue would results in non-converging Markov chains for one or a few of the unknown parameters. If the Markov chains converged, then the assumption of varying prevalence between population is probably not a problem.

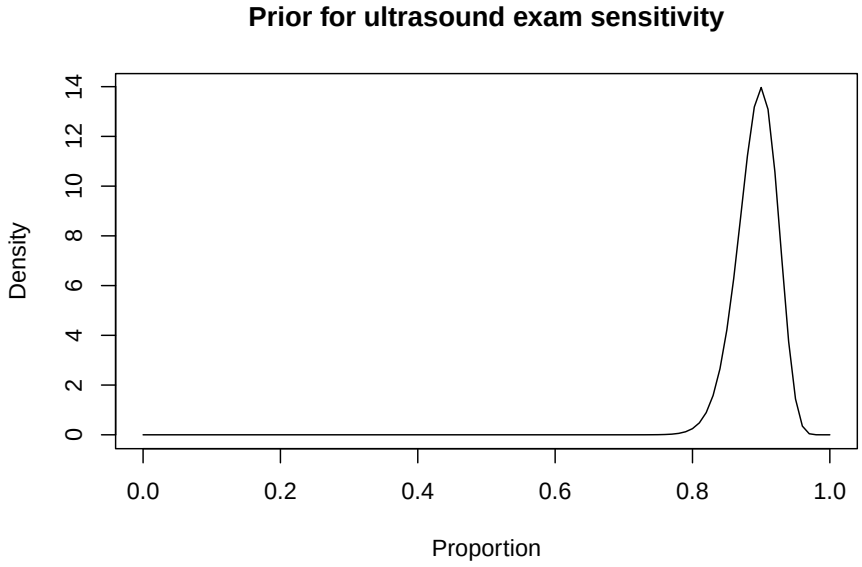
14.2 Choice of priors

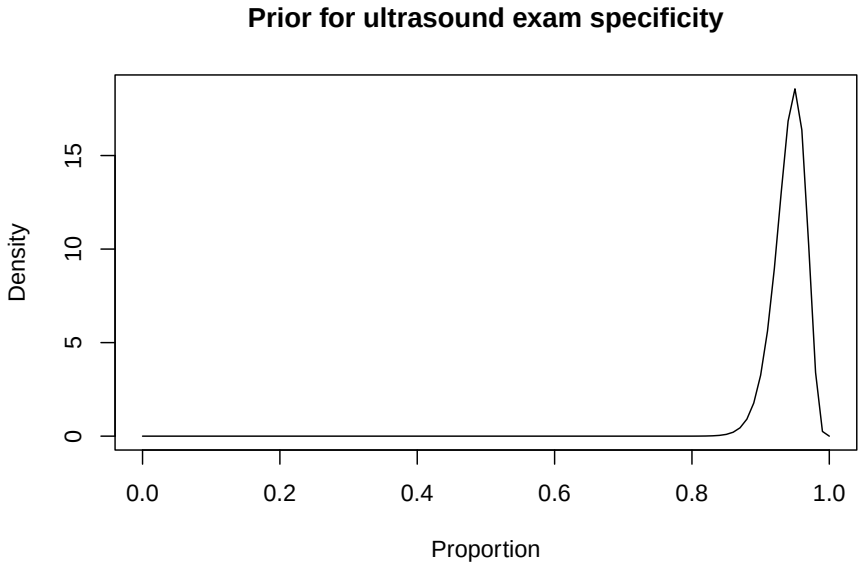
Whenever informative priors are used in a LCM, the sensitivity of the LCM to the choice of priors should be assessed. This can be done by running the same LCM, but with "perturbed" and usually "more diffuse" priors. Johnson et al. (2019) recommended, for instance, to move the mode of the prior distribution of a given parameter by a few percentage-points (e.g., -15 percentage-points) and to increase the width of its prior distribution. Then, the results from the LCM

using the scientific literature-derived priors can be compared to those of the LCM using perturbed priors to assess how sensitive to the choice of priors the results are. The amount and direction by which the mode will be moved and the level of increase of the width of the distribution will depend on the problem at hand.

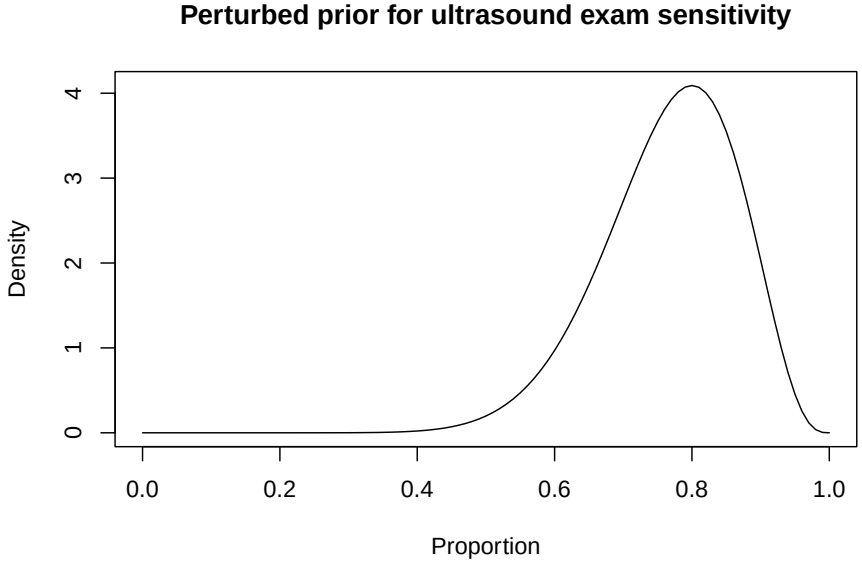
For instance, in the preceding sections we used the Romano et al. (2006) results, to inform the priors for the Se and Sp of a US exam conducted between 28-45 days post-insemination:
-Se = 0.90 (95CI: 0.85, 0.95)
-Sp = 0.95 (95CI: 0.90, 0.97)

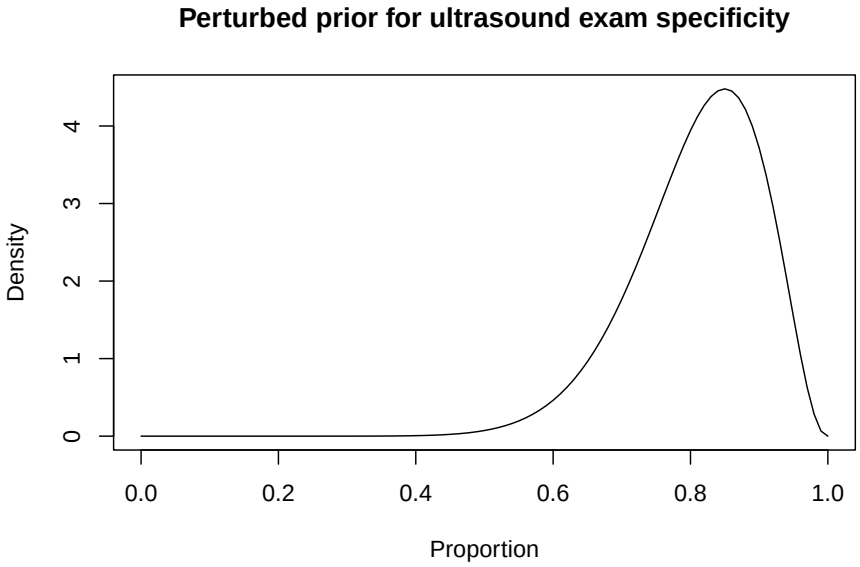
We thus came up with these prior distributions for US' Se and sp.





Thus, a possibility could be to move down the Se and Sp modes by 10 percentage points (i.e., mode for Se: 0.80 and mode for Sp: 0.85) and to increase the width of the distribution by moving the 2.5th percentiles by 30 percentage-points (Se 2.5th percentile: 0.55; Sp 2.5th percentile: 0.60). Thus:





Then, the results from the LCM with the informative and perturbed priors could be compared.

For instance, in exercise 3 we compared two conditionally independent diagnostic tests (PAG and US) in three populations (herd #1, herd #2, and herd #3). For that model, though the model is identifiable, we could use the Romano et al. (2006) informative priors on US' Se and Sp. Then, we could run the same LCM, but with the perturbed US' Se and Sp priors suggested above. If we do that, we would obtain the following results:

Table. Results from a 2 tests, 3 populations LCM using different priors (literature-based *vs.* perturbed informative priors on US' Se and Sp). The priors differing between LCM are indicated in bold.

Parameter	Romano et al. 2006 priors		Perturbed priors	
	Prior	**Posterior**	**Prior**	**Posterior**
Prev1	Beta(1.0, 1.0)	0.558 (0.496, 0.618)	Beta (1.0, 1.0)	0.556 (0.494, 0.617)
Prev2	Beta(1.0, 1.0)	0.523 (0.440, 0.604)	Beta (1.0, 1.0)	0.520 (0.435, 0.603)
Prev3	Beta(1.0, 1.0)	0.559 (0.452, 0.662)	Beta (1.0, 1.0)	0.557 (0.449, 0.657)

Parameter	Romano et al. 2006 priors		Perturbed priors	
Se US	**Beta (100, 12)**	0.925 (0.893, 0.954)	**Beta (13.6, 4.1)**	0.931 (0.890, 0.970)
Sp US	**Beta (100, 6.2)**	0.977 (0.956, 0.990)	**Beta (14.0, 3.3)**	0.981 (0.957, 0.994)
Se PAG	Beta (1.0, 1.0)	0.996 (0.981, 1.00)	Beta (1.0, 1.0)	0.996 (0.980, 1.00)
Sp PAG	Beta (1.0, 1.0)	0.981 (0.934, 0.999)	Beta (1.0, 1.0)	0.977 (0.922, 0.999)

In this specific example, we can see that the results are virtually unchanged. Thus, we could say that our initial analysis was quite robust to the choice of prior distributions. Personally, we would present the LCM with the Romano et al. (2006) priors as our "main" model. If we do have valid information from the literature about one or many of the unknown parameters, then why not using them?

Chapter 15

Sample size

When designing a study for estimating disease prevalence or diagnostic test accuracy we will want to evaluate the required sample size to achieve a certain precision in our test accuracy and disease prevalence estimates. In the following sections, we will first present, as a reminder, the method that can be used to estimate the sample size needed when a gold standard test is used for comparison. This method, however, is not appropriate when the test used for comparison is not a gold standard.

Then we will present a method specific for BLCM, which will provide a more accurate sample size estimation when imperfect tests are compared. This latter method will also be very helpful to quantify the impacts of choosing a given comparison test over another, or for choosing the prevalence that we should aim for in a second population of individuals.

15.1 Method when comparing to a gold standard

Remember, the main parameters we want to estimate are a disease prevalence and a sensitivity and specificity, and these are all proportions. More precisely:

- The prevalence is $\frac{\text{Number of diseased}}{\text{Number individuals}}$
- The sensitivity is $\frac{\text{Number of test+}}{\text{Number diseased}}$
- The specificity is $\frac{\text{Number of test-}}{\text{Number healthy}}$

For any of these fractions, increasing the denominator will result in a decrease in the width of the 95% confidence interval (i.e., a greater precision). To determine

the denominator (i.e., the sample size) that could generate a given width for the 95%CI, we could use the following, very simple Frequentist methods.

$n = p * (1 - p) * (\frac{1.96}{E})^2$

Where n is the required denominator, p is the expected proportion, E is the width of the 95%CI we would like to achieve (i.e., the precision), and 1.96 is the Z value corresponding to a type-I error rate of 0.05.

Alternatively, we could compute the width of the 95%CI as function of a given denominator.

$E = 1.96 * \sqrt{\frac{p*(1-p)}{n}}$

Therefore, in order to estimate the required sample size, we would first need to provide an educated guess regarding:
- The true prevalence of disease we will observe;
- The sensitivity and specificity of the test under investigation.

As an example, if we were to design a study:
- On a population where we think the disease prevalence will be around 30%;
- Using a test that we think will have a sensitivity of 85% and specificity of 95%;
- Where we would like to have a width of the 95%CI of 10 percentage-points (i.e., +/- 5% on each side of the median estimate).

We could use the following scripts to get an estimate of the required sample size.

```
#Sample size for the prevalence
p <- 0.30
E <- 0.10

n <- p*(1-p)*(1.96/E)^2
n
```

```
## [1] 80.6736
```

For the prevalence, we would need 81 individuals to obtain a 95%CI width of 10 percentage-points if the prevalence is around 30%.

Note that, to obtain a 95%CI width of 5 percentage-points:

```
#Sample size for the prevalence
p <- 0.30
E <- 0.05

n <- p*(1-p)*(1.96/E)^2
n
```

```
## [1] 322.6944
```

We would now need 323 individuals.

Regarding the sensitivity, remember that the denominator will be the truly diseased individuals, rather than all sampled individuals.

```
#Sample size for the sensitivity
p <- 0.85
E <- 0.10

n <- p*(1-p)*(1.96/E)^2
n
```

```
## [1] 48.9804
```

We would need 49 **diseased individuals** to obtain a 95%CI width of 10 percentage-points if the sensitivity is around 85%.

Regarding the specificity, again the denominator will be the truly healthy individuals, rather than all sampled individuals.

```
#Sample size for the specificity
p <- 0.95
E <- 0.10

n <- p*(1-p)*(1.96/E)^2
n
```

```
## [1] 18.2476
```

We would need 18 **healthy individuals** to obtain a 95%CI width of 10 percentage-points if the specificity is around 95%.

Now, we need to reunite these different estimates together. For instance, we saw that measuring 81 individuals was enough to get the precision we needed for the prevalence of disease. But, with 81 individuals and a prevalence of 30%, we would, theoretically, have 24 diseased (30% of 81) and 57 healthy (70% of 81) individuals. And we just computed that we needed 49 diseased individuals to get a precise estimate of the test's sensitivity.

In this case, we can see that the sensitivity is the limiting parameter. We could, thus, possibly recruit **a minimum** of 150 individuals and, hopefully, 50 will be diseased and 100 will be healthy. With that latter sample, we would possibly achieve the precision needed for the test's sensitivity, and we would exceed what we needed for the test's specificity and for disease prevalence. Of course this will work if our guess estimates were not too "off the mark".

This basic method, however, is not considering the loss of power that will occur when comparing imperfect tests with one another (as compared to comparing a novel test to a gold-standard test). This should, therefore, be considered as an overoptimistic scenario if the test used for comparison is not a gold standard.

15.2 Method for the Hui-Walter BLCM

Georgiadis et al. (2005) proposed a method to calculate sample size to estimate disease prevalence, sensitivity and specificity with a desired precision, when using the Hui and Walter (1980) model (two conditionally independent tests for screening individuals from two populations). In their article, Georgiadis et al. (2005) provided the hyperlink to an Excel spreadsheet template. We included this spreadsheet in the course material.

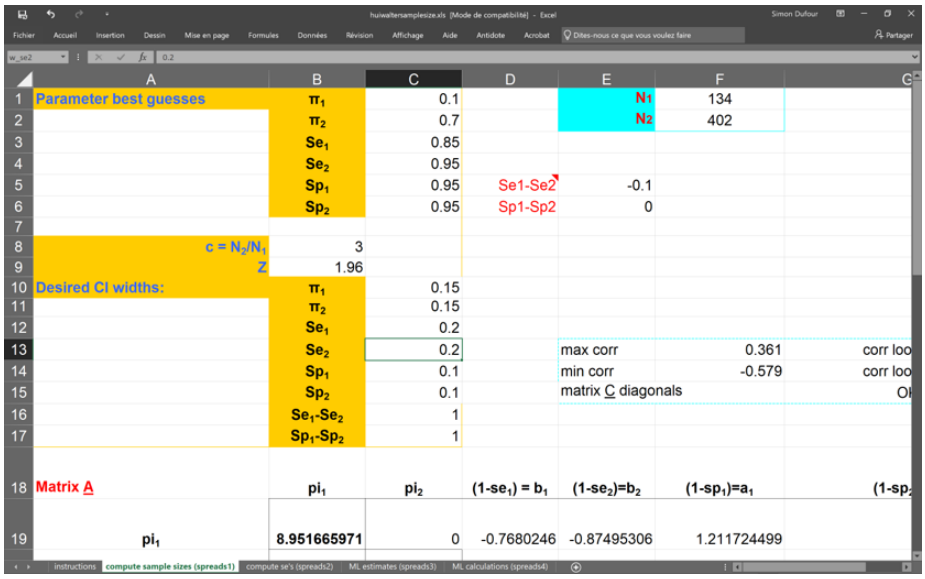
Briefly, we need to provide our best guess on:
- Disease prevalence in population 1 ($\pi 1$);
- Disease prevalence in population 2 ($\pi 2$);
- Sensitivity of the first ($Se1$) and second test ($Se2$);
- Specificity of the first ($Sp1$) and second test ($Sp2$).

And, then, the desired 95% BCI width for these different parameters.

Below is a screen capture of the spreadsheet. In this case, we proposed to study two populations where:
- Disease prevalence ($\pi 1$ and $\pi 2$) would be 0.10 and 0.70;
- Sensitivity of the two tests ($Se1$ and $Se2$) would be 0.85 and 0.95;
- Specificity of the two tests ($Sp1$ and $Sp2$) would both be 0.95;
- Desired width of the 95% BCI for disease prevalence ($\pi 1$ and $\pi 2$) would be 0.15;
- Desired width of the 95% BCI for sensitivity ($Se1$ and $Se2$) would be 0.20;
- Desired width of the 95% BCI for specificity ($Sp1$ and $Sp2$) would be 0.10;

Additional information are provided in the spreadsheet, but we can already see that we would need to sample 134 individuals from the low (10%) prevalence population (*N1*) and 402 from the high (70%) prevalence population (*N2*) to obtain the desired width for the 95% BCIs.

Again, this will work if our guess estimates were not too "off the mark". One additional note, when using the Georgiadis et al. (2005) method, we are not considering any informative priors that we may want to use. Typically, if we were to use informative priors on some or all of the unknown parameters the width of the 95% BCI will possibly be modified. For instance, if we provided relatively narrow priors for a given parameter, and if these priors agree closely with the observed data, then the 95% BCI could be more precise than anticipated (i.e.,



	A	B	C	D	E	F	G
1	Parameter best guesses	π_1	0.1		N1	134	
2		π_2	0.7		N2	402	
3		Se_1	0.85				
4		Se_2	0.95				
5		Sp_1	0.95	Se1-Se2	-0.1		
6		Sp_2	0.95	Sp1-Sp2	0		
7							
8	$c = N_2/N_1$	3					
9	Z	1.96					
10	Desired CI widths:	π_1	0.15				
11		π_2	0.15				
12		Se_1	0.2				
13		Se_2	0.2		max corr	0.361	corr loo
14		Sp_1	0.1		min corr	-0.579	corr loo
15		Sp_2	0.1		matrix C diagonals		Ol
16		Se_1-Se_2	1				
17		Sp_1-Sp_2	1				
18	Matrix A	pi_1	pi_2	$(1-se_1) = b_1$	$(1-se_2)=b_2$	$(1-sp_1)=a_1$	$(1-sp_2$
19	pi_1	8.951665971	0	-0.7680246	-0.87495306	1.211724499	

Figure 15.1: **Excel spreadsheet used to estimate sample size for BLCM comparing two independent tests in two populations.**

narrower). On the other hand, narrow priors which are not consistent with the observed data would possibly yield larger 95% BCI.

Nevertheless, this spreadsheet is quite handy to get rough sample size estimates. It is also quite useful to explore, for instance, how larger differences in disease prevalence between populations can improve the study power. It is also very interesting to investigate how the choice of the second test used for comparison will affect the study power.

As an example, in the following figure we illustrated, for the Hui and Walter (1980) model, how the **difference in prevalence between the two populations** would impact sample size. As you can see, for this specific scenario, targeting populations with more important prevalence differences could have a tremendous impact on power of the study. For instance, in this example, **with prevalence of 0.10 and 0.20 in the first and second population, a total of 14356 individuals would have to be tested to achieve the desired precision. On the other hand, with prevalence of 0.10 and 0.70 in the first and second population, only 218 individuals would have to be tested.**

Now, we could use two populations with expected disease prevalence of 0.10 and 0.50, but we are, this time, wondering what would be the best test to use for comparison so we can achieve the required precision with the smallest possible sample size. Below, we quantified how a change in the specificity of the second test would affect the require sample size. We see that, **if the test**

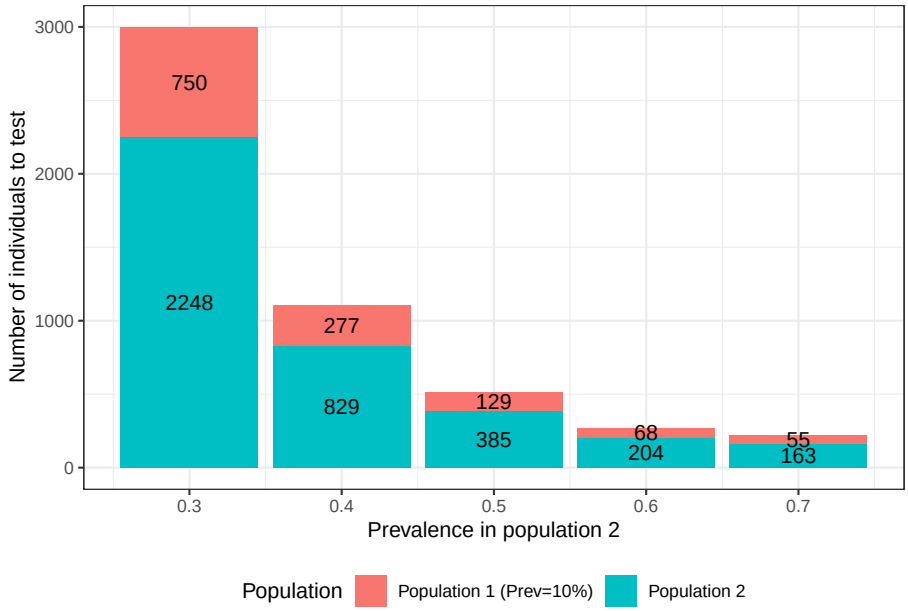


Figure 15.2: Number of individuals to test as function of the difference in prevalence between populations. For these calculations we assumed that: 1) the first population has a prevalence of disease of 0.10; 2) the two tests are expected to have Se and Sp of 0.90; 3) we are only interested in estimating the accuracy of the first test; and 4) we wish to obtain a 95 BCI width of <20 percentage-points (i.e., < +/- 10 percentage points) when reporting accuracy of the first test.

used for comparison has a specificity of 0.70, we will need to test 1664 individuals to achieve the desired precision. On the other hand, if this test specificity is 1.0 (i.e., no false positive results), only 218 individuals would have to be tested.

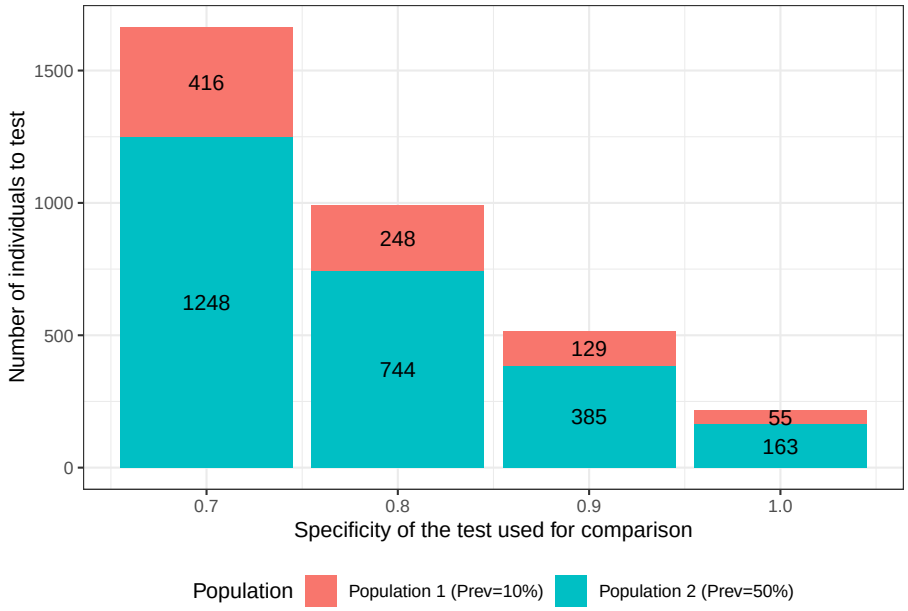


Figure 15.3: Number of individuals to test as function of specificity of the test used for comparison. For these calculations we assumed that: 1) the first and second population have disease prevalence of 0.10 and 0.50; 2) the test under investigation is expected to have a Se and Sp of 0.90; 3) we are only interested in estimating the accuracy of this first test; 4) we wish to obtain a 95 BCI width of <20 percentage-points (i.e., < +/- 10 percentage points) when reporting accuracy of the first test; and 5) the test used for comparison is expected to have a sensitivity of 0.90.

We could also quantify how a change in the sensitivity of the test used for comparison would affect the require sample size. We see that, **if the test has a sensitivity of 0.70, we will need to test 851 individuals to achieve the desired precision. On the other hand, if the test's sensitivity is 1.0 (i.e., no false negative results), only 420 individuals would have to be tested.**

For this specific scenario, we can see that there is a greater gain in study power, when improving the specificity of the test used for comparison *vs.* improving its sensitivity. Indeed, with prevalence of 0.10 and 0.50 in the two populations, we have more healthy individuals than diseased individuals. As a consequence, in this example, an increase in the specificity of one of the tests was a better

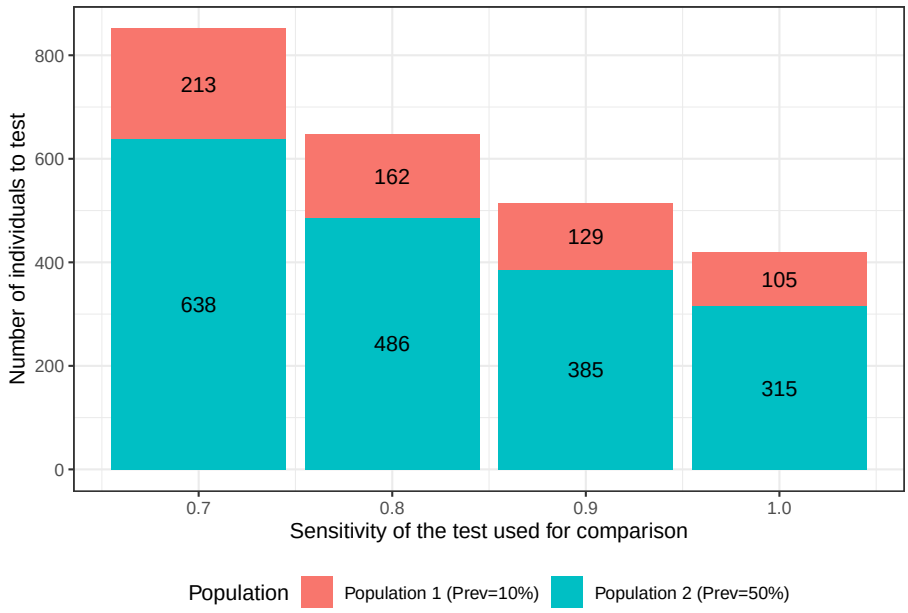


Figure 15.4: Number of individuals to test as function of sensitivity of the test used for comparison. For these calculations we assumed that: 1) the first and second population have disease prevalence of 0.10 and 0.50; 2) the test under investigation is expected to have a Se and Sp of 0.90; 3) we are only interested in estimating the accuracy of this first test; 4) we wish to obtain a 95 BCI width of <20 percentage-points (i.e., < +/- 10 percentage points) when reporting accuracy of the first test; and 5) the test used for comparison is expected to have a specificity of 0.90.

strategy to reduce misclassification issues than increasing its sensitivity. Finding a second test with the latter properties is often doable. For instance, for many diseases we have tests that can lead to the isolation of the microorganism and, if the microorganism is a strict pathogen, then such a test would be deemed to be 100% specific. As a comparison, using the same example, if we would have used a gold standard test for comparison, 196 individuals would have to be tested, as compared to 218 when using a second test with perfect specificity and a sensitivity of 0.90. Not a huge difference… And in many cases, another imperfect test would be cheaper than a gold standard test, and that is when a gold standard test is actually available (often it is not).

As we can see, conducting an *a priori* sample size estimation is extremely important when designing a diagnostic test validation study, since relatively minor decisions regarding the study design can greatly affect its power. One final note, if we are planning a study with more than two populations, or more than two tests, this spreadsheet could still be used to obtain a "worst-case scenario" sample size estimation (i.e., study power would mainly be increased, not decreased, by adding extra populations or tests).

Chapter 16

Exercise 6 - Sample size estimation and study design

16.1 Questions

For the following questions we will work with the Microsoft Excel sheet developed by Georgiadis et al. (2005). You will find this document in the folder named "Exercise 6 - Sample size estimation and study design" in the course materials.

1. Imagine that you are helping a research team for planning a study aiming at estimating the accuracy of a new test for detection of *Staphylococcus aureus* intra-mammary infections (**IMI**) using milk samples from dairy cattle.

- The diagnostic test development team is expecting that this new test will have a **sensitivity (Se) around 95%** and a **specificity (Sp) around 90%**.
- They would like to report accuracy of the new test with a **precision of +/- 5 percentage-points**, but they are not really interested in reporting the disease prevalence nor the second test's accuracy.
- In the literature, Dohoo et al. (2011) reported a **Se and Sp of, respectively, 90% and 100% for conventional bacteriological culture**.
- Moreover, a veterinary practitioner working with the test's development team indicated that, when sampling a population of cows without specific selection criteria, we could expect a *S. aureus* IMI **prevalence of 5%**.

- The practitioner also indicated that, **in a population of cows with persistently high somatic cells count (SCC), we would possibly have 20% of *S. aureus* IMI**.

How would you see the study design for this study and what would be the number of cows to test?

2. The diagnostic test development team was not too happy with the number of tests required by the darn epidemiologist (as usual). They told you that they have the budget for testing 400 cows and are asking what would be the precision in the accuracy parameters (Se and Sp) of the new test that they could achieve with such a sample size?

3. One of the person from the test's development team suggested to use for comparison, PCR instead of conventional bacteriological culture. Indeed, PCR as a Se of 99% and Sp of 99%, so quite superior to that of culture (90% and 100%). They are asking if, with 400 cows, they could now achieve a more precise estimation of the new test accuracy?

4. The research team is getting quite frustrated with your answers. The practitioner on the test's development team indicate that, when sampling cows that were positive to *S. aureus* in the past, we could expect that 50% of them would still be positive. They are asking if this could be a "better" second population in term of study power? When comparing the new test to bacteriological culture in a population of cows selected randomly and a population of cows that were positive to *S. aureus* in the past, what would be the precision in the accuracy parameters (Se and Sp) of the new test that could be obtained using 400 cows?

16.2 Answers

1. How would you see the study design for this study and what would be the number of cows to test?

Answer: We could imagine a validation test study where the new test and conventional milk bacteriological culture will be applied to samples coming from a number of cows in two populations:
1- Randomly chosen cows (anticipated prevalence of 5%)
2- Cows with persistently high SCC (anticipated prevalence of 20%)

Using this approach we would estimate that 1330 cows would be needed; 333 cows from the low prevalence population and 997 from the high prevalence population. This is, of course, assuming that the tests are conditionally independent.

2. What would be the precision in the accuracy parameters (Se and Sp) of the new test when testing 400 cows?

Answer: In the Excel sheet, we had to play with the desired CI width for the Se and Sp of the first test until we reached a total sample size of approximately 400

cows. Therefore, you may have obtained results that are slightly different than us. But, in our case, we were approaching 400 animals (exactly 411 animals, actually) when using CI width of 0.18 for Se and of 0.16 for Sp, thus **+/- 9 and +/- 8 percentage-points around the median Se and Sp estimates, respectively**.

3. What would be the precision of the new test accuracy when using PCR as comparison and with 400 cows?

Answer: If we use 99% for the second test's Se and Sp and play with the first test's Se and Sp desired CI width until we get around 400 cows to test, we find that, with 400 cows (403 to be precise), **we would report Se and Sp with precision of +/- 14 and +/- 8 percentage-points around the median estimates, respectively.** It's worst than before! As you can see, when disease prevalence is low, having a second test with a perfect Sp (i.e., bacteriological milk culture in this case) can have a tremendous impact!

4. What would be the precision of the new test accuracy when using a population of cow selected randomly and a population of cows that were positive to *S. aureus* in the past, and with a total of 400 cows tested?

Answer: With these parameters we would report Se and Sp with precision of **+/- 5 and +/- 7 percentage-points around the median estimates, respectively.** The test's development team is super happy with this; they now love epidemiologists. When you leave the room you hear them say:

